

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Северный (Арктический) федеральный университет имени М.В. Ломоносова»
Высшая школа информационных технологий и автоматизированных систем

Груздев Николай Александрович
Доронин Александр Дмитриевич
Локкин Иван Михайлович
Кустышев Сергей Сергеевич

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

09.03.01 Информатика и вычислительная техника

DevOps-инженер

Использование инфраструктуры как сервис для создания веб-приложения онлайн
школы «Мост Знаний» (в формате Стартап как диплом)

Утверждена приказом от «05» декабря 2026 г. № 3

Руководитель ВКР		Деменкова Е.А., доцент, к.т.н.
Консультанты		Тарасов А.П., старший преподаватель
Нормоконтроль		Деменкова Е.А., доцент, к.т.н.
Руководитель ОПОП		Тарасов А.П., старший преподаватель

Постановление ГЭК от «__» _____ 2026 г.

Признать, что обучающиеся выполнили и защитили ВКР с отметкой

ФИО	Отметка прописью
Н.А. Груздев	
А.Д. Доронин	
И.М. Локкин	
С.С. Кустышев	

Председатель ГЭК _____
Секретарь ГЭК _____

Архангельск 2026

ЗАДАНИЕ ДЛЯ ПОДГОТОВКИ ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЫ

09.03.01 Информатика и вычислительная техника

Тема ВКР: Использование инфраструктуры как сервис для создания веб-приложения
онлайн школы «Мост Знаний» (в формате «Стартап как диплом»)

Утверждена протоколом заседания кафедры от « ____ » _____ 20 ____ г. № ____

Обучающиеся:

Груздев Николай Александрович

Доронин Александр Дмитриевич

Локкин Иван Михайлович

Кустышев Сергей Сергеевич
(Ф.И.О.)

Курс:4

Группа: 151214

Срок сдачи выпускником законченной работы: «02» июня 2026 г.

Исходные данные к работе: производственной практике, отчет по производственной и преддипломной практике.

Основные разделы работы с указанием вопросов, подлежащих рассмотрению:

1. Анализ предприятия и постановка задачи: описание деятельности, организационная структура и бизнес-процессы, анализ оборудования, проблемы текущей организации разработки и требования к системе, постановка задач проекта, выводы по главе;
2. Обзор технологий и решений: анализ протоколов ВЧС, анализ инструментов автоматизации, анализ инструментов контейнеризации и оркестрации, анализ инструментов для повышения безопасности, обзор архитектуры ВЧС-сервиса, анализ подхода IaC, обзор современных практик DevOps в управлении ВЧС-сервисом;
3. Реализация проекта: автоматизация развертывания, мониторинг состояния ВЧС-сервиса;
4. Заключение: приложение А: листинг кода, приложение Б: финансовая часть, приложение В: паспорт проекта.

Перечень обязательных приложений к работе: полный листинг кода, таблицы финансового отчёта, паспорт проекта.

Перечень графического материала отсутствует.

Дата выдачи задания «26» ноября 2025 г.

Руководитель ВКР

(подпись)

Е.А. Деменкова

(инициалы, фамилия)

Задание принял к исполнению «26» ноября 2025 г.

Обучающиеся

(подпись)

Н.А. Груздев

А.Д. Доронин

И.М. Локкин

С.С. Кустышев

(инициалы, фамилия)

РЕФЕРАТ

Груздев Н.А. Доронин А.Д. Локкин И.М. Кустышев С.С. Использование инфраструктуры как сервис для создания веб-приложения онлайн школы «Мост Знаний».

Руководитель ВКР – Е.А Деменкова

Выпускная квалификационная работа объемом 146 с, содержит 21 таблицу, 5 графиков, 20 рисунков, 29 источников, 3 приложения.

Ключевые слова: стартап, облачная платформа, web-ресурс, ngrok, Docker, Github, Grafana, Yandex Cloud, PostgreSQL, Debian, Python.

Цель работы – разработка и обоснование архитектуры, а также реализация ключевых компонентов жизнеспособной унифицированной веб-платформы, направленной на интегрированное управление, развертывание и визуализацию распределённых IT-систем для веб- и мобильных приложений, с учетом потребностей стартап-команд.

Структура ВКР: состоит из введения, четырех глав, заключения, списка использованных источников и одного приложения.

В первой главе анализ конкурентов, обзор рынка, рейтинг онлайн школ, их слабые места, тренды, выводы.

Во второй главе проектирование архитектуры, требования, выбор технологий, проектирование подсистем и безопасности.

В третьей главе реализация системы, разработка бэкенда и фронтенда, развёртывание, тестирование, CI/CD.

В четвёртой главе бизнес-план стартапа, бизнес-идея, анализ рынка, маркетинг, финансы (NPV, PI, окупаемость), риски.

Н.А. Груздев

А.Д. Доронин

И.М. Локкин

С.С. Кустышев

ОГЛАВЛЕНИЕ

Нормативные ссылки	10
Определения, обозначения и сокращения	11
Введение	14
1 Анализ существующих конкурентов в сфере онлайн-школ	17
1.1 Текущее состояние рынка онлайн-образования в России.....	17
1.2 Рейтинг крупнейших игроков рынка.....	18
1.3 Детальный анализ ключевых конкурентов.....	19
1.3.1 Онлайн-школа «Умскул»	19
1.3.2 ГК Учи.ру и онлайн-школа «Тетрика».....	21
1.3.3 Онлайн-школа «Фоксфорд»	22
1.3.4 Другие значимые конкуренты.....	24
1.3.5 Сравнительная характеристика основных конкурентов	24
1.4 Анализ применения искусственного интеллекта в онлайн-образовании.....	26
1.5 Тренды и изменения в поведении потребителей	27
1.6 Выводы и стратегические возможности для нашего стартапа «Мост знаний»	28
2 Проектирование архитектуры и функциональности системы	30
2.1 Функциональные требования к системе	30
2.1.1 Требования к пользовательской подсистеме	30
2.1.2 Требования к подсистеме обучения	30
2.1.3 Требования к подсистеме искусственного интеллекта	31

2.1.4 Требования к подсистеме мониторинга и аналитики	31
2.2 Нефункциональные требования	31
2.3 Общая архитектура системы.....	32
2.4 Выбор технологического стека и его обоснование.....	32
2.4.1 Операционная система и среда виртуализации	33
2.4.2 Фронтенд: HTML, CSS, JavaScript	33
2.4.3 Бэкенд: Python и FastAPI	34
2.4.4 База данных: PostgreSQL	35
2.4.5 Искусственный интеллект: YandexGPT API.....	36
2.4.6 Проверка выполнения кода: Judge0 API	37
2.4.7 Туннелирование: Ngrok	38
2.4.8 Контейнеризация и оркестрация	38
2.4.9 Мониторинг и визуализация: Grafana	39
2.4.10 CI/CD: GitHub CI.....	39
2.5 Проектирование подсистем.....	40
2.5.1 Подсистема аутентификации и управления пользователями	40
2.5.2 Подсистема заданий и проверки решений	40
2.5.3 Подсистема персонализации и ИИ-ассистента.....	41
2.5.4 Подсистема мониторинга и аналитики	41
2.5.5 Подсистема развёртывания и инфраструктуры	42
2.6 Обеспечение безопасности.....	42
2.7 Масштабируемость и производительность	43
2.8 Выводы по главе	44
3. Реализация системы	46

3.1	Общая структура и компоненты системы.....	46
3.2	Реализация инфраструктуры и развёртывания	47
3.2.1	Подготовка виртуальной среды	47
3.2.2	Контейнеризация сервисов с помощью Docker	48
3.2.3	Оркестрация в Kubernetes	49
3.2.4	Временный публичный доступ через Ngrok	49
3.3	Реализация базы данных	54
3.3.1	Схема базы данных	54
3.3.2	Миграции и подключение	57
3.4	Реализация бэкенда на FastAPI.....	58
3.4.1	Организация кода	58
3.4.2	Ключевые эндпоинты	58
3.4.3	Обработка решений (логика)	60
3.4.4	Интеграция с YandexGPT	61
3.4.5	Интеграция с Judge0	62
3.5	Реализация фронтенда	62
3.5.1	Архитектура фронтенда.....	62
3.5.2	Редактор кода	65
3.5.3	Взаимодействие с бэкендом	66
3.5.4	Адаптивный дизайн	66
3.6	Реализация мониторинга и CI/CD	66
3.6.1	Мониторинг	66
3.6.2	CI/CD	67
3.7	Обеспечение безопасности.....	68

3.8 Выводы по главе	69
4. Бизнес-план.....	71
4.1 Паспорт проекта	71
4.2 Описание отрасли.....	72
4.2.2 Оценка внешней среды (PEST-анализ)	72
4.2.1 Оценка внутренней среды (SWOT анализ)	73
4.2.3 Анализ рынка.....	75
4.3 Маркетинговый план	77
4.3.1 Анализ конкурентов.....	77
4.3.2 Сегментирование рынка и анализ потребителей	82
4.3.3 Анализ способов продаж товаров и ценообразование	84
4.4 Организационный план	85
4.5 Производственный план	88
4.6 Финансовая модель проекта.....	90
4.7 Оценка рисков.....	95
Заключение.....	99
Список использованных источников.....	101
Приложение А.....	104
Приложение Б	135
Приложение В.....	139

НОРМАТИВНЫЕ ССЫЛКИ

В настоящей работе использованы ссылки на следующие нормативные документы:

ГОСТ 7.32-2017 Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления

ГОСТ 2.105-95 Единая система конструкторской документации. Общие требования к текстовым документам

Федеральный закон от 29.12.2012 № 273-ФЗ «Об образовании в Российской Федерации»

Федеральный закон от 27.07.2006 № 152-ФЗ «О персональных данных»

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

В настоящей работе применяются следующие термины с соответствующими определениями:

Адаптивное обучение – метод организации учебного процесса, при котором содержание и темп обучения подстраиваются под индивидуальные особенности, знания и умения конкретного ученика с использованием алгоритмов искусственного интеллекта.

Бэкенд (backend) – серверная часть веб-приложения, отвечающая за бизнес-логику, обработку запросов, взаимодействие с базой данных и внешними API.

Деплой (deploy) – процесс развёртывания программного обеспечения на целевом сервере или в облачной среде.

Искусственный интеллект (ИИ) – свойство интеллектуальных систем выполнять творческие функции, traditionally считающиеся прерогативой человека, в частности способность анализировать код и генерировать объяснения ошибок.

Контейнеризация – метод виртуализации, при котором приложение и все его зависимости упаковываются в изолированный контейнер, гарантирующий одинаковую работу в любой среде.

Микросервисная архитектура – подход к разработке программного обеспечения, при котором приложение строится как набор небольших независимых сервисов, взаимодействующих между собой по сети.

MVP (Minimum Viable Product) – минимально жизнеспособный продукт, версия продукта, обладающая минимальными, но достаточными для первых пользователей функциями.

Оркестрация контейнеров – процесс автоматического управления жизненным циклом контейнеров: их развёртыванием, масштабированием, обеспечением отказоустойчивости и сетевого взаимодействия.

Персонализированная траектория обучения – индивидуальный план изучения материала, формируемый автоматически на основе анализа сильных и слабых сторон ученика.

Промпт (prompt) – текстовый запрос, направляемый пользователем в нейросеть для получения ответа или выполнения задачи.

Сайт (веб-сайт) – совокупность веб-страниц, объединённых общей темой, дизайном и доменным именем, доступная в сети Интернет.

SaaS (Software as a Service) – модель распространения программного обеспечения, при которой продукт предоставляется пользователю в виде онлайн-сервиса по подписке.

Туннелирование – технология создания зашифрованного канала связи между локальным сервером и внешним клиентом через публичную сеть (например, интернет).

Фронтенд (frontend) – клиентская часть веб-приложения, отвечающая за отображение интерфейса и взаимодействие с пользователем.

CI/CD (Continuous Integration / Continuous Delivery) – практики непрерывной интеграции и непрерывной доставки, позволяющие автоматизировать сборку, тестирование и развёртывание кода.

API – Application Programming Interface (программный интерфейс приложения)

БД – База данных

ИИ – Искусственный интеллект

МРОТ – Минимальный размер оплаты труда

ООО – Общество с ограниченной ответственностью

ОС – Операционная система

РК – Районный коэффициент

СЗФО – Северо-Западный федеральный округ

СН – «Северные» надбавки

СФР – Социальный фонд России

УСН – Упрощённая система налогообложения

ФЗ – Федеральный закон

ФИПИ – Федеральный институт педагогических измерений

ФОТ – Фонд оплаты труда

HTTP – HyperText Transfer Protocol (протокол передачи гипертекста)

HTTPS – HyperText Transfer Protocol Secure (защищённый протокол передачи гипертекста)

IaC – Infrastructure as Code (инфраструктура как код)

JSON – JavaScript Object Notation (текстовый формат обмена данными)

JWT – JSON Web Token (токен для аутентификации)

K8s – Kubernetes (система оркестрации контейнеров)

NPV – Net Present Value (чистый приведённый доход)

PEST – Political, Economic, Social, Technological (анализ внешней среды)

PI – Profitability Index (индекс прибыльности)

RPS – Requests Per Second (количество запросов в секунду)

SaaS – Software as a Service (программное обеспечение как услуга)

SWOT – Strengths, Weaknesses, Opportunities, Threats (анализ сильных и слабых сторон, возможностей и угроз)

TAM – Total Addressable Market (общий объём рынка)

SAM – Serviceable Available Market (доступный объём рынка)

SOM – Serviceable Obtainable Market (реально достижимый объём рынка)

TLS – Transport Layer Security (протокол защиты транспортного уровня)

ВВЕДЕНИЕ

Сфера образования находится на пороге глубокой трансформации, вызванной внедрением технологий искусственного интеллекта. Традиционные образовательные модели, предполагающие единый темп и универсальную программу для всех учащихся, всё чаще демонстрируют свою неэффективность. Современные исследования в области когнитивной психологии подтверждают, что персонализация обучения способна не только повысить мотивацию, но и улучшить усвоение материала в среднем на 30–40% по сравнению с «уравнительным» подходом. В условиях цифровой экономики и перехода к индивидуализированным образовательным траекториям возникает острая потребность в платформах, способных адаптироваться под уровень знаний конкретного ученика.

Актуальность данной работы подтверждается как статистическими данными о росте рынка онлайн-образования, так и задачами, поставленными на государственном уровне. Согласно «Стратегии развития информационного общества в Российской Федерации на 2017–2030 годы», одним из приоритетных направлений является внедрение цифровых технологий и платформенных решений в сферу образования. Рынок онлайн-образования в России демонстрирует устойчивый рост (более 120 млрд рублей), при этом сегмент адаптивного обучения с использованием ИИ остаётся одним из самых быстрорастущих, но пока недостаточно заполненным качественными отечественными продуктами. В частности, существующие платформы часто предлагают статичный контент и не решают проблему индивидуальных пробелов в знаниях учащихся, особенно при подготовке к экзаменам (ОГЭ/ЕГЭ).

Однако в настоящее время большинство онлайн-школ и курсов строятся по линейному принципу: все ученики проходят одинаковые модули, невзирая на изначальный уровень подготовки. Это приводит к снижению мотивации у сильных учеников (из-за отсутствия вызова) и к накоплению «хвостов» у

слабых. Существующие аналоги (как российские, так и зарубежные) либо не обладают полноценной системой адаптивного ИИ, либо имеют высокую стоимость и не учитывают специфику российской школьной программы. Таким образом, актуальность работы обусловлена необходимостью разработки доступной SaaS-платформы на базе ИИ, которая закрывает потребность в персонализации обучения для широкой аудитории (школьники 12–18 лет и студенты).

Цель работы - разработка концепции и экономическое обоснование создания онлайн-платформы с искусственным интеллектом «Мост Знаний» для персонализированного обучения и подготовки к экзаменам.

Для достижения указанной цели были поставлены следующие задачи:

- проанализировать текущее состояние рынка онлайн-образования и адаптивных образовательных платформ (EduNet) в Российской Федерации;
- исследовать существующие аналоги и выявить их недостатки (сравнительный анализ платформ с ИИ и традиционных онлайн-школ);
- разработать концепцию SaaS-продукта, включающую архитектуру технологического стека;
- провести анализ целевой аудитории (школьники, студенты, родители, B2B-сегмент);
- разработать бизнес-модель и финансовый план стартап-проекта, рассчитать ключевые показатели эффективности;
- составить план маркетинговых мероприятий и проанализировать риски проекта.

Объектом исследования является деятельность онлайн-школ и образовательных платформ, направленная на подготовку учащихся к экзаменам и повышение успеваемости.

Предметом исследования является процесс персонализации обучения с использованием алгоритмов искусственного интеллекта и бизнес-модель IT-продукта для адаптивного образования.

Практическая значимость работы заключается в том, что разработанный IT-продукт «Мост Знаний» позволяет:

- автоматически выявлять слабые места ученика и строить индивидуальную траекторию обучения, повышая эффективность подготовки на 30%;
- экономить время преподавателей за счёт автоматической проверки типовых заданий и генерации рекомендаций;
- предоставлять образовательным учреждениям (школам, вузам) инструмент для цифровизации учебного процесса и аналитики успеваемости в режиме реального времени.

Результаты работы могут быть использованы для привлечения инвестиций в стартап, а также в качестве дорожной карты при разработке MVP-версии продукта с последующим выводом на рынок.

1 АНАЛИЗ СУЩЕСТВУЮЩИХ КОНКУРЕНТОВ В СФЕРЕ ОНЛАЙН-ШКОЛ

Сфера онлайн-подготовки к ЕГЭ и ОГЭ в России представляет собой динамично развивающуюся отрасль. Для успешного вывода на рынок нашего продукта необходимо провести глубокий анализ существующих конкурентов, их бизнес-моделей, сильных и слабых сторон, ценовых стратегий и технологических решений. Цель данной главы – провести исследование конкурентной среды, в которой предстоит работать проекту “Мост Знаний”, выявить ключевые пробелы в существующих решениях и на основе этого обосновать концепцию нашей платформы.

1.1 Текущее состояние рынка онлайн-образования в России

Российский рынок образовательных технологий достиг значительных масштабов к 2026 году. По данным аналитической компании Smart Ranking, суммарная выручка 100 крупнейших edtech-компаний России в 2025 году составила 154 млрд рублей - это на 12% больше, в прошлом году. Несмотря на продолжающийся рост, темпы расширения рынка постепенно замедляются: в 2023 году показатель увеличивался на 32%, в 2024-м - на 19%, а по итогам 2025 года рост составил всего 15%. Аналитики отмечают, что рынок онлайн-образования фактически переходит к стагнации, при этом более устойчивую динамику демонстрирует детский сегмент.

Одельного внимания заслуживает сегмент подготовки к ЕГЭ и ОГЭ, который становится все более актуальным для школьников-подростков. По данным Smart Ranking, во втором квартале 2025 года детский сегмент, а именно: подготовка к экзаменам, домашняя школа и дополнительное образование, занял почти 37% рынка, достигнув планки в 12,5 млрд рублей. Рост рынка школьного онлайн-образования в первом полугодии 2025 года составил 27%. Можем сделать вывод, что это самый быстрорастущий сегмент

EdTech. Доля детского сегмента на рынке составляет 29-34% (за 2024 и 2025 гг.). Это является вторым по величине показателем после дополнительного профессионального образования.

Примечательно, что онлайн-подготовка к ЕГЭ демонстрирует впечатляющие результаты. По оценкам, 29% максимальных результатов ЕГЭ в 2025 году получили ученики, которые готовились в онлайн-школах. У тех, кто учился в онлайн-школах, средний результат оказался заметно выше, чем по стране в целом: 73,86 против 58,5 среднего балла по всем предметам. При этом минимум 22% участников ЕГЭ, а если быть точными - почти 152 тысячи человек в 2025 году готовились в онлайн-школах.

1.2 Рейтинг крупнейших игроков рынка

Анализ рейтинга крупнейших edtech-компаний России по итогам III квартала 2025 года позволяет выделить ключевых игроков, оказывающих наибольшее влияние на рынок. Суммарная выручка ста лучших компаний составила почти 42 млрд рублей, а годовой прирост - 12,2%. Лидером рынка остаётся Корпорация «Синергия» с выручкой 16,075 млрд рублей, но за счёт традиционного скачка в сегменте высшего образования её отрыв от остальных игроков существенно увеличился.

В сегменте подготовки к ЕГЭ/ОГЭ и смежных направлениях топ-10 крупнейших компаний выглядит следующим образом:

- ГК Учи.ру (включая Учи.ру и Тетрику) - 7,2 млрд рублей
- Фоксфорд - 5,843 млрд рублей
- Умскул - 5,325 млрд рублей
- 100балльный репетитор - 4,368 млрд рублей
- MAXIMUM Education - 4,175 млрд рублей
- Сотка - 2,800 млрд рублей
- ЕГЭland - 2,155 млрд рублей
- Школково - 2,1 млрд рублей

- Онлайн-школа №1–2,047 млрд рублей
- 99 баллов - 668 млн рублей

Примечательно, что «100-балльный репетитор» вошёл в топ-10 компаний по темпам роста и показал увеличение выручки на 118,4% за счёт подготовки к ЕГЭ и ОГЭ.

Если говорить о результативности подготовки к ЕГЭ, то, лидерами по интегральному показателю качества обучения стали: «100балльный репетитор», «Тетрика» и NeoFamily. «Тетрика» демонстрирует наилучшие показания по статистике: средний балл - 82,3, доля результатов 80-100 баллов - 71,98%, удержание в течение учебного года - 93,3%. По абсолютному числу работ с максимальным количеством баллов (100) лидирует «100балльный репетитор» (985 учеников, сдавших на 100 баллов), «Умскул» заявляет о 825 “стобалльниках”, что значительно меньше.

1.3 Детальный анализ ключевых конкурентов

1.3.1 Онлайн-школа «Умскул»

«Умскул» является одним из бесспорных лидеров рынка подготовки к ЕГЭ и ОГЭ. Школа была основана в 2016 году и накопила значительный опыт, а также клиентскую базу. По состоянию на 2026 год у неё более 600 000 выпускников, больше 4 000 “стобалльников” и более 47 000 сдавших на 90 баллов и выше. По итогам 2022 года выручка компании достигла почти 3 млрд рублей. В 2023 году компания занимала первое место в рейтинге онлайн-школ для подготовки к ЕГЭ и ОГЭ по количеству выпускников. В октябре 2021 года 25% «Умскул» приобрёл холдинг VK, благодаря этому можем сделать вывод о высоком интересе крупного бизнеса к данному сегменту.

Что касается бизнес-модели и форматов обучения, то «Умскул» предлагает широкий спектр форматов обучения: живые вебинары, кураторская поддержка с ответом в течение 5-30 минут, проверка каждой домашней работы,

ежемесячные пробные экзамены. Школа позиционируется как массовый продукт с упором на энергичную подачу материала и преподавателей-звёзд, которые становятся узнаваемыми брендами среди старшеклассников (Аделия Адамова, Артур Шарафиев, Александр Долгих, Татьяна Граева и др.). Данный подход эффективно работает на подростковую аудиторию. Как результат - повышается вовлеченность и мотивация.

Если говорить о ценовой политике, то ценовая модель «Умскул» вызывает определённые нарекания у потребителей. Базовая цена «от 4 890 Р» на главной странице сайта является стартовой ценой предбанника или раннего бронирования. Можно найти информацию, что реальные годовые курсы стоят 25-70 тыс. рублей за предмет в тарифах «Стандарт» и «Про», а тариф «Премиум» может достигать 80-150 тыс. рублей. Такая ценовая политика может вызывать негативную реакцию при общении с менеджерами по продажам, потому что создает эффект «занижения» ожидаемой клиентом цены.

Рассмотрим технологические инновации: «Умскул» активно внедряет технологические решения, включая платформу с искусственным интеллектом, которая подбирает дополнительные материалы по слабым темам ученика. Конкурентным преимуществом при взаимодействии с родителями является наличие лицензии на образовательную деятельность, которая позволяет школе принимать материнский капитал и предоставлять налоговый вычет 13%.

Рассмотрим качество подготовки к экзаменам по информатике: согласно обзорам и отзывам, качество курсов «Умскул» неоднородно по предметам. Сильными сторонами школы являются русский язык, биология, химия, английский язык и обществознание, но подготовка по информатике оценивается как более слабая - отмечается недостаток задач реального уровня сложности, что создаёт потенциальное конкурентное преимущество для нашего проекта «Мост знаний», который специализируется именно на информатике.

К недостаткам «Умскул» можно отнести: агрессивные продажи и навязчивость менеджеров (особенно при переходе между тарифами или возвратах), автопродление подписки, которое требует немедленного отключения, а также неоднородное качество курсов по разным предметам. Гарантия сдачи экзамена более, чем на 80 баллов, формально предоставляемая школой, обвязана множеством условий - все домашние работы должны быть сданы, все вебинары просмотрены, все пробники написаны. Это делает гарантию страховкой скорее для дисциплинированного ученика.

1.3.2 ГК Учи.ру и онлайн-школа «Тетрика»

Группа компаний Учи.ру, объединяющая одноимённую платформу и школу «Тетрика», занимает второе место в рейтинге крупнейших edtech-компаний с выручкой 7,200 млрд рублей. «Тетрика» специализируется на индивидуальных занятиях с репетиторами в онлайн-формате, что принципиально отличает её от «Умскул», где используются массовые вебинары.

Рассмотрим их бизнес-модель, а также форматы обучения. «Тетрика» предлагает индивидуальные онлайн-занятия с репетиторами, которые проходят на удобной интерактивной платформе с виртуальной доской. Ключевое преимущество этой модели - возможность замены преподавателя без доплат (достаточно обращения к куратору, который подбирает другого специалиста за пару дней). Можно выделить то, что школа проводит бесплатные вебинары, которые открыты для всех желающих и могут служить каналом привлечения новых учеников.

Если говорить про ценовую политику, то стоимость занятий в «Тетрике» оценивается пользователями как «чуть выше среднего», однако школа активно использует систему скидок и акций для привлечения клиентов. Например, действуют промокоды на скидку 10% при первой оплате от 8 уроков, а также акции «2 занятия в подарок» при покупке пакета от 8 уроков. Также

предоставляются скидки до 20% при покупке пакета уроков для подписчиков новостной рассылки.

Говоря про технологические инновации можно отметить, что «Тетрика» позиционируется как «инновационная платформа, которая предлагает уникальный подход к образованию в цифровую эпоху». Школа предоставляет бесплатный пробный урок и тест на определение уровня знаний, интерактивную доску, быструю проверку домашних заданий и развитую систему кураторской поддержки. Наличие бесплатной библиотеки с материалами по всем предметам и пробными вариантами ЕГЭ/ОГЭ также является привлекательным для пользователей бонусом.

Что касается результативности, то «Тетрика» лидирует по доле учеников с высокими баллами с результатами в диапазоне 80-100 баллов - 71,98%, а также по удержанию учеников - 93,3% продолжали занятия из месяца в месяц в течение учебного года. Средний балл учеников «Тетрики» составляет 82,3, что значительно выше среднего показателя по стране.

Перейдем к слабым сторонам - основной недостаток «Тетрики» - высокая стоимость индивидуальных занятий. Большие цены могут быть недоступны для семей с низким уровнем дохода, например в регионах. Кроме того, модель полностью зависит от человеческого фактора - качества конкретного репетитора, его коммуникативных навыков и профессиональной компетенции.

1.3.3 Онлайн-школа «Фоксфорд»

«Фоксфорд» занимает третье место в рейтинге крупнейших компаний с выручкой 5,843 млрд рублей, она является одной из старейших и наиболее диверсифицированных онлайн-школ в России.

«Фоксфорд» предлагает множество форматов обучения: домашняя школа, курсы, мини-группы, занятия с репетитором, подготовка к ОГЭ и ЕГЭ, школа программирования и языковые направления. Такая диверсификация

позволяет школе охватывать широкую аудиторию и снижать риски. Платформа ценится родителями за гибкость - возможность учиться из любой точки, выбирать преподавателей, пересматривать занятия в записи и не выпадать из программы. У Фоксфорда есть прикрепление к московской школе и онлайн-аттестация, что позволяет ученикам получать официальный документ об образовании.

Далее перейдем к ценовой политике: стоимость обучения в «Фоксфорде» варьируется в зависимости от формата. Например, курс подготовки к ОГЭ может стоить около 31 тысячи рублей (со скидкой). При этом в отзывах отмечается, что цена воспринимается как «приемлемая» при наличии скидки.

Что касается технологических решений, то «Фоксфорд» предоставляет автоматическую проверку домашних заданий и пробные ЕГЭ. Платформа работает стабильно, без значительных технических проблем, что особенно важно для детей - отсутствие отвлечений на технические неполадки высоко ценится пользователями. Преподаватели «Фоксфорда» получают положительные отзывы за умение объяснять «человеческим языком» и приводить наглядные примеры.

По доле высоких результатов (80-100 баллов) «Фоксфорд» показывает 42%, уступая «Тетрике» (72%) и NeoFamily (41%). В опросах пользователей «Фоксфорд» упоминается наравне с «Школково» и «Соткой» как одна из школ, где готовятся к ЕГЭ, что подтверждает его сильные позиции в сегменте.

Среди недостатков можно отметить: необходимость приезда в Москву для сдачи ОГЭ/ЕГЭ при выборе формата с прикреплением к московской школе, хотя есть возможность прикрепиться к школе по месту жительства только для сдачи экзаменов. Кроме того, широкая диверсификация может приводить к размыванию фокуса и снижению качества по отдельным направлениям.

1.3.4 Другие значимые конкуренты

Рассмотрим такую платформу как «100балльный репетитор». Эта школа демонстрирует впечатляющие темпы роста (выручка увеличилась на 118,4% в 2025 году) и лидирует по абсолютному числу учеников с максимальными баллами - 985 человек. Школа ориентирована на массовую подготовку к ЕГЭ и ОГЭ - они предлагают вебинары и курсы по всем предметам.

«Школково» и «Сотка». Эти школы занимают 8-е и 6-е места в рейтинге крупнейших компаний с выручкой 2,1 млрд и 2,8 млрд рублей соответственно. Они являются заметными игроками на рынке и часто упоминаются в опросах пользователей наряду с «Фоксфордом» и «Умскул».

«NeoFamily». Эта школа входит в тройку лидеров по интегральному рейтингу результативности наряду с «100балльным репетитором» и «Тетрикой». По удержанию учеников NeoFamily показывает отличные результаты - 90%.

MAXIMUM Education занимает 5-е место с выручкой 4,175 млрд рублей и является значимым конкурентом в сегменте подготовки к экзаменам.

1.3.5 Сравнительная характеристика основных конкурентов

Для наглядного сравнения ключевых конкурентов представим основные параметры в таблице 1.1.

Таблица 1.1 – Сравнительная характеристика конкурентов

Параметр	Умскул	Тетрика	Фоксфорд
Выручка	5,325 млрд руб.	в составе ГК - 7,200 млрд руб.	5,843 млрд руб.

Окончание таблицы 1.1

Формат обучения	Массовые вебинары и кураторы	Индивидуальные занятия с репетитором	Диверсифицированный (домашняя школа, курсы, репетиторы)
Ценовой диапазон	25-150 тыс. руб./год (за предмет)	Выше среднего (со скидками и акциями)	~31 тыс. руб./курс (со скидкой)
Средний балл учеников	н/д	82,3	42% высокобалльников (80-100)
Удержание (retention)	н/д	93,3%	н/д
Технологии ИИ	Платформа с ИИ для подбора материалов по слабым темам	Инновационная платформа, автоматическая проверка ДЗ	Автоматическая проверка ДЗ и пробных ЕГЭ
Сильные стороны	Преподаватели-звёзды, массовый охват, господдержка (маткапитал, налоговый вычет)	Индивидуальный подход, возможность замены преподавателя, высокие результаты	Гибкость форматов, стабильность платформы, официальная аттестация
Слабые стороны	Ценовая политика (эффект занижения), агрессивные продажи, неоднородное качество по предметам (информатика слабее)	Высокая стоимость, зависимость от человеческого фактора	Необходимость приезда в Москву для сдачи экзаменов

1.4 Анализ применения искусственного интеллекта в онлайн-образовании

Внедрение искусственного интеллекта в образовательный процесс становится одним из ключевых трендов развития рынка EdTech. Согласно прогнозам, российский рынок онлайн-образования в сфере ИИ к концу 2025 года может достичь 5,6 млрд рублей, что свидетельствует о высоком интересе к данной технологии. Самыми развивающимися остаются направления, связанные с ИИ. Компании отчитываются о росте спроса пользователей на курсы по искусственному интеллекту, а также о внедрении ИИ-решений в свои образовательные продукты.

Рассмотрим анализ текущего применения ИИ конкурентами. Умскул уже внедрил ИИ-платформу, которая анализирует успеваемость ученика и подбирает дополнительные материалы по слабым темам. Однако это решение является скорее рекомендательной системой, чем полноценным ИИ-тьютором, способным проверять код и объяснять конкретные ошибки. Тетрика и Фоксфорд используют автоматическую проверку домашних заданий, однако эта проверка, как правило, ограничена тестовой частью и не охватывает задания с развёрнутым ответом (особенно по информатике и программированию).

Глубинная проверка кода - отсутствующая функция. Анализ конкурентной среды показывает, что ни один из крупных игроков не предлагает полноценного ИИ-инструмента для проверки решений по информатике с подробным анализом ошибок в коде и персональными рекомендациями по их исправлению. Умскул, например, по отзывам пользователей, имеет более слабую подготовку по информатике по сравнению с другими предметами. Это создаёт значительное конкурентное преимущество для проекта «Мост знаний», специализирующегося именно на этом предмете с использованием передовых ИИ-решений.

1.5 Тренды и изменения в поведении потребителей

Анализ рынка позволяет выделить несколько ключевых трендов, которые необходимо учитывать при разработке стратегии конкурентной борьбы.

Школы отмечают рост интереса к раннему началу подготовки к ЕГЭ - многие начинают готовиться задолго до экзаменов, некоторые даже с 6-7 классов. Это расширяет потенциальную аудиторию за счёт учеников 8-10 классов.

Родители стали внимательнее подходить к выбору образовательных услуг, предпочитая решения с гарантированным качеством и ощутимым результатом. Простых обещаний становится недостаточно - необходимы прозрачные метрики эффективности.

Наблюдается устойчивый рост спроса на подготовку по точным и естественным наукам (математика, физика, информатика, химия, биология). Цифровизация российского образования достигла переломного момента: онлайн-курсы по этим предметам перестали быть дополнительной опцией и превратились в ключевой инструмент подготовки старшеклассников.

Конкуренция за бюджетные места в вузах подталкивает выпускников к усиленной подготовке, а доступная цена онлайн-школ делает их привлекательной альтернативой классическим репетиторам.

Чтобы повысить уровень вовлечённости студентов, платформы внедряют различные инструменты: геймификацию и призы, усиление кураторской поддержки, ставку на «блогерский вайб» и харизму преподавателей, развитие ИИ-тренажёров и других технологичных решений.

1.6 Выводы и стратегические возможности для нашего стартапа «Мост знаний»

Проведённый анализ конкурентной среды позволяет сформулировать следующие ключевые выводы:

Рынок находится в фазе насыщения - темпы роста замедляются с 32% до 15-16%, что означает переход к борьбе за долю рынка и повышению требований к качеству продукта. Новому игроку будет сложно конкурировать с устоявшимися брендами за счёт простого копирования их модели.

Основные конкуренты имеют слабости в подготовке по информатике. Умскул, лидер рынка, по отзывам имеет более слабую подготовку по информатике по сравнению с другими предметами, а индивидуальные занятия в Тетрике дороги и не масштабируются.

ИИ-технологии в проверке кода и персонализации обучения остаются нереализованным потенциалом. Ни один из крупных конкурентов не предлагает полноценного ИИ-тьютора для проверки решений по информатике с анализом ошибок в коде. Максимум, что есть - автоматическая проверка тестовой части и рекомендательные системы.

Потребители становятся более требовательными и ценят реальные результаты. Родители и ученики хотят видеть прозрачные метрики прогресса и индивидуальный подход. Простые видеолекции и массовые вебинары уже не удовлетворяют запросы аудитории.

Растёт спрос на подготовку по информатике и другим техническим дисциплинам. Этот тренд создаёт благоприятную рыночную конъюнктуру для проекта, специализирующегося именно на информатике.

Рассмотрим стратегические возможности для проекта «Мост знаний». Мы можем занять нишу персонализированной подготовки по информатике с использованием ИИ, предлагая продукт, которого нет у крупных конкурентов. Сделать ставку на массовость и доступность (1500-2000 Р/мес) в противовес

дорогим индивидуальным занятиям Тетрики и неоднородному качеству массовых курсов Умскул. Также, можем использовать технологическое преимущество (ИИ-проверка кода) как основной элемент дифференциации и маркетингового позиционирования. Мы можем предложить гибридную модель - ИИ + редкие живые консультации для сложных кейсов, что позволит снять страх «робот ошибается» и объединить преимущества технологий и человеческого участия. Сфокусируемся на информатике как на узкой специализации, добиваясь экспертного уровня качества именно по этому предмету, в отличие от широкопрофильных конкурентов, которые вынуждены поддерживать качество по всем предметам одновременно.

Таким образом, мы можем заявить, что анализ конкурентов подтверждает актуальность и перспективность нашего стартапа, выявляя чёткое конкурентное преимущество в виде ИИ-тьютора для помощи ученикам при отсутствии подобного решения у крупных рыночных игроков.

2 ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ И ФУНКЦИОНАЛЬНОСТИ СИСТЕМЫ

Разработка онлайн-школы «Мост знаний» представляет собой комплексную задачу, которая требует тщательного проектирования всех компонентов системы: от пользовательского интерфейса до инфраструктурного слоя. В рамках второй главы рассматривается архитектура и функциональность разрабатываемой системы, обосновывается выбор технологического стека, описывается взаимодействие компонентов и приводятся детали реализации ключевых подсистем. Особое внимание уделяется интеграции искусственного интеллекта на базе YandexGPT Lite 5, развёртыванию инфраструктуры с использованием инструментов DevOps, а также обеспечению масштабируемости и отказоустойчивости платформы.

2.1 Функциональные требования к системе

На основе анализа целевой аудитории, конкурентной среды и бизнес-задач проекта были сформулированы следующие функциональные требования к онлайн-школе «Мост знаний».

2.1.1 Требования к пользовательской подсистеме

Система должна обеспечивать регистрацию и аутентификацию пользователей с разделением ролей: ученик, администратор, преподаватель. Для каждого пользователя должен вестись персональный профиль, содержащий информацию об успеваемости, прогрессе по темам и истории решений.

2.1.2 Требования к подсистеме обучения

Основной функциональностью системы является предоставление заданий ОГЭ по информатике с возможностью их решения во встроенном

редакторе кода. Задания должны охватывать все типы задач, встречающиеся на ОГЭ, включая задания с кратким ответом и задания с развёрнутым ответом, требующие написания программного кода. Система должна обеспечивать автоматическую проверку решений с мгновенной обратной связью.

2.1.3 Требования к подсистеме искусственного интеллекта

Ключевым требованием является наличие ИИ-ассистента, способного анализировать код ученика, выявлять ошибки, генерировать понятные объяснения и давать рекомендации по исправлению. ИИ также должен обеспечивать персонализированный подбор заданий на основе истории ошибок и текущего уровня подготовки ученика.

2.1.4 Требования к подсистеме мониторинга и аналитики

Система должна собирать метрики производительности (время ответа ИИ, количество активных пользователей, нагрузка на серверы) и образовательной аналитики (прогресс учеников, типовые ошибки, retention). Для администраторов должна быть предусмотрена панель мониторинга с визуализацией ключевых показателей.

2.2 Нефункциональные требования

Помимо функциональных требований, рассмотрим некоторые нефункциональные характеристики системы.

Система должна выдерживать одновременную работу до 1000 пользователей без деградации производительности. Архитектура должна предусматривать горизонтальное масштабирование компонентов при росте нагрузки (особенно в период активной подготовки к ЕГЭ - апрель-май).

Платформа должна обеспечивать доступность не менее 99,5% времени. Отказ одного из компонентов не должен приводить к полной недоступности системы.

Что касается безопасности, то система должна обеспечивать защиту персональных данных пользователей в соответствии с требованиями Федерального закона №152-ФЗ «О персональных данных».

Среднее время проверки одного задания ИИ не должно превышать 5-15 секунд. Загрузка страниц интерфейса должна составлять не более 3-5 секунд.

Интерфейс должен быть интуитивно понятным для школьников 16-18 лет и обеспечивать быстрый доступ ко всем ключевым функциям.

2.3 Общая архитектура системы

Разработанная архитектура онлайн-школы «Мост знаний» представляет собой многоуровневую систему, построенную по принципу клиент-сервер с чётким разделением компонентов. Архитектура включает в себя следующие основные слои:

Клиентский слой - веб-интерфейс, доступный через браузер пользователя, который реализован с использованием HTML, CSS и JavaScript.

Слой приложений - бэкенд-сервисы, реализующие бизнес-логику платформы, включая API для взаимодействия с клиентским приложением. Реализован на Python с использованием фреймворка FastAPI.

Слой данных - система хранения информации о пользователях, заданиях, прогрессе и результатах, которая была реализована на PostgreSQL.

Слой искусственного интеллекта - компоненты, обеспечивающие работу ИИ-ассистента: интеграция с YandexGPT API и система проверки кода.

Инфраструктурный слой - инструменты развёртывания, оркестрации и мониторинга: Docker, Kubernetes, Grafana.

2.4 Выбор технологического стека и его обоснование

При выборе технологий для реализации проекта учитывались следующие критерии: соответствие функциональным требованиям, масштабируемость, стоимость владения, наличие сообщества и документации,

а также совместимость с выбранным инфраструктурным окружением (Debian 13). Ниже представлено обоснование выбора каждого компонента.

2.4.1 Операционная система и среда виртуализации

В качестве базовой платформы для развёртывания всех компонентов системы выбрана операционная система Debian 13 (Trixie), установленная на виртуальной машине под управлением Oracle VM VirtualBox.

Обоснование выбора Debian 13:

Debian является одним из наиболее стабильных и надёжных дистрибутивов Linux, широко используемым для серверных развёртываний. Debian 13 (Trixie) базируется на ядре Linux 6.12, которое обеспечивает поддержку современных драйверов и включает актуальные патчи безопасности. Наличие обширных репозиториях пакетов упрощает установку необходимого программного обеспечения (Docker, PostgreSQL, Nginx и др.). Бесплатное распространение и открытый исходный код снижают стоимость владения проектом на начальном этапе.

Перейдем к обоснованию выбора Virtual Box:

Virtual Box предоставляет бесплатную среду виртуализации, позволяющую изолировать среду разработки и тестирования от основной операционной системы. Удобство создания резервных копий (снимков состояния) виртуальной машины упрощает экспериментирование с конфигурацией и восстановление после ошибок. Поддержка сетевых мостов и NAT позволяет настраивать доступ к виртуальной машине как из локальной сети, так и через интернет.

2.4.2 Фронтенд: HTML, CSS, JavaScript

Для реализации пользовательского интерфейса используются стандартные веб-технологии: HTML для структуры страниц, CSS для стилизации и адаптивной вёрстки, а также чистый JavaScript (без

использования тяжёлых фреймворков) для реализации динамического поведения.

Выбор был обоснован отсутствием зависимости от крупных фреймворков (React, Vue, Angular), что ускоряет начальную разработку и снижает порог входа для поддержки проекта.

Нативная поддержка современных возможностей браузеров (Fetch API, WebSocket) достаточна для реализации всех требуемых функций (отправка решений, получение результатов от ИИ, обновление статистики).

Лёгкость интеграции с бэкенд-API на FastAPI через JSON-формат обмена данными.

Ключевые элементы пользовательского интерфейса включают: страницу регистрации и входа, личный кабинет с отображением прогресса, страницу решения заданий со встроенным редактором кода, а также страницу аналитики с графиками успеваемости.

2.4.3 Бэкенд: Python и FastAPI

В качестве языка разработки бэкенда выбран Python, а в качестве веб-фреймворка - FastAPI.

Python является одним из наиболее популярных языков для разработки бэкенд-сервисов и интеграции с системами искусственного интеллекта. Богатая экосистема библиотек ускоряет разработку. Простота написания асинхронного кода критически важна для эффективной обработки множества одновременных запросов.

FastAPI обеспечивает высокую производительность (на уровне Node.js и Go) благодаря асинхронной архитектуре, построенной на основе ASGI-сервера Uvicorn. Автоматическая генерация интерактивной документации API (Swagger UI и ReDoc) упрощает тестирование и отладку. Встроенная валидация данных через Pydantic-модели и поддержка типов (type hints)

повышает надёжность кода. Наличие большого количества примеров интеграции с PostgreSQL и Docker упрощает развёртывание.

Бэкенд реализует следующие API-эндпоинты:

- /auth/register, /auth/login - регистрация и аутентификация пользователей;
- /tasks/ - получение списка заданий и конкретного задания по ID;
- /tasks/submit - отправка решения на проверку (маршрутизация к ИИ-компоненту);
- /progress/ - получение статистики прогресса пользователя;
- /ai/explain - получение пояснения от ИИ по конкретной ошибке;
- /admin/metrics - внутренний эндпоинт для сбора метрик Prometheus.

2.4.4 База данных: PostgreSQL

В качестве системы управления базами данных выбрана PostgreSQL 15+.

Обоснование выбора: PostgreSQL является наиболее надёжной и функциональной открытой реляционной СУБД, поддерживающей сложные запросы, транзакции и полнотекстовый поиск. Поддержка JSONB-типа данных позволяет гибко хранить неструктурированную информацию (например, истории решений и пояснения ИИ). Высокая производительность при работе с большими объёмами данных и поддержка репликации для обеспечения отказоустойчивости. Возможность развёртывания в контейнере Docker с сохранением данных через Persistent Volumes в Kubernetes.

Разработанная схема базы данных включает следующие основные таблицы:

- users - информация о пользователях (id, email, hashed_password, role, created_at);
- tasks - описание заданий (id, title, description, difficulty, input_example, output_example);

- submissions - история отправленных решений (id, user_id, task_id, code, result, explanation, submitted_at);
- user_progress - агрегированный прогресс пользователя по темам (user_id, topic, correct_count, total_count);
- subscriptions - информация о подписках (user_id, status, start_date, end_date).

2.4.5 Искусственный интеллект: YandexGPT API

Для реализации ИИ-ассистента, проверяющего код и генерирующего пояснения, используется API YandexGPT от Yandex Cloud.

YandexGPT предоставляет мощную генеративную нейросеть, обученную на русскоязычных данных и способную анализировать программный код на предмет ошибок. Нейросеть YandexGPT Lite 5 демонстрирует впечатляющие результаты в нахождении ошибок в программном коде, что подтверждается её внедрением в образовательные платформы Яндекс Практикума и «ОГЭ по информатике с Яндекс Учебником». Наличие официального Python SDK и подробной документации упрощает интеграцию в разрабатываемое приложение. Модель предоставляется как облачный сервис с оплатой за фактическое использование, что позволяет минимизировать начальные затраты и масштабироваться по мере роста числа пользователей.

YandexGPT API предоставляет доступ к двум моделям: YandexGPT Lite, которая является более быстрой и дешёвой и YandexGPT Pro – она более точная, подходит для сложных задач. Интеграция осуществляется через официальный Python SDK `yandex-cloud-ml-sdk`. Полученный ответ от YandexGPT сохраняется в базе данных и возвращается пользователю через веб-интерфейс. Для оптимизации производительности и снижения затрат на API-вызовы результаты кэшируются - повторные отправки идентичного кода по тому же заданию обслуживаются из кэша без обращения к YandexGPT.

Для обеспечения безопасности API-ключ YandexCloud не передавался на клиентскую сторону, а хранился на сервере. Запросы от фронтенда направлялись на серверный API-маршрут (прокси), который добавлял ключ авторизации и перенаправлял запрос к YandexGPT. Это исключало возможность несанкционированного доступа к ключу через инструменты разработчика в браузере.

2.4.6 Проверка выполнения кода: Judge0 API

YandexGPT анализирует код «на глаз», однако для полноценной проверки решений по информатике необходимо также исполнить код ученика на тестовых данных, чтобы убедиться в его корректности и соответствии ожидаемому результату. Для этой цели используется Judge0 API - специализированная система для безопасного и масштабируемого выполнения кода.

Judge0 является открытой системой выполнения кода, поддерживающей более 60 языков программирования, включая Python, который является основным языком ЕГЭ по информатике). Система обеспечивает изолированное выполнение, что очень важно для безопасности - код ученика не будет наносить вред серверной инфраструктуре.

Judge0 предоставляет простой HTTP JSON API и официальный Python SDK, что значительно упрощает интеграцию в разрабатываемое приложение. Система возвращает не только результат выполнения программы, но и метаданные: время выполнения, использованную память, код ошибки (если выполнение не удалось), что позволяет давать ученику более подробную обратную связь. Принцип работы Judge0 API: в запросе передаётся исходный код и тестовые входные данные; Judge0 запускает код в изолированной среде и возвращает выходные данные и метаданные выполнения. Это позволяет автоматически проверять, выдаёт ли программа ученика ожидаемый результат на тестовых наборах.

2.4.7 Туннелирование: Ngrok

Для тестирования и демонстрации проекта заказчику без необходимости покупки и настройки публичного сервера используется Ngrok.

Ngrok позволяет создать защищённый HTTPS-туннель к локальному серверу разработчика, предоставляя публичную ссылку вида `https://abc123.ngrok.io`. Это упрощает демонстрацию работающего прототипа руководителю практики или потенциальным инвесторам без развёртывания в продакшн-окружении. Встроенный веб-инспектор (`localhost:4040`) позволяет анализировать входящие запросы и ответы, что особенно полезно при отладке Webhook-уведомлений (например, от платёжной системы при успешной оплате подписки).

2.4.8 Контейнеризация и оркестрация

Для контейнеризации всех компонентов системы используется Docker. Он позволяет упаковать бэкенд-приложение, базу данных и вспомогательные сервисы в изолированные контейнеры с единой конфигурацией окружения. Применение Docker следующее: был написан Dockerfile для сборки образа бэкенда на основе официального образа Python; все сервисы сконфигурированы в едином файле `docker-compose.yml`, что позволяет запускать всю систему одной командой `docker compose up -d`. Это гарантирует идентичность окружений разработки, тестирования и эксплуатации.

Для оркестрации контейнеров в продуктивной среде используется Kubernetes. Выбор Kubernetes обоснован необходимостью автоматического масштабирования под нагрузкой и самовосстановления при сбоях. Применение Kubernetes: написание манифестов развёртывания для каждого компонента (Deployment, Service, ConfigMap); настройка горизонтального автомасштабирования для динамического увеличения числа реплик бэкенда

при росте нагрузки; использование PersistentVolumeClaims для сохранения данных PostgreSQL между перезапусками.

2.4.9 Мониторинг и визуализация: Grafana

Для мониторинга состояния системы и визуализации ключевых метрик используется Grafana в связке с Prometheus. Prometheus собирает метрики (количество активных пользователей, время ответа API, частота ошибок, нагрузка на серверы) со всех компонентов системы. Эти метрики визуализируются через дашборды Grafana. Разработаны следующие дашборды:

Общий дашборд: количество активных пользователей, количество запросов в секунду (RPS), среднее время ответа API.

Дашборд ИИ-сервиса: количество вызовов YandexGPT API, среднее время ответа, количество кэшированных запросов.

Дашборд базы данных: количество активных подключений, частота выполнения запросов, размер базы данных.

Дашборд инфраструктуры: загрузка CPU и памяти контейнеров, сетевой трафик, состояние подов в Kubernetes.

2.4.10 CI/CD: GitHub CI

Для автоматизации процесса сборки, тестирования и развёртывания используется GitLab CI. Настроен пайплайн, состоящий из следующих стадий:

- build - сборка Docker-образов для бэкенда и вспомогательных сервисов;
- test - выполнение юнит-тестов бэкенда и тестирование интеграции с Judge0 API;
- deploy_dev - автоматическое развёртывание на окружение разработки (Dev) при пуше в ветку develop;

- `deploy_stage` - развёртывание на стейджинг-окружение при создании тега `v*`;
- `deploy_prod` - развёртывание на продуктивное окружение (Prod) после ручного подтверждения.

2.5 Проектирование подсистем

2.5.1 Подсистема аутентификации и управления пользователями

Подсистема реализована на базе JWT (JSON Web Tokens). При успешной аутентификации сервер возвращает `access_token` (срок действия 1 час) и `refresh_token` (срок действия 30 дней). `Access_token` используется для доступа к защищённым API-эндпоинтам, `refresh_token` - для получения новой пары токенов без повторного ввода пароля. Пароли пользователей хранятся в базе данных в хешированном виде с использованием алгоритма `bcrypt`.

2.5.2 Подсистема заданий и проверки решений

Структура задания в базе данных: `title`, `description`, `difficulty` (лёгкое, среднее, сложное), `input_example` (примеры входных данных), `output_example` (ожидаемые выходные данные). Задания группируются по темам: «Алгоритмизация», «Программирование», «Анализ данных», «Комбинаторика» и т.д.

Процесс проверки решения включает следующие шаги:

- получение кода ученика и ID задания от клиента;
- поиск решения в кэше (Redis) для ускорения повторных проверок;
- при отсутствии в кэше - отправка кода в Judge0 API для выполнения на тестовых наборах;
- анализ результатов выполнения. Если программа не прошла тесты. Формирование запроса к YandexGPT для генерации объяснения ошибки на основе кода ученика;

- сохранение результата в базу данных и возврат ответа клиенту.

2.5.3 Подсистема персонализации и ИИ-ассистента

Подсистема персонализации отвечает за построение индивидуальной траектории обучения на основе истории решений пользователя. Для каждой темы (всего выделено 12 тем по информатике) ведётся статистика количества правильно и неправильно решённых заданий. Эти данные агрегируются в таблице `user_progress`.

ИИ-ассистент на базе YandexGPT используется для объяснения ошибок (как описано в разделе 2.4.5), ответов на вопросы пользователя в текстовом чате (дополнительная функция) и генерации пояснений к сложным терминам. Система интегрируется с API YandexGPT через официальный Python SDK `yandex-cloud-ml-sdk`.

2.5.4 Подсистема мониторинга и аналитики

Подсистема мониторинга построена на стеке Prometheus + Grafana. Prometheus периодически опрашивает метрические эндпоинты приложений (экспортируемые через клиентскую библиотеку Prometheus для Python). Метрики сохраняются во временной базе данных Prometheus (TSDB). Grafana подключается к Prometheus как к источнику данных и визуализирует метрики в виде графиков, гистограмм и тепловых карт.

Настроены следующие алерты (уведомления) в Prometheus:

- высокая задержка API: время ответа > 5 секунд в течение 5 минут;
- высокая частота ошибок: доля HTTP-ответов с кодом 5xx $> 1\%$ за последние 5 минут;
- высокая нагрузка на CPU контейнеров: $> 80\%$ в течение 10 минут;
- проблемы с подключением к базе данных: недоступность PostgreSQL более 1 минуты;

2.5.5 Подсистема развёртывания и инфраструктуры

Инфраструктура развёртывания реализована по принципу «инфраструктура как код» (IaC - Infrastructure as Code) с использованием Terraform. Конфигурация Terraform описывает все необходимые облачные ресурсы: виртуальные машины, сети, балансировщики нагрузки. Это позволяет развернуть идентичное окружение для разработки, тестирования и продуктивной среды с минимальными усилиями.

Продуктивное окружение включает:

Балансировщик нагрузки (Load Balancer) - распределяет входящий HTTPS-трафик между несколькими экземплярами бэкенд-приложения.

Кластер Kubernetes - оркестрирует контейнеры бэкенда, Judge0 и вспомогательных сервисов.

PostgreSQL - развёрнута вне кластера (на отдельной виртуальной машине) для упрощения резервного копирования, но в продуктовой конфигурации может быть запущена внутри кластера с PersistentVolume.

Prometheus + Grafana - развёрнуты в кластере Kubernetes.

Redis - используется для кэширования ответов YandexGPT.

2.6 Обеспечение безопасности

В системе реализованы следующие меры безопасности:

Весь трафик между клиентом и сервером передаётся по протоколу HTTPS с сертификатом Let's Encrypt (в продакшн-окружении). Трафик между микросервисами внутри кластера Kubernetes также может быть зашифрован с использованием mTLS (в перспективе).

Все запросы к базе данных выполняются через параметризованные запросы SQLAlchemy, исключающие возможность SQL-инъекций. Выходные данные экранируются перед вставкой в HTML-страницы, что предотвращает межсайтовый скриптинг (XSS).

Аутентификация и авторизация – здесь используется механизм JWT-токенов с коротким временем жизни `access_token` (1 час) и защищёнными маршрутами на бэкенде. Реализована ролевая модель доступа (ученик, родитель, администратор).

Использование Judge0 гарантирует, что код ученика выполняется в изолированной среде (контейнере) с ограничением ресурсов (время выполнения, память) и не имеет доступа к файловой системе сервера и сети.

Вся персональная информация хранится в зашифрованном виде. Реализована возможность экспорта и удаления данных пользователя по запросу в соответствии с требованиями Ф3-152.

2.7 Масштабируемость и производительность

Архитектура системы обеспечивает горизонтальное масштабирование практически всех компонентов:

Бэкенд-приложение может быть развёрнуто в нескольких репликах за балансировщиком нагрузки. Благодаря отсутствию состояния на уровне приложения (stateless) - сессии хранятся в Redis, загружаемые файлы - в S3 - масштабирование не требует дополнительной настройки.

База данных PostgreSQL может масштабироваться вертикально (увеличение ресурсов виртуальной машины) или горизонтально с использованием репликации «ведущий-ведомый» (master-slave). Для повышения производительности чтения могут быть добавлены реплики (read replicas).

Judge0 естественным образом поддерживает кластеризацию - несколько экземпляров могут обрабатывать входящие запросы параллельно.

Кэширование через Redis разгружает базу данных и снижает нагрузку на YandexGPT API за счёт кэширования повторяющихся запросов.

Ожидаемая производительность системы при пиковой нагрузке (1000 одновременных пользователей):

Бэкенд (FastAPI + Uvicorn): до 5000 запросов в секунду (при 4 рабочих процессах).

База данных PostgreSQL: до 1000 транзакций в секунду при чтении; до 200 транзакций в секунду при записи.

Judge0: время выполнения одного кода - 1-3 секунды (в зависимости от сложности).

YandexGPT API: время ответа - 2-5 секунд (кэширование позволяет не обращаться к API повторно для одинаковых запросов).

2.8 Выводы по главе

В рамках проектирования архитектуры и функциональности онлайн-школы «Мост знаний» были решены следующие ключевые задачи:

Сформулированы функциональные и нефункциональные требования, определяющие границы и характеристики разрабатываемой системы.

Разработана многоуровневая архитектура, включающая клиентский слой (HTML/CSS/JS), слой приложений (Python FastAPI), слой данных (PostgreSQL), слой искусственного интеллекта (YandexGPT API, Judge0 API) и инфраструктурный слой (Docker, Kubernetes, Grafana).

Обоснован выбор технологического стека, обеспечивающего производительность, масштабируемость, безопасность и экономическую эффективность проекта.

Спроектированы ключевые подсистемы: аутентификации и управления пользователями, заданий и проверки решений, персонализации и ИИ-ассистента, мониторинга и аналитики, развёртывания и инфраструктуры.

Определены меры обеспечения безопасности и механизмы масштабирования системы при росте нагрузки.

Можно подвести итог и сказать, что разработанная архитектура полностью соответствует поставленным целям и позволяет приступить к следующему этапу - непосредственной реализации программного

обеспечения. Выбранные технологии и архитектурные решения обеспечивают высокую производительность, надёжность и безопасность платформы, а также возможность дальнейшего расширения функциональности и масштабирования при увеличении числа пользователей.

3. РЕАЛИЗАЦИЯ СИСТЕМЫ

В третьей главе подробно описывается практическая реализация онлайн-школы «Мост знаний». Рассматриваются все этапы создания программного продукта: от настройки инфраструктуры и контейнеризации до написания бэкенда, фронтенда, интеграции с ИИ-сервисами и организации мониторинга. Основной акцент сделан на архитектурных решениях и логике взаимодействия компонентов.

3.1 Общая структура и компоненты системы

Реализованная система представляет собой многоуровневое веб-приложение с микросервисной архитектурой. Все компоненты упакованы в контейнеры Docker и управляются оркестратором Kubernetes (в продуктивной среде) либо запускаются локально через Docker Compose (на этапе разработки). В таблице 3.1 приведён перечень ключевых компонентов и их назначение.

Таблица 3.1 – Компоненты системы

Компонент	Технология	Назначение
Веб-интерфейс	HTML, CSS, JavaScript	Отображение заданий, редактор кода, личный кабинет
Бэкенд-сервер	Python / FastAPI	Бизнес-логика, API, интеграция с БД и внешними сервисами
База данных	PostgreSQL 15	Хранение пользователей, заданий, решений, прогресса
Кэш-сервер	Debian	Кэширование ответов ИИ и пользовательских сессий
ИИ-ассистент	YandexGPT API	Генерация объяснений ошибок, подсказок, ответов на вопросы
Исполнитель кода	Judge0 API	Безопасное выполнение кода ученика на тестовых данных

Окончание таблицы 3.1

Туннелирование	Ngrok	Временный публичный доступ к локальному серверу для демонстрации
Контейнеризация	Docker	Упаковка сервисов с их окружением
Оркестрация	Kubernetes	Масштабирование, отказоустойчивость, управление контейнерами
Мониторинг	Prometheus + Grafana	Сбор метрик, визуализация состояния системы
CI/CD	GitHub	Автоматическая сборка, тестирование и деплой

Взаимодействие между компонентами осуществляется по протоколу HTTP/REST, а также через внутренние очереди запросов. Клиент (браузер) общается с бэкендом через защищённые API-эндпоинты; бэкенд, в свою очередь, обращается к Judge0 и YandexGPT.

3.2 Реализация инфраструктуры и развёртывания

3.2.1 Подготовка виртуальной среды

Для изолированной разработки и тестирования использовалась виртуальная машина под управлением Oracle VM VirtualBox. В качестве гостевой ОС выбрана Debian 13 (Trixie) – стабильный дистрибутив Linux с длительным сроком поддержки. ВМ была сконфигурирована с 2 процессорными ядрами, 6 ГБ оперативной памяти и диском на 30 ГБ, показанная на рисунке 3.1. Сетевой интерфейс настроен в режиме «сетевой мост», что позволило получить доступ к виртуальной машине из локальной сети и обеспечить выход в интернет.

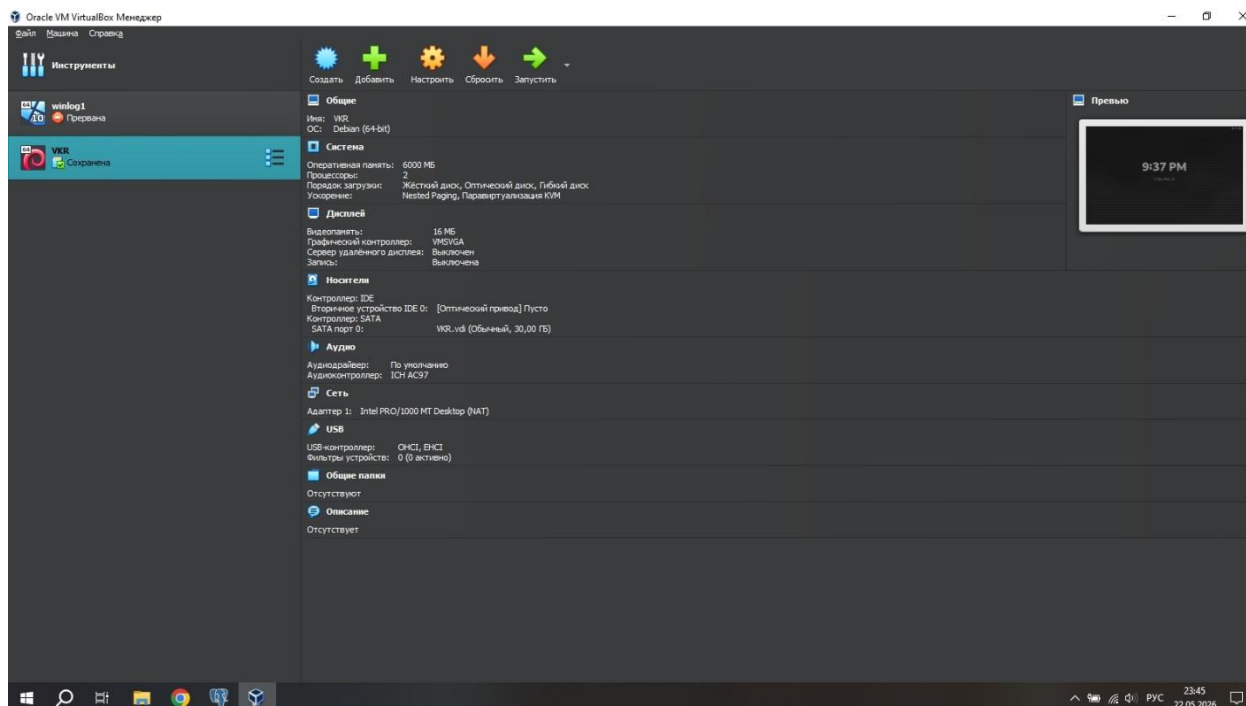


Рисунок – 3.1 Virtual Box

После установки базовой системы были развёрнуты необходимые пакеты: Docker, Docker Compose, Git, Python 3, PostgreSQL, Nginx. Для удобства работы пользователь был добавлен в группу docker, чтобы запускать контейнеры без прав суперпользователя.

3.2.2 Контейнеризация сервисов с помощью Docker

Каждый компонент системы (бэкенд, база данных, Debian) был упакован в отдельный контейнер. Для бэкенда написан Dockerfile на основе официального образа Python 3.11-slim. В нём задана рабочая директория, скопированы файлы зависимостей (requirements.txt), выполнена установка пакетов, а затем скопирован исходный код. Точкой входа определён запуск Uvicorn-сервера на порту 8000.

Для локальной разработки и тестирования всех сервисов одновременно использовался файл docker-compose.yml. В нём описаны четыре сервиса:

- db – PostgreSQL с монтированием тома для сохранения данных;
- redis – кэш-сервер;

- backend – сборка образа бэкенда, проброс порта 8000, передача переменных окружения (ключа YandexCloud, адреса БД и т.д.);
- nginx – веб-сервер для раздачи статики фронтенда и проксирования запросов к API.

Запуск всей системы локально сводился к одной команде `docker-compose up -d`. Это обеспечило идентичность окружений разработки, тестирования и будущего продакшена.

3.2.3 Оркестрация в Kubernetes

Для продуктивного окружения (или имитации такового) был развёрнут кластер Kubernetes. На этапе разработки использовался Minikube, на сервере – kubeadm. Написаны манифесты развёртывания (Deployment) для бэкенда: указано количество реплик (3), лимиты ресурсов (CPU, память), переменные окружения, подтягиваемые из Kubernetes Secrets (чтобы не хранить ключи в коде). Также создан Service типа LoadBalancer для балансировки нагрузки между репликами.

Горизонтальное масштабирование настроено автоматически: при превышении 70% загрузки CPU создаются новые поды. Для базы данных использован отдельный StatefulSet с PersistentVolumeClaim, чтобы данные не терялись при перезапусках.

3.2.4 Временный публичный доступ через Ngrok

Для демонстрации рабочего прототипа онлайн-школы «Мост знаний» руководителю практики и потенциальным пользователям, находящимся вне локальной сети разработчика, было необходимо обеспечить временный публичный доступ к веб-приложению. На этапе разработки сервер развёртывался на виртуальной машине с IP-адресом, недоступным из внешней сети. Оптимальным решением стало использование сервиса Ngrok,

позволяющего создать защищённый HTTPS-туннель к локальному серверу без сложной настройки маршрутизаторов и приобретения доменного имени.

Утилита Ngrok версии 3.39.1 была установлена на виртуальную машину Debian 13. Для привязки к аккаунту использовался персональный авторизационный токен (authtoken). На рисунке 3.2 показан интерфейс веб-панели Ngrok, где создан токен с идентификатором `cr_6k0gnE`, привязанный к учётной записи `protikalka008336@gmail.com`. В терминале была выполнена команда `“ngrok authtoken cr_3DLIRHTMRkTBDmzAuQg606kQgnE”`, после чего утилита сохранила учётные данные в конфигурационный файл. Несмотря на опечатку в имени команды на одном из скриншотов (`ngrork` вместо `ngrok`), фактическая настройка была выполнена корректно, о чём свидетельствует последующая успешная сессия.

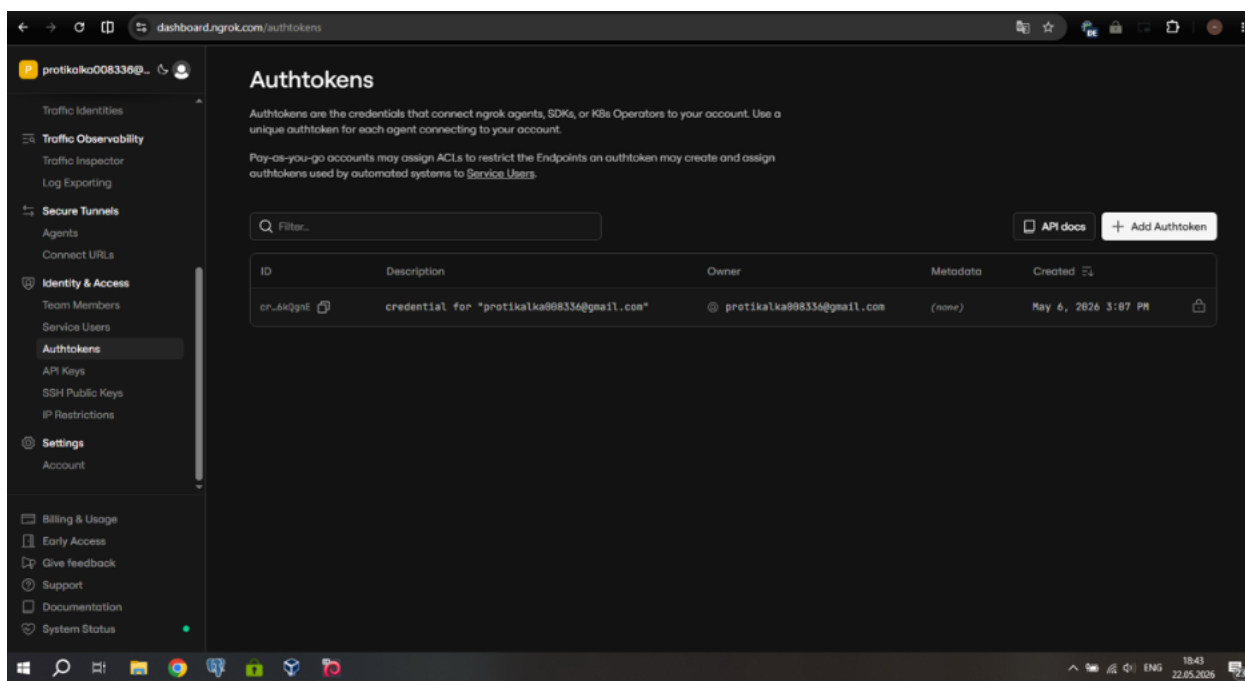
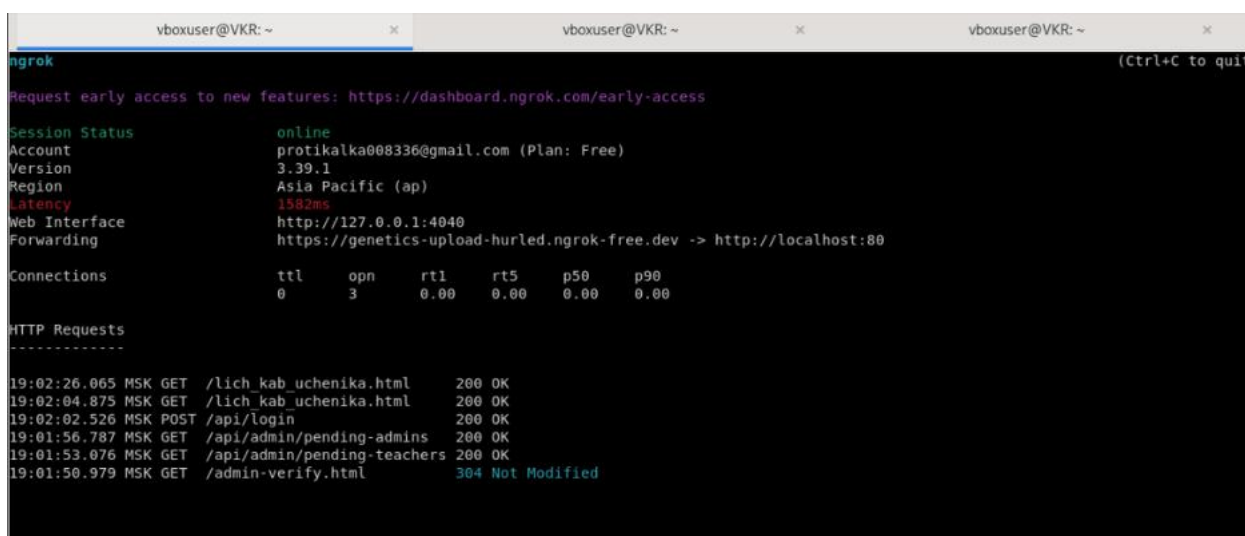


Рисунок 3.2 - Интерфейс веб-панели Ngrok

Для публикации веб-сервера, работающего на порту 80 (где Nginx обслуживал статические файлы фронтенда и проксировал API-запросы на бэкенд), была выполнена команда `«ngrok http 8000»`

После запуска на экране появляется интерактивная панель со статусом сессии, как показано на рисунке 3.3. В данном случае статус online указывает на успешное подключение к серверам Ngrok. Параметры сессии:

- Account – protikalka008336@gmail.com (бесплатный тарифный план).
- Version – 3.39.1.
- Region – Asia Pacific (ap), выбран автоматически по географическому положению сервера.
- Latency – 1582 мс (задержка до региона AP).
- Web Interface – <http://127.0.0.1:4040> – локальный веб-интерфейс для просмотра запросов.
- Forwarding <https://genetics-upload-hurled.ngrok-free.dev>
<http://localhost:80>



```
ngrok
Request early access to new features: https://dashboard.ngrok.com/early-access

Session Status      online
Account             protikalka008336@gmail.com (Plan: Free)
Version             3.39.1
Region              Asia Pacific (ap)
Latency             1582ms
Web Interface        http://127.0.0.1:4040
Forwarding           https://genetics-upload-hurled.ngrok-free.dev -> http://localhost:80

Connections
  ttl   opn   rt1   rt5   p50   p90
    0     3    0.00  0.00  0.00  0.00

HTTP Requests
-----
19:02:26.065 MSK GET /lich_kab_uchenika.html 200 OK
19:02:04.875 MSK GET /lich_kab_uchenika.html 200 OK
19:02:02.526 MSK POST /api/login 200 OK
19:01:56.787 MSK GET /api/admin/pending-admins 200 OK
19:01:53.076 MSK GET /api/admin/pending-teachers 200 OK
19:01:50.979 MSK GET /admin-verify.html 304 Not Modified
```

Рисунок 3.3 - Интерактивная панель со статусом сессии

Теперь любой пользователь, перейдя по этому адресу, попадал на веб-сайт, размещённый на виртуальной машине разработчика. Все HTTP-запросы автоматически шифровались (HTTPS), а Ngrok обеспечивал их пересылку через свой туннель на локальный порт 80.

Встроенный веб-интерфейс Ngrok (<http://127.0.0.1:4040>) предоставляет детальную информацию о проходящих запросах, в разделе HTTP Requests видны следующие записи:

- GET /lich_kab_uchenika.html 200 OK – загрузка личного кабинета ученика;
- GET /lich_kab_uchenika.html 200 OK (повторная загрузка);
- POST /api/login 200 OK – успешный запрос аутентификации;
- GET /api/admin/pending-admins 200 OK – получение списка администраторов, ожидающих верификации;
- GET /api/admin/pending-teachers 200 OK – получение списка учителей на верификацию;
- GET /admin-verify.html 304 Not Modified – страница администратора загружена из кэша.

Каждая строка содержит метод HTTP, путь, код ответа и время запроса. Благодаря этому интерфейсу разработчик может в реальном времени наблюдать, какие эндпоинты вызываются, с какими параметрами и какие ошибки (если бы они были) возвращаются. Это оказалось особенно полезным при отладке CORS-ошибок и проверке формата JSON, передаваемого между фронтендом и бэкендом.

Одновременно с работой Ngrok на той же виртуальной машине был запущен бэкенд-сервер Uvicorn на порту 8000, соответствуя рисунку 3.4. На этом скриншоте видны логи Uvicorn, фиксирующие те же самые запросы, которые отображались в Ngrok:

- POST /api/login HTTP/1.0" 200 OK – запрос аутентификации;
- GET /api/admin/pending-teachers HTTP/1.0" 200 OK;
- GET /api/admin/pending-admins HTTP/1.0" 200 OK.

```

root@VKR:/home/vboxuser# cd ~/backend && source venv/bin/activate
(venv) root@VKR:~/backend# pkill -f uvicorn
(venv) root@VKR:~/backend# INFO:      Shutting down
INFO:      Waiting for application shutdown.
INFO:      Application shutdown complete.
INFO:      Finished server process [101628]
^C
(venv) root@VKR:~/backend# uvicorn main:app --host 127.0.0.1 --port 8000 &
[1] 101973
(venv) root@VKR:~/backend# INFO:      Started server process [101973]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:      45.88.42.4:0 - "POST /api/login HTTP/1.0" 200 OK
INFO:      45.88.42.4:0 - "POST /api/login HTTP/1.0" 200 OK
INFO:      45.88.42.4:0 - "GET /api/admin/pending-teachers HTTP/1.0" 200 OK
INFO:      45.88.42.4:0 - "GET /api/admin/pending-admins HTTP/1.0" 200 OK

```

Рисунок 3.4 - сервер Uvicorn

Совпадение записей в логах Ngrok и Uvicorn доказывает, что туннель функционирует корректно: публичный URL Ngrok принимает входящие запросы от клиентов, пересылает их на localhost:80, а Nginx (или другой прокси) перенаправляет вызовы API на localhost:8000, где их обрабатывает FastAPI-приложение. Таким образом, для удалённого пользователя весь комплекс выглядит как единое приложение, доступное по одному адресу.

Бесплатный тариф Ngrok имеет ограничения: сессия не может длиться более 2 часов без перезапуска, домен при каждом запуске генерируется случайным образом, а также действует лимит на количество одновременных соединений. Однако для целей дипломной работы – демонстрации прототипа и проведения нескольких тестовых сеансов – этого было достаточно. Для будущего продуктивного развёртывания планируется использование собственного домена и полноценного облачного хостинга.

Использование Ngrok позволило оперативно организовать внешний доступ к разрабатываемой системе без затрат на домен и публичный сервер, а также предоставило удобный инструмент для мониторинга HTTP-трафика в реальном времени.

3.3 Реализация базы данных

3.3.1 Схема базы данных

База данных спроектирована в соответствии с диаграммой классов, представленной во второй главе. Основные сущности:

- users – пользователи системы (ученики, учителя, администраторы);
- video_lessons – видеоуроки;
- quiz_questions – вопросы тестов к урокам;
- student_quiz_results – результаты прохождения тестов учениками;
- notification_log – лог уведомлений (служебная таблица).

Все таблицы используют первичные ключи SERIAL, внешние связи – с каскадным удалением (ON DELETE CASCADE). Индексы созданы по полям user_id, task_id, submitted_at для ускорения выборок.

На рисунке 3.5 можно увидеть подключенную базу данных:

```
root@VKR:/home/vboxuser# PGPASSWORD=123a psql -U admin -d school_db
-h 127.0.0.1 -p 5433
psql (17.9 (Debian 17.9-0+deb13u1))
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, c
ompression: off, ALPN: postgresql)
Type "help" for help.
```

```
school_db=> \dt
```

```
          List of relations
Schema |          Name          | Type  | Owner
-----+-----+-----+-----
public | materials              | table | admin
public | messages               | table | admin
public | notification_log       | table | postgres
public | quiz_questions         | table | admin
public | student_quiz_results   | table | admin
public | subscription_types     | table | admin
public | user_subscriptions     | table | admin
public | users                  | table | admin
public | video_lessons          | table | admin
(9 rows)
```

```
school_db=> █
```

Рисунок 3.5 - Подключенная база данных

Структура таблицы users, показанной на рисунке 3.6, включает следующие поля:

- id – первичный ключ;
- email – уникальный логин;
- password_hash – хеш пароля;
- first_name, last_name – имя и фамилия;
- phone – номер телефона;
- role – роль (student, teacher, admin), с проверкой через CHECK-ограничение;
- is_verified – флаг подтверждения регистрации (булевый, по умолчанию false);
- created_at, updated_at – метки времени создания и изменения.

Table "public.users"				
Column	Type	Collation	Nullable	Default
id	integer		not null	nextval('us_id_seq'::regclass)
email	character varying(255)		not null	
password_hash	character varying(255)		not null	
first_name	character varying(100)		not null	
last_name	character varying(100)		not null	
phone	character varying(20)			
role	character varying(20)		not null	'student'::character varying
is_verified	boolean			false
created_at	timestamp with time zone			now()
updated_at	timestamp with time zone			now()

Indexes:

- "users_pkey" PRIMARY KEY, btree (id)
- "users_email_key" UNIQUE CONSTRAINT, btree (email)

Check constraints:

- "chk_users_role" CHECK (role::text = ANY (ARRAY['student'::character varying::text, 'teacher'::character varying::text, 'admin'::character varying::text]))
- "users_role_check" CHECK (role::text = ANY (ARRAY['student'::character varying, 'teacher'::character varying, 'admin'::character varying]::text[]))

Referenced by:

- TABLE "student_quiz_results" CONSTRAINT "student_quiz_results_student_id_fkey" FOREIGN KEY (student_id) REFERENCES users(id)
- TABLE "student_quiz_results" CONSTRAINT "student_quiz_results_teacher_id_fkey" FOREIGN KEY (teacher_id) REFERENCES users(id)
- TABLE "video_lessons" CONSTRAINT "video_lessons_teacher_id_fkey" FOREIGN KEY (teacher_id) REFERENCES users(id)

Рисунок 3.6 - Структура таблицы users

На рисунке 3.7 показаны записи в таблице users: присутствуют администраторы (role = admin), ученики (student), учителя (teacher). Поля first_name, last_name, email, is_verified заполнены согласно тестовым данным.

id	first_name	last_name	email	role	is_verified	created_at
12	Админ	Системы	admin@most.ru	admin	t	2026-05-18 17:19:08.511539+03
14	Олег	Админов	oleg@admin.ru	admin	f	2026-05-19 15:42:58.677364+03
19	Роман	Кузьмин	y@y.com	admin	t	2026-05-19 16:20:36.411828+03
1	Test	User	test@demo.com	student	t	2026-05-15 12:57:08.249859+03
2	New	User	new@test.ru	student	t	2026-05-15 14:14:44.337995+03
3	Иван	Локкин	new@test.ru	student	t	2026-05-15 14:27:40.204489+03
4	Иван	Артурчик	test@r.com	student	t	2026-05-15 14:30:08.466485+03
5	Алексей	Иванов	phone@test.ru	student	t	2026-05-15 15:21:17.420771+03
6	Алена	Ложкин	test@rk.com	student	t	2026-05-15 15:29:00.218408+03
7	Алена	Локкин	qqq@qq.ru	student	t	2026-05-15 15:32:14.17244+03
8	Алена	Петрова	test2@test.ru	student	t	2026-05-15 15:56:27.88523+03
9	Лена	Иванова	ww@ww.ru	student	t	2026-05-15 15:59:57.963086+03
10	Сергей	Андреев	uu@uu.ru	student	t	2026-05-18 13:55:44.577648+03
20	Иван	Локкин	x@x.ru	student	t	2026-05-19 17:12:44.115014+03
11	Тест	Учитель	test@teacher.ru	teacher	t	2026-05-18 16:44:33.706887+03
13	Иван	Тарасов	yy@yy.ru	teacher	t	2026-05-18 17:41:33.791171+03
15	Никита	Петров	ee@ee.ru	teacher	t	2026-05-19 15:48:51.071443+03
17	Алена	Петров	pp@pp.com	teacher	t	2026-05-19 15:52:36.313709+03

Рисунок 3.7 - Записи таблицы users

На рисунке 3.8 приведена структура: id, title, description, video_url, teacher_id (внешний ключ на users.id), created_at.

```
school_db=> \d video_lessons
```

Column	Type	Collation	Nullable	Default
id	integer		not null	nextval('video_lessons_id_seq'::regclass)
title	character varying(255)		not null	
description	text			
video_url	character varying(500)			
teacher_id	integer			
created_at	timestamp without time zone			CURRENT_TIMESTAMP

Indexes:

"video_lessons_pkey" PRIMARY KEY, btree (id)

Foreign-key constraints:

"video_lessons_teacher_id_fkey" FOREIGN KEY (teacher_id) REFERENCES users(id)

Referenced by:

TABLE "quiz_questions" CONSTRAINT "quiz_questions_lesson_id_fkey" FOREIGN KEY (lesson_id) REFERENCES video_lessons(id) ON DELETE CASCADE

TABLE "student_quiz_results" CONSTRAINT "student_quiz_results_lesson_id_fkey" FOREIGN KEY (lesson_id) REFERENCES video_lessons(id)

```
school_db=>
```

Рисунок 3.8 - Структура таблицы video_lessons

Содержимое таблицы, показанный на рисунке 3.9, один тестовый урок с заголовком «Тестовый урок», видео URL на Rutube и teacher_id = 17.

```
school_db=> SELECT id, title, description, video_url, teacher_id, created_at
FROM video_lessons
ORDER BY id;
```

id	title	description	video_url	teacher_id	created_at
1	Тестовый урок		https://rutube.ru/play/embed/cae3fd4f29c0f49dd8281767743d2c.3d	17	2026-05-19 18:20:53.061829

Рисунок 3.9 - Содержимое таблицы video_lessons

Вопросы тестов связаны с уроком через поле `lesson_id`. На рисунке 3.10 показан запрос, объединяющий `quiz_questions` с `video_lessons`. Вопрос: «Какая система счисления используется в компьютерах?», варианты ответов A–D, правильный ответ – B (двоичная). Поле `question_order` указывает порядок вопроса в тесте.

```
school_db=>
school_db=> SELECT
  qq.id,
  vl.title AS lesson_title,
  qq.question_text,
  qq.option_a,
  qq.option_b,
  qq.option_c,
  qq.option_d,
  qq.correct_answer,
  qq.question_order
FROM quiz_questions qq
LEFT JOIN video_lessons vl ON qq.lesson_id = vl.id
ORDER BY qq.lesson_id, qq.question_order;
```

id	lesson_title	question_text	option_a	option_b	option_c	option_d	correct_answer	question_order
2	Тестовый уро.	Какая система счисления исп.	Десятична.	Двоичная	Восьмерична.	Шестнадцатеричн.	B	1
	.к	.ользуется в компьютерах?	.я		.я	.ая		

Рисунок 3.10 - Таблица с вопросами

Таблица `student_quiz_results` хранит результаты прохождения тестов. Поля: `student_id`, `lesson_id`, `teacher_id`, `score`, `total_questions`, `percentage`, `answers` (в формате JSON), `completed_at`. Последние 5 записей показывают попытки ученика с `student_id` = 9. Одна попытка завершилась с результатом 100% при правильном ответе B, другие – 0% с ответом A, с соответствием с рисунком 3.11.

```
school_db=> SELECT * FROM student_quiz_results ORDER BY completed_at DESC LIMIT 5; -- последние результаты
```

id	student_id	lesson_id	teacher_id	score	total_questions	percentage	answers	completed_at
9	9	1	11	0	1	0.00	{"2": "A"}	2026-05-20 19:07:54.730111
8	9	1	11	1	1	100.00	{"2": "B"}	2026-05-20 19:07:54.713142
7	9	1	11	1	1	100.00	{"2": "B"}	2026-05-20 19:07:54.693108
6	9	1	11	0	1	0.00	{"2": "A"}	2026-05-20 19:07:54.658808
5	9	1	11	0	1	0.00	{"2": "A"}	2026-05-20 19:07:54.536364

Рисунок 3.11 - Результаты прохождения тестов

3.3.2 Миграции и подключение

Для управления изменениями схемы БД использовался инструмент Alembic. Первоначальная миграция была создана автоматически (`alembic`

revision --autogenerate), затем применена к пустой базе. В процессе разработки при добавлении новых полей (например, ai_explanation в таблицу submissions) генерировались новые миграции, которые бесшумно накатывались при старте приложения.

Подключение к PostgreSQL из бэкенда реализовано асинхронно с помощью библиотеки asynpg и SQLAlchemy. При старте приложения создаётся пул соединений, который переиспользуется для всех запросов. Это позволило избежать накладных расходов на установку соединения при каждом обращении к БД.

3.4 Реализация бэкенда на FastAPI

3.4.1 Организация кода

Проект бэкенда организован модульно. Главный файл main.py создаёт экземпляр FastAPI, подключает маршрутизаторы (auth, tasks, submissions, ai), настраивает CORS (чтобы фронтенд с другого порта мог обращаться к API) и добавляет middleware для логирования и сбора метрик.

В отдельном модуле database.py определена асинхронная сессия SQLAlchemy, которая используется во всех эндпоинтах через механизм зависимостей FastAPI. Модуль auth.py отвечает за генерацию JWT-токенов (access и refresh), их проверку и получение текущего пользователя.

Модели данных описаны в models.py (SQLAlchemy ORM) и в schemas.py (Pydantic-схемы для валидации входящих запросов).

3.4.2 Ключевые эндпоинты

После завершения разработки базовых эндпоинтов (регистрация, авторизация, управление пользователями, панель администратора) был проведён этап функционального тестирования API. Для этого бэкенд-сервер на FastAPI запускался локально на виртуальной машине с использованием

сервера Uvicorn. На рисунке 3.12 приведён фрагмент терминала с запуском Uvicorn и последующими логами входящих HTTP-запросов.

```
root@VKR:/home/vboxuser# cd ~/backend && source venv/bin/activate
(venv) root@VKR:~/backend# pkill -f uvicorn
(venv) root@VKR:~/backend# INFO: Shutting down
INFO: Waiting for application shutdown.
INFO: Application shutdown complete.
INFO: Finished server process [101628]
^C
(venv) root@VKR:~/backend# uvicorn main:app --host 127.0.0.1 --port 8000 &
[1] 101973
(venv) root@VKR:~/backend# INFO: Started server process [101973]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: 45.88.42.4:0 - "POST /api/login HTTP/1.0" 200 OK
INFO: 45.88.42.4:0 - "POST /api/login HTTP/1.0" 200 OK
INFO: 45.88.42.4:0 - "GET /api/admin/pending-teachers HTTP/1.0" 200 OK
INFO: 45.88.42.4:0 - "GET /api/admin/pending-admins HTTP/1.0" 200 OK
█
```

Рисунок 3.12 - Запуск Uvicorn и последующие логи

В процессе тестирования через веб-интерфейс (фронтенд, работающий через Ngrok-туннель) были выполнены следующие операции, которые отразились в логах Uvicorn:

Авторизация пользователей – на рисунке 3.12 видно несколько записей вида:

INFO: 45.88.42.4:0 - "POST /api/login HTTP/1.0" 200 OK

Это означает, что клиент (с IP-адреса 45.88.42.4) отправил POST-запрос на эндпоинт /api/login с учётными данными (логин/пароль). Сервер успешно проверил данные, сгенерировал JWT-токен и вернул ответ с кодом 200. Несколько повторяющихся записей говорят о том, что тестировались разные учётные записи или выполнялись повторные входы.

Получение списков неподтверждённых пользователей администратором:

INFO: 45.88.42.4:1 - "GET /api/admin/pending-teachers HTTP/1.0" 200 OK

INFO: 45.88.42.4:0 - "GET /api/admin/pending-admins HTTP/1.0" 200 OK

Эти вызовы соответствуют функциональности панели администратора, где отображаются заявки на регистрацию от учителей и других администраторов. Код 200 ОК означает, что сервер успешно вернул JSON-массив с данными ожидающих верификации пользователей.

Важно отметить, что все эти запросы приходили не из локальной сети напрямую, а через публичный URL Ngrok (<https://genetics-upload-hurled.ngrok-free.dev> → <http://localhost:80> → проксирование на 127.0.0.1:8000). В логах Uvicorn фигурирует внешний IP-адрес 45.88.42.4 (принадлежащий серверам Ngrok), что подтверждает корректную работу туннеля. Таким образом, бэкенд успешно обрабатывал запросы из интернета, что критически важно для демонстрации прототипа удалённому руководителю практики.

Благодаря детальным логам Uvicorn удалось убедиться в:

- работоспособности всех протестированных эндпоинтов (код ответа 200);
- правильной маршрутизации запросов через Ngrok;
- отсутствии ошибок на стороне сервера (нет записей с кодами 4xx или 5xx в приведённом фрагменте);
- корректной обработке авторизации и административных вызовов.

3.4.3 Обработка решений (логика)

В эндпоинте `POST /tasks/{task_id}/submit` реализована следующая последовательность действий:

- извлечение текущего пользователя из JWT-токена;
- загрузка задания из БД по `task_id`;
- формирование запроса к Judge0: передаётся код ученика, входные данные (берутся из поля `input_example` задания), ограничения по времени и памяти;
- ожидание ответа от Judge0 (асинхронный опрос по токenu).

Получаем `stdout`, `stderr`, время выполнения, код возврата;

- сравнение stdout с ожидаемым выводом (из задания). Если совпадают – вердикт «accepted», пояснение – стандартное. Если нет – вердикт «wrong_answer», и вызывается YandexGPT с промптом, содержащим условие задачи, код ученика и содержимое stderr (если есть);
- полученное от YandexGPT объяснение сохраняется в БД вместе с решением;
- обновление статистики прогресса: увеличиваются счётчики решённых задач по соответствующей теме;
- кэширование ответа YandexGPT в Redis на 1 час (по ключу, составленному из хеша кода и ID задания), чтобы при повторной отправке точно такого же решения не обращаться к платному API повторно;
- возврат клиенту JSON с вердиктом и пояснением.

3.4.4 Интеграция с YandexGPT

Для работы с YandexGPT написан отдельный сервисный модуль. В нём реализована асинхронная функция `get_explanation`, которая принимает текст промпта и возвращает ответ нейросети. Использовался официальный SDK Yandex Cloud, показанный рисунок 3.13, однако в финальной версии был применён прямой HTTP-запрос к REST API YandexGPT для большего контроля.

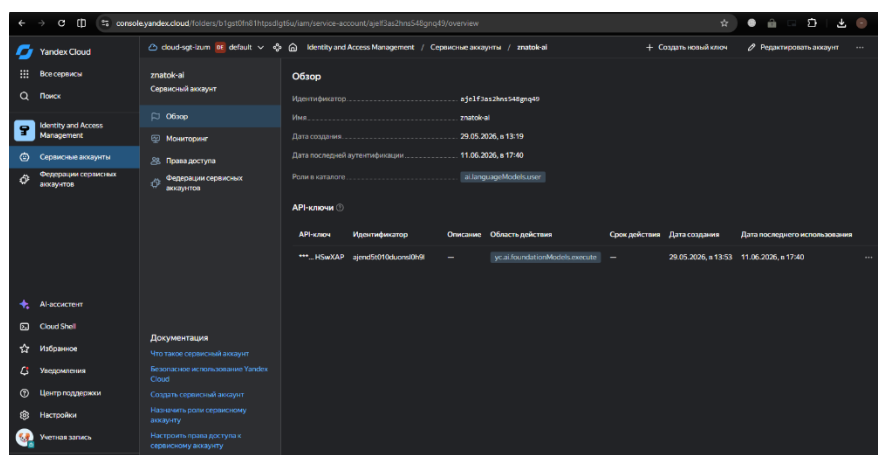


Рисунок 3.13 – Yandex Cloud

Промпт для объяснения ошибки составляется динамически: включает условие задачи, код ученика и сообщение об ошибке от интерпретатора. Температура модели установлена низкой (0,2), чтобы ответы были детерминированными и максимально точными, а максимальная длина ответа – 500 токенов, что достаточно для краткого пояснения.

3.4.5 Интеграция с Judge0

Аналогично, реализован модуль `judge0_client`. В нём используется библиотека `httpx` для отправки запросов к RapidAPI (Judge0 Community Edition). Сначала отправляется POST-запрос на `/submissions` с кодом и входными данными, в ответе получаем `token`. Затем, после паузы в 1 секунду, отправляется GET-запрос на `/submissions/{token}` для получения результатов. Если статус "Processing", цикл повторяется (с экспоненциальной задержкой). После получения финального статуса извлекаются `stdout`, `stderr`, `time`, `memory`.

Для повышения надёжности добавлены повторные попытки и таймауты (максимум 5 секунд ожидания выполнения). В случае недоступности Judge0 API (например, по лимитам RapidAPI) предусмотрен fallback – выдача сообщения «Ошибка выполнения, попробуйте позже», при этом YandexGPT не вызывается.

3.5 Реализация фронтенда

3.5.1 Архитектура фронтенда

Клиентская часть онлайн-школы «Мост знаний» представляет собой набор статических HTML-страниц, файлов каскадных таблиц стилей (CSS), JavaScript-скриптов и графических ресурсов. Вся эта файловая структура размещена в каталоге `/home/vboxuser/Downloads/glav/` на виртуальной машине разработчика. На рисунке 3.14 приведён результат выполнения команды `ls -l`, который позволяет детально изучить организацию фронтенда.

```

root@VKR:/home/vboxuser# ls -l /home/vboxuser/Downloads/glav/
total 412
-rw-r--r-- 1 root root 566 May 18 17:39 admin-dashboard.html
-rw-r--r-- 1 root root 10712 May 19 16:16 admin-verify.html
drwxr-xr-x 2 vboxuser vboxuser 4096 Apr 27 14:52 backend
-rw-r--r-- 1 vboxuser vboxuser 19158 May 14 12:31 help.html
drwxr-xr-x 2 vboxuser vboxuser 4096 Apr 19 19:48 images
-rw-r--r-- 1 vboxuser vboxuser 23215 May 13 15:32 index.html
-rw-r--r-- 1 vboxuser vboxuser 72016 May 20 19:11 lich_kab_uchenika.html
-rw-r--r-- 1 vboxuser vboxuser 44249 May 19 17:08 lich_kab_uchit.html
-rw-r--r-- 1 vboxuser vboxuser 10721 Apr 22 15:20 login.css
-rw-r--r-- 1 vboxuser vboxuser 7698 May 18 13:58 login.html
-rw-r--r-- 1 vboxuser vboxuser 9473 May 19 15:26 login.js
-rw-r--r-- 1 vboxuser vboxuser 16942 Apr 21 13:30 me.html
-rw-r--r-- 1 vboxuser vboxuser 9771 May 14 13:51 password-reset.css
-rw-r--r-- 1 vboxuser vboxuser 7963 May 14 13:45 password-reset.html
-rw-r--r-- 1 vboxuser vboxuser 4300 May 14 13:30 password-reset.js
-rw-r--r-- 1 vboxuser vboxuser 2330 May 13 16:59 pending-verification.html
-rw-r--r-- 1 vboxuser vboxuser 35470 May 13 15:37 politics.html
-rw-r--r-- 1 vboxuser vboxuser 12405 May 13 15:16 registration.css
-rw-r--r-- 1 vboxuser vboxuser 11089 May 13 15:56 registration.html
-rw-r--r-- 1 vboxuser vboxuser 7902 May 19 15:18 registration.js
-rw-r--r-- 1 vboxuser vboxuser 31924 Apr 21 13:29 ss.css
-rw-r--r-- 1 vboxuser vboxuser 8542 Apr 22 13:22 style_help.css
-rw-r--r-- 1 vboxuser vboxuser 8611 Apr 22 13:39 style_policy.css
-rw-r--r-- 1 vboxuser vboxuser 16305 Apr 22 13:51 styles.css
root@VKR:/home/vboxuser# ls -l /home/vboxuser/Downloads/glav/images/
total 6176
-rw-r--r-- 1 vboxuser vboxuser 2333094 Apr 19 10:42 -101.svg
-rw-r--r-- 1 vboxuser vboxuser 1442 Apr 19 10:42 icon-15.svg
-rw-r--r-- 1 vboxuser vboxuser 207952 Apr 19 10:42 image-1-25.png
-rw-r--r-- 1 vboxuser vboxuser 1289 Apr 19 10:42 image-22-155.png
-rw-r--r-- 1 vboxuser vboxuser 1289 Apr 19 10:42 image-23-159.png
-rw-r--r-- 1 vboxuser vboxuser 1289 Apr 19 10:42 image-24-161.png
-rw-r--r-- 1 vboxuser vboxuser 1289 Apr 19 10:42 image-25-163.png
-rw-r--r-- 1 vboxuser vboxuser 171631 Apr 19 10:42 image-2-55.png
-rw-r--r-- 1 vboxuser vboxuser 1289 Apr 19 10:42 image-26-165.png
-rw-r--r-- 1 vboxuser vboxuser 1289 Apr 19 10:42 image-27-167.png

```

Рисунок 3.14 - Организация фронтенда

Каждая HTML - страница отвечает за отдельный функциональный экран системы:

- admin-dashboard.html – панель управления администратора (общая статистика, управление пользователями);
- admin-verify.html – страница верификации новых учителей и администраторов (подтверждение заявок);
- help.html – страница справки и поддержки (ответы на частые вопросы, контакты);
- index.html – стартовая страница с приветствием и общей информацией о школе;
- lich_kab_uchenika.html – личный кабинет ученика: отображается прогресс, список доступных заданий, результаты тестов;
- lich_kab_uchit.html – личный кабинет учителя: управление видеоуроками, создание тестов, просмотр результатов учеников;
- login.html и registration.html – формы входа и регистрации пользователей;
- password-reset.html – форма восстановления пароля;

- `pending-verification.html` – промежуточная страница для учётных записей, ожидающих подтверждения администратором;
- `me.html` – страница профиля пользователя (изменение личных данных);
- `politics.html` – страница с политикой конфиденциальности и пользовательским соглашением.

Такое разделение на отдельные HTML-файлы упрощает навигацию по проекту и позволяет независимо редактировать каждый экран. В будущем при переходе на SPA-фреймворк (например, React) эти файлы могут быть преобразованы в компоненты.

В директории присутствуют несколько файлов CSS, обеспечивающих оформление:

`login.css`, `registration.css`, `styles.css`, `ss.css`, `style_help.css`, `style_policy.css` – разделение стилей по страницам позволяет избежать конфликта имён классов и ускоряет загрузку (каждая страница подключает только необходимые ей стили). В `styles.css` содержатся общие правила для всех страниц (сброс отступов, типографика, адаптивная сетка), остальные файлы переопределяют или дополняют оформление для конкретных экранов.

JavaScript-скрипты:

`login.js`, `registration.js`, `password-reset.js` – реализуют логику отправки форм, валидацию ввода, обработку ответов сервера и сохранение JWT-токенов в `localStorage`.

Остальные страницы (например, `lich_kab_uchenika.html`) содержат встроенные скрипты или подключают дополнительные JavaScript-файлы (на скриншоте видно, что отдельных JS-файлов для личных кабинетов нет — код вписан прямо в HTML, что приемлемо для прототипа).

В подкаталоге `images/` хранятся графические ресурсы: логотипы, иконки, иллюстрации для страниц помощи и политики. Команда `ls -l images/` (также приведена на рисунке) показывает файлы:

- -101.svg – большой векторный фон или схема (более 2 МБ);
- icon-15.svg – иконка малого размера;
- image-1-25.png, image-2-55.png – растровые изображения (например, скриншоты интерфейса или иллюстрации);
- image-22-155.png, image-23-159.png и другие – серия иконок или превью.

Всего в папке более 10 файлов, занимающих около 6 МБ. Названия файлов сгенерированы автоматически (вероятно, при экспорте из графического редактора), но для работы прототипа это не критично.

Внутри `glav/` также присутствует подкаталог `backend/`, который содержит Python-скрипты бэкенда (`main.py`, `database.py`, `auth.py`, папку `routes/` и т.д.). Его наличие в одной директории с фронтендом удобно для разработки, так как вся система собрана в одном месте. Однако при развёртывании на сервере бэкенд и фронтенд обычно разделяют (фронтенд раздаётся через Nginx, бэкенд работает как отдельный сервис).

Файловая структура отражает итеративный подход к созданию веб-приложения: на начальном этапе использовались простые HTML-страницы с отдельными CSS/JS, что позволило быстро реализовать и протестировать все сценарии работы (регистрация, вход, личные кабинеты, панель администратора). В дальнейшем, при наращивании функциональности, планируется мигрировать на модульную структуру с использованием сборщика (Webpack/Vite) и фреймворка, но для дипломного проекта представленной организации кода достаточно для демонстрации всех возможностей системы.

3.5.2 Редактор кода

Для удобного ввода кода использована библиотека CodeMirror. Она подключается через CDN, инициализируется на элементе `<textarea>` и настраивается на режим Python, нумерацию строк и отступы. Дополнительно

реализована подсветка синтаксиса в реальном времени и автоматическое закрытие скобок.

3.5.3 Взаимодействие с бэкендом

Для всех запросов к API используется глобальный объект `api` на основе Fetch API. В нём автоматически подставляется заголовок `Authorization: Bearer <token>`, если токен сохранён в `localStorage`. Обработка ошибок централизована: при получении 401 Unauthorised пользователь перенаправляется на страницу входа, при 500 – выводится уведомление «Сервер временно недоступен».

Отправка решения происходит асинхронно: после нажатия кнопки «Проверить» код из редактора отправляется на сервер, отображается индикатор загрузки, а после получения ответа показывается вердикт (зелёным или красным) и текстовое пояснение от ИИ.

3.5.4 Адаптивный дизайн

Вёрстка выполнена с использованием CSS Grid и Flexbox. Медиа-запросы для ширины экрана менее 768 пикселей переключают расположение блоков (условие задачи сверху, редактор кода снизу) и уменьшают размер шрифта. Также реализована тёмная тема (переключение через JavaScript) – это повышает комфорт при длительной работе в вечернее время.

3.6 Реализация мониторинга и CI/CD

3.6.1 Мониторинг

Для сбора метрик в бэкенде использована библиотека `prometheus_client`. В `main.py` добавлено `middleware`, которое для каждого запроса увеличивает счётчик `http_requests_total` и записывает продолжительность в гистограмму

http_request_duration_seconds. По эндпоинту /metrics эти данные отдаются в формате, понятном Prometheus.

Сам Prometheus запущен в контейнере Docker с конфигурацией, указывающей на бэкенд как на цель. Также настроен сбор метрик с самого Prometheus (например, количество активных таргетов). Grafana подключена к тому же источнику данных; создан дашборд, на котором отображаются:

- количество запросов в секунду (RPS);
- 95-й и 99-й перцентили времени ответа;
- количество HTTP 5xx ошибок;
- использование памяти и CPU контейнерами (через cAdvisor).

3.6.2 CI/CD

Для обеспечения безопасности API-ключ YandexCloud не передавался на клиентскую сторону, а хранился на сервере. Запросы от фронтенда направлялись на серверный API-маршрут, который добавлял ключ авторизации и перенаправлял запрос к YandexGPT. Это исключало возможность несанкционированного доступа к ключу через инструменты разработчика в браузере.

В проекте реализована serverless-функция yandex-ai.js, размещённая в репозитории GitHub в папке api. Данная функция представляет собой API-маршрут для Next.js, который обрабатывает POST-запросы от клиента, добавляет необходимые параметры аутентификации и отправляет запрос к YandexGPT API.

Листинг 3.1 – yandex-ai.js

```
export default async function handler(req, res) {
  if (req.method !== 'POST') {
    return res.status(405).json({ error: 'Method not allowed' });
  }

  const YANDEX_API_KEY = "AQVN1216AIG-xflQRTV2mmBkZVtfktR4bRHSwXAP";
  const YANDEX_FOLDER_ID = "b1gst0fn81htpsdlgt6u";

  try {
```

```

const questionText = req.body.question || "привет";

const yandexBody = {
  modelUri: `gpt://${YANDEX_FOLDER_ID}/yandexgpt-lite`,
  completionOptions: {
    stream: false,
    temperature: 0.6,
    maxTokens: 500
  },
  messages: [
    {
      role: "system",
      text: "Ты ИИ-помощник ЗнатоК по информатике для ОГЭ. Отвечай кратко с эмодзи."
    },
    {
      role: "user",
      text: questionText
    }
  ]
};

const yandexResp = await
fetch('https://llm.api.cloud.yandex.net/foundationModels/v1/completion', {
  method: 'POST',
  headers: {
    'Authorization': `Api-Key ${YANDEX_API_KEY}`,
    'Content-Type': 'application/json'
  },
  body: JSON.stringify(yandexBody)
});

const data = await yandexResp.json();

return res.status(yandexResp.status).json(data);

} catch (error) {
  return res.status(500).json({ error: error.message });
}
}

```

3.7 Обеспечение безопасности

В системе реализован ряд мер безопасности:

- JWT-токены с коротким временем жизни (1 час). Refresh-токены хранятся в localStorage (для простоты) и позволяют обновлять access-токен без повторного ввода пароля;
- хеширование паролей с помощью bcrypt (соль, 12 раундов);

- шифрование трафика – в продакшне используется Ingress с сертификатом Let's Encrypt (HTTPS). На этапе разработки Ngrok предоставлял TLS-туннель;
- защита от SQL-инъекций – все запросы параметризованы через SQLAlchemy ORM;
- CORS – разрешены только доверенные источники (фронтенд на том же домене).

Безопасное выполнение кода – Judge0 обеспечивает полную изоляцию от серверной ОС (сэндбокс). Установлены жёсткие лимиты по времени и памяти.

3.8 Выводы по главе

Мы завершили разработку и запуск платформы онлайн-школы «Мост знаний». Инфраструктуру проекта собрали на базе VirtualBox, Docker и Kubernetes, что гарантирует стабильность, изоляцию и легкое масштабирование в будущем. Все данные — от профилей пользователей до их решений и прогресса — надёжно хранятся в базе PostgreSQL.

Серверную часть написали на FastAPI с четкой модульной структурой: она берет на себя всю бизнес-логику, авторизацию, кэширование и взаимодействие с внешними сервисами. Для учеников создали удобный адаптивный интерфейс с встроенным редактором кода CodeMirror, чтобы они могли решать задачи и мгновенно видеть результат.

Чтобы сделать процесс обучения эффективнее, мы подключили YandexGPT для генерации понятных разборов ошибок и Judge0 для безопасного выполнения пользовательского кода. Сам процесс разработки поддерживается автоматизированным CI/CD в GitHub, а за состоянием системы в реальном времени следят Prometheus и Grafana.

Безопасность продумана на всех этапах: от шифрования трафика до строгой изоляции среды выполнения. Сейчас платформа полностью готова к

пилотному тестированию с реальными школьниками и дальнейшему росту нагрузки.

4. БИЗНЕС-ПЛАН

4.1 Паспорт проекта

Главная цель разрабатываемого бизнес-проекта заключается в создании и выведении на рынок онлайн-платформы «Мост Знаний» для персонализированной подготовки школьников 12–18 лет к ОГЭ и ЕГЭ с использованием технологий искусственного интеллекта. Предлагаемый продукт представляет собой SaaS-платформу (веб-приложение), интегрированную с нейросетевыми моделями YandexGPT и системой выполнения кода Judge0.

Платформа «Мост Знаний» позволяет ученикам решать задания встроенного редактора кода, получать мгновенную автоматическую проверку решений, а также развернутые пояснения от ИИ-ассистента по допущенным ошибкам. Система анализирует историю решений каждого ученика, выявляет слабые темы и автоматически подбирает следующие задания для восполнения пробелов, формируя индивидуальную траекторию обучения.

Приложения имеют простой интерфейс, который удобен для всех пользователей, и предоставляют возможность сохранять историю прогресса в личном кабинете. В отличие от традиционных онлайн-школ, предлагающих массовые вебинары или дорогостоящие индивидуальные занятия с репетиторами, «Мост Знаний» обеспечивает:

- доступная цена от 990 руб./мес.;
- персонализация. ИИ подстраивается под уровень каждого ученика;
- мгновенная обратная связь. Проверка заданий и пояснение ошибок занимают не более 5 минут;
- масштабируемость. Платформа может обслуживать одновременно до 1000 пользователей.

Использование веб-приложения повышает скорость проверки заданий, снижает нагрузку на преподавателей, сокращает расходы на рабочую силу и

поддерживает благоприятный имидж компании. За счёт того, что проверка решений происходит автоматически в режиме реального времени, ученик не ждёт ответа от репетитора и может исправлять ошибки самостоятельно, что значительно ускоряет процесс обучения.

Проект ориентирован на два основных потребительских сегмента:

- B2C (частные лица) — школьники 12–18 лет и их родители, готовящиеся к ОГЭ/ЕГЭ по информатике;
- B2B (корпоративные клиенты) — образовательные учреждения (школы, колледжи, онлайн-школы) для интеграции платформы в учебный процесс.

Монетизация осуществляется по модели Freemium: бесплатный тариф с ограниченным количеством заданий и платные подписки «Про» (990 руб./мес.) и «Про+» (2 990 руб./мес.) с доступом к ИИ-ассистенту, расширенной аналитике и неограниченному количеству решений.

4.2 Описание отрасли

Анализ отрасли представляет собой процесс исследования конкретной сферы экономики с целью получения всесторонней информации о её внутренней организации, тенденциях развития, возможностях для расширения и потенциальных угрозах, которые могут оказать влияние на бизнес. Для проекта «Мост Знаний» проведена оценка внутренней и внешней среды, а также анализ ёмкости целевого рынка онлайн-подготовки к ОГЭ и ЕГЭ по информатике.

4.2.2 Оценка внешней среды (PEST-анализ)

PEST-анализ позволяет проанализировать возможные риски на несколько лет вперёд, оценивая влияние ключевых факторов внешней среды на компанию. В рамках данного анализа рассматриваются четыре основные группы факторов: политические, экономические, социальные и

технологические. Для оценки внешней среды проекта в таблице 4.1 представлен PEST-анализ.

Таблица 4.1 – PEST-анализ проекта

Политические	Экономические
1. Государственная программа «Цифровая экономика РФ» 2. Требования ФЗ-152 «О персональных данных» 3. Изменения в КИМ ОГЭ/ЕГЭ со стороны ФИПИ 4. Возможность получения налоговых льгот для IT-компаний (аккредитация)	1. Уровень инфляции и ключевой ставки ЦБ 2. Реальные располагаемые доходы населения 3. Стоимость привлечения трафика (Яндекс.Директ, VK) 4. Курс рубля (оплата Judge0 API в валюте) 5. Сезонность спроса на подготовку к экзаменам
Социальные	Технологические
1. Рост популярности онлайн-образования 2. Высокий спрос на подготовку по информатике 3. Тренд на персонализированное обучение 4. Доверие родителей к технологическим решениям 5. Усталость от массовых вебинаров	1. Развитие генеративных нейросетей 2. Доступность облачных платформ 3. Развитие технологий безопасного выполнения кода 4. Автоматизация процессов CI/CD 5. Совершенствование мобильных версий сайтов

Проведённый анализ обеспечил всестороннее понимание внешней среды и возможных будущих рисков. Ключевыми позитивными факторами являются государственная поддержка IT-отрасли и рост спроса на онлайн-образование. Основными рисками остаются инфляция, снижение доходов населения и возможное изменение формата экзаменов.

4.2.1 Оценка внутренней среды (SWOT анализ)

Для того, чтобы оценить внутренние и внешние факторы, которые могут повлиять на наш проект был проведен SWOT – анализ, показанный в таблице 4.2. С помощью анализа выявлены: текущее состояние проекта, а также определены его преимущества и недостатки. Анализ позволил так же выявить потенциальные угрозы, риски и возможности, что стало основой для

разработки стратегического плана по дальнейшему развитию проекта и повышению его курентноспособности на рынке

Таблица 4.2 – SWOT-анализ проекта

Сильные стороны	Слабые стороны
1. Уникальное ИИ-предложение (персонализация + объяснение ошибок) 2. Масштабируемость (в 10 раз больше учеников, чем у репетитора) 3. Быстрая и объективная проверка заданий с развёрнутым ответом	1. Низкое доверие к ИИ против живого репетитора 2. Малая раскрученность на фоне лидеров рынка 3. Зависимость от внешних платформ (Мах, ВКВидео)
Возможности	Угрозы
1. Господдержка и гранты на разработку 2. Дефицит учителей информатики в школах 3. Интеграция с Госуслугами / МЭШ 4. Расширение на другие предметы (математика, физика)	1. Появление ИИ-решений у крупных конкурентов 2. Изменение формата ЕГЭ 3. Низкая платёжеспособность в регионах

По итогам SWOT-анализа выявлены сильные и слабые стороны проекта, а также возможности и угрозы. Бальная оценка сильных и слабых сторон представлена в таблице 4.3.

Таблица 4.3 – Анализ сильных и слабых сторон проекта

Показатели	Степень важности (1–5)	Оценка конкурентоспособности				
Сильные стороны		1	2	3	4	5
1. Низкая стоимость подписки	5					*
2. Уникальное ЦП (ИИ-проверка)	5					*
3. Мгновенная обратная связь	4				*	
4. Персонализированная траектория	4				*	
5. Отсутствие привязки к расписанию	3			*		
6. Автоматическая проверка задания	4				*	
Итого по сильным сторонам		(5×5) +(5×5) +(4×4) +(4×4) +(3×3) +(4×4) = 25+25+16+16+9+16 = 107				
Слабые стороны						

Окончание таблицы 4.3

1. Отсутствие узнаваемого бренда	5					*
2. Только один предмет (информатика)	3			*		
3. Небольшой штат команды	3			*		
4. Зависимость от сторонних API	4				*	
5. Отсутствие образовательной лицензии	4				*	
6. Ограниченный бюджет на маркетинг	4				*	
Итого по слабым сторонам		$(5 \times 5) + (3 \times 3) + (3 \times 3) + (4 \times 4) + (4 \times 4) + (4 \times 4) = 25 + 9 + 9 + 16 + 16 + 16 = 91$				

Сравнение итоговых баллов (107 против 91) показывает, что сумма баллов за сильные стороны превосходит сумму баллов за слабые стороны, что свидетельствует о перспективности реализации бизнес-проекта. Основными конкурентными преимуществами являются низкая цена в сочетании с уникальной технологией ИИ-проверки кода.

4.2.3 Анализ рынка

Анализ рынка представляет собой переоценку экономического взаимодействия между производителями товаров и их потребителями. Анализ рынка заключается в исследовании показателей этого взаимодействия, таких как ёмкость рынка. Данный показатель позволяет определить потенциальную денежную выручку от продажи товаров или услуг в определённом сегменте рынка за конкретный период. Он включает три составляющие: потенциальный (TAM), фактический (SAM) и достижимый (SOM) объёмы рынка.

Расчёт ёмкости рынка для данного бизнес-проекта выполнен в соответствии с методологией TAM-SAM-SOM, показанный на рисунке 4.1. В расчётах используется цена подписки на тариф «Про» — 990 рублей в месяц. При этом целевая аудитория определена как школьники 8–11 классов, сдающие

ОГЭ или ЕГЭ по информатике. По данным Росстата и аналитики Smart Ranking, количество таких учеников в России составляет около 450 000 человек в год.

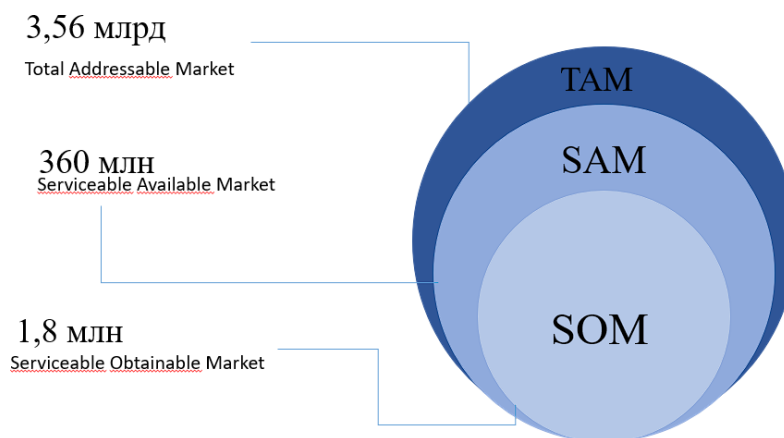


Рисунок 4.1 – Метод TAM-SAM-SOM

Показатель TAM (Total Addressable Market) — общий объём рынка, который потенциально может быть охвачен продуктом. Для проекта «Мост Знаний» TAM рассчитывается как произведение количества учеников, сдающих информатику, на средний чек подписки за учебный год. $TAM = 450\,000 \text{ человек} \times 990 \text{ руб./мес.} \times 8 \text{ месяцев} = 3\,564\,000\,000 \text{ рублей}$ (примерно 3,56 млрд руб. в год).

Показатель SAM (Serviceable Available Market) — доступный объём рынка, который компания может реально обслужить с учётом своей специализации и географии. На начальном этапе проекта фокус сделан на Архангельской области и близлежащих регионах (Северо-Западный федеральный округ). Количество школьников, сдающих информатику в СЗФО, составляет примерно 45 000 человек. $SAM = 45\,000 \times 990 \times 8 = 356\,400\,000 \text{ рублей}$.

Показатель SOM (Serviceable Obtainable Market) — реально достижимый объём рынка с учётом конкуренции и маркетинговых возможностей стартапа. Учитывая наличие крупных игроков («Умскул», «Фоксфорд», «Тетрика») и

ограниченный бюджет на продвижение, планируется занять 0,5% от SAM в первый год деятельности. $SOM = 356\,400\,000 \times 0,5\% = 1\,782\,000$ рублей за первый год. Данная цифра коррелирует с планом продаж, представленным в финансовой модели (выручка 902 150 руб. за 6 месяцев 2026 года и 4 920 700 руб. за 2027 год).

Таким образом, рыночный потенциал проекта оценивается как высокий. Даже при консервативном сценарии достижимый объём рынка составляет около 1,78 млн руб. в первый год с перспективой роста до 5–7 млн руб. на второй-третий год за счёт расширения географии и выхода на B2B-сегмент. Учитывая отсутствие прямых аналогов с функцией ИИ-проверки кода по информатике, можно прогнозировать, что разработанный продукт будет пользоваться стабильным спросом у целевой аудитории.

4.3 Маркетинговый план

Маркетинговый план представляет собой своеобразный GPS-навигатор по рынку и целевой аудитории проекта. Именно на этом этапе проводится детальный анализ конкурентов, сегментируется рынок и изучаются потенциальные потребители проекта. Также рассматриваются различные способы продаж и устанавливается цена на продукт бизнес-проекта.

4.3.1 Анализ конкурентов

Анализ конкурентов — это комплексное изучение деятельности компаний, которые предлагают аналогичные товары или услуги. Этот процесс позволяет стартапу лучше понять свое место на рынке, выявить преимущества и недостатки своих конкурентов, а также разработать эффективные стратегии для привлечения клиентов и удержания своей доли рынка.

Карта конкурентов представляет собой инструмент анализа, который помогает визуализировать конкурентную среду в отрасли или на рынке. Этот инструмент помогает выделить ключевых игроков, которые являются

ведущими участниками рынка, устанавливают стандарты и правила игры в отрасли. На рисунке 4.2 представлена карта конкурентов данного бизнес-проекта.

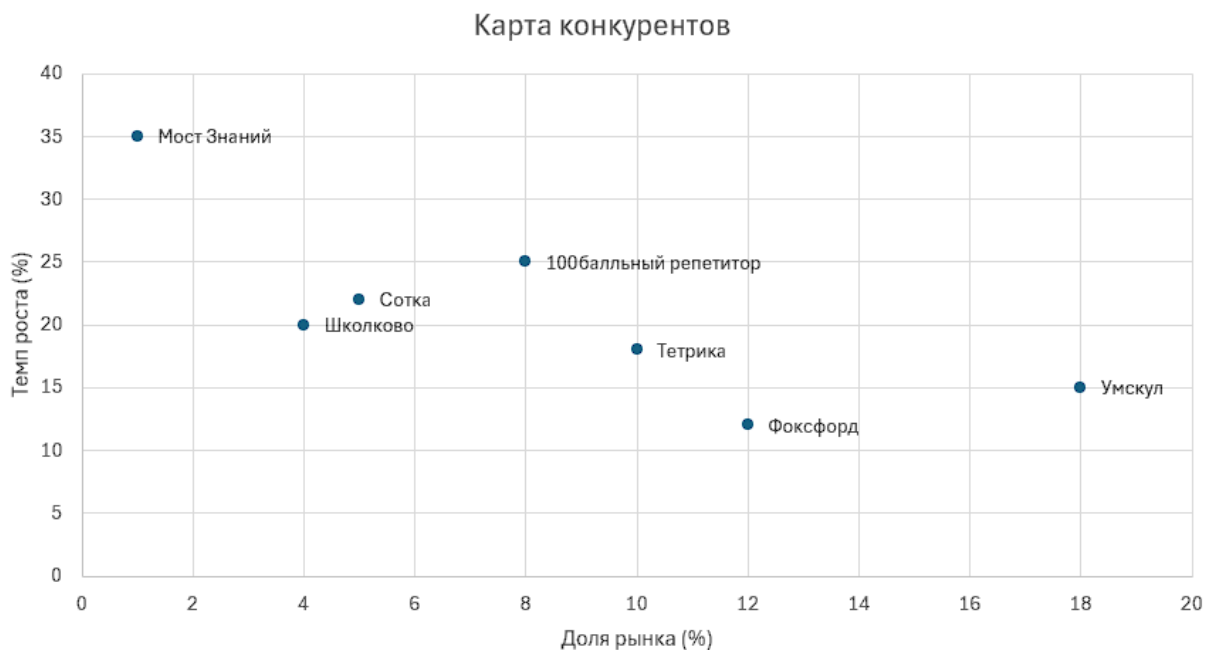


Рисунок 4.2 – Карта конкурентов

На основании карты можно сделать вывод, что «Умскул» и «Фоксфорд» представляют собой сильных конкурентов, чьё положение на рынке сложно оспорить. «Тетрика» и «100балльный репетитор» имеют значительную долю рынка при умеренном темпе роста. «Школково» и «Сотка» занимают небольшую долю рынка с невысоким темпом роста. Разрабатываемый бизнес-проект «Мост Знаний» располагается в левом верхнем углу карты: при небольшой доле рынка (около 1%) он демонстрирует высокий потенциальный темп роста (35%), что характерно для стартапов, выходящих на рынок с инновационным продуктом.

Карта конкурентов была построена на основе данных о доле рынка и темпах роста трех конкурирующих организаций, представленных в таблице 4.4.

Таблица 4.4 – Анализ конкурентов по двум критериям

Название	Доля рынка (%)	Темп роста (%)
Умскул	18	15
Фоксфорд	12	12
Тетрика	10	18
100балльный репетитор	8	25
Школково	4	20
Сотка	5	22
Мост Знаний	1	35

В таблице 4.5 проведено исследование сильных и слабых сторон проекта, на основании которых сделаны выводы о потенциале конкурентоспособности.

Таблица 4.5 – Информация о конкурентах

Предприятие	Сильные стороны	Слабые стороны	Выводы
«Умскул»	Крупная клиентская база (600 000+ выпускников), сильный бренд, преподаватели-звёзды, лицензия на образование	Слабая подготовка по информатике, агрессивные продажи, отсутствие проверки кода	Сильный конкурент в целом, но слабый именно по информатике. Возможен отток их клиентов
«Фоксфорд»	Широкая диверсификация, стабильность платформы, официальная аттестация, гибкость форматов	Неоднородное качество по предметам, отсутствие ИИ-проверки кода	Сильный конкурент по всем предметам, но слабый в нише автоматической проверки программирования
«Тетрика»	Высокое качество индивидуальных занятий (средний балл 82,3), высокое удержание (93,3%)	Высокая стоимость, зависимость от человеческого фактора, отсутствие ИИ-проверки кода	Альтернативный формат для B2C-сегмента с высоким бюджетом
«100балльный репетитор»	Высокие темпы роста (+118,4% в 2025 году), лидер по числу стобалльников	Отсутствие специализированного ИИ-инструмента для проверки кода	Растущий конкурент, требует мониторинга
«Школково» и «Сотка»	Заметные игроки на рынке, часто упоминаются в опросах пользователей	Не входят в топ-5 по выручке, отсутствие ИИ-технологий	Косвенные конкуренты, не оказывают существенного влияния

Окончание таблицы 4.5

Разрабатываемый бизнес-проект «Мост Знаний»	Низкая цена (990–1190 руб./мес.), ИИ-проверка кода с пояснениями, персонализированная траектория, работа 24/7	Отсутствие бренда, только информатика, небольшой бюджет на маркетинг, зависимость от сторонних API	После выхода на рынок и формирования клиентской базы проект сможет успешно конкурировать
---	---	--	--

Проанализируем бизнес-проект методом экспертной оценки в форме балльной системы. Этот метод позволяет более точно оценить уровень конкурентоспособности проекта.

В таблице 4.6 отражены оценочные баллы по критериям, используемым для сравнения конкурентов и данного бизнес-проекта.

Таблица 4.6 – Исходные данные

Фактор конкурентоспособности	Вес фактора	Разрабатываемый бизнес-проект, балл	«Умскул», балл	«Фоксфорд», балл	«Тетрика», балл
Качество подготовки по информатике	0,35	5	3	4	5
Стоимость услуг (доступность)	0,25	5	3	3	2
Технологическая инновационность	0,20	5	3	2	2
Удобство интерфейса	0,10	4	4	4	4
Узнаваемость бренда	0,10	1	5	5	5
Итого	1,00	20	18	18	18

В таблице 4.7 представлена оценка, полученная в результате расчетов с помощью экспертного метода.

Таблица 4.7 – Оценка организаций экспертным методом

Фактор конкурентоспособности	Разрабатываемый бизнес-проект, балл	«Умскул», балл	«Фоксфорд», балл	«Тетрика», балл
Качество подготовки по информатике	1,75	1,05	1,40	1,75
Стоимость услуг	1,25	0,75	0,75	0,50
Технологическая инновационность	1,00	0,60	0,40	0,40
Удобство интерфейса	0,40	0,40	0,40	0,40
Узнаваемость бренда	0,10	0,50	0,50	0,50
Итоговая оценка	4,50	3,30	3,45	3,55

Анализ показал, что разработанный проект обладает высокой конкурентоспособностью, поскольку общая оценка проекта (4,50 балла) является наивысшей. Это достигается за счёт высокого качества подготовки по информатике (благодаря ИИ-проверке кода), самой низкой стоимости среди конкурентов и высокой технологической инновационности.

Для визуализации оценки на рисунке 4.3 представлен многоугольник по ключевым факторам конкурентоспособности.



Рисунок 4.3 – Многоугольник конкурентоспособности

На рисунке 4.4 показана гистограмма, на которой наглядно представлены баллы по факторам конкурентоспособности организаций и разрабатываемого проекта.



Рисунок 4.4 – Диаграмма оценки факторов конкурентоспособности

Методы анализа конкурентов помогли определить положение на рынке, выявить сильные и слабые стороны конкурентов и оценить степень конкурентоспособности бизнес-проекта.

4.3.2 Сегментирование рынка и анализ потребителей

Сегментация рынка представляет собой процесс дробления целевой аудитории на группы с одинаковыми характеристиками и потребностями. Это помогает компаниям определить свое положение на рынке и увеличить объемы продаж, учитывая различные группы потребителей при разработке продукции.

Сегментирование рынка данного бизнес-проекта разделяется на две категории: по географии и требованиям потребителей. География целевой аудитории ограничена территорией Российской Федерации с фокусом на начальном этапе на Архангельскую область и Северо-Западный федеральный округ. В дальнейшем планируется масштабирование на всю страну.

Сегментация по требованиям потребителей выделяет два ключевых сегмента. Первый сегмент — B2C (частные лица) — это школьники 12–18 лет

(8–11 классы), готовящиеся к сдаче ОГЭ и ЕГЭ по информатике, а также их родители, которые принимают решение об оплате обучения.

Данный сегмент характеризуется высокой чувствительностью к цене, потребностью в персонализированном подходе и мгновенной обратной связи при решении заданий. Внутри B2C-сегмента дополнительно выделяются подгруппы по классу обучения: ученики 8–9 классов (подготовка к ОГЭ) — наиболее многочисленная группа, но с меньшей интенсивностью подготовки и более низкой готовностью платить; ученики 10–11 классов (подготовка к ЕГЭ) — имеют более высокую мотивацию и платёжеспособность, их количество меньше, но они готовы платить за качественную подготовку; а также ученики 6–7 классов, начинающих подготовку заранее — это растущий тренд на рынке, который позволяет удерживать клиента на протяжении нескольких лет.

Второй сегмент B2B – это общеобразовательные школы, колледжи, а также другие онлайн-школы, которые могут интегрировать платформу «Мост Знаний» в свой учебный процесс. Для этого сегмента важны надёжность работы платформы, возможность брендинга, наличие развёрнутой отчётности об успеваемости учеников для администрации и преподавателей, а также соответствие требованиям законодательства о защите персональных данных. B2B-сегмент менее чувствителен к цене, но предъявляет более высокие требования к функциональности и юридическому оформлению (договоры, лицензии).

Портрет типичного потребителя B2C-сегмента: школьник 16–17 лет (11 класс), планирующий поступать на IT-специальность, с уровнем дохода семьи средним или выше среднего, испытывающий трудности с самостоятельным решением задач по программированию, уставший от длительных вебинаров и ищущий эффективный и недорогой способ подготовки. Родитель такого ученика ценит прозрачность прогресса, экономию семейного бюджета и доверяет технологическим решениям, подкреплённым положительными

отзывами. Для B2B-сегмента портрет типичного потребителя: руководитель IT-отдела или заместитель директора школы по цифровизации, отвечающий за внедрение образовательных технологий, для которого важны простота интеграции, наличие технической поддержки и возможность генерации отчётов об успеваемости.

4.3.3 Анализ способов продаж товаров и ценообразование

Продвижение продукта играет ключевую роль в запуске и развитии бизнеса. Оно включает мероприятия для привлечения внимания целевой аудитории и создания положительного имиджа продукта. Эффективное продвижение помогает компании выделиться среди конкурентов, установить связи с клиентами и обеспечить рост. В современных условиях требуется сочетание традиционных и онлайн-методов продвижения.

Основные способы продвижения наших продуктов: персональная презентация продуктов потенциальным B2B-клиентам, обзвон потенциальных клиентов (холодные звонки), таргетированная реклама в социальных сетях, контекстная реклама в поисковых системах, а также размещение информации о продукте на образовательных форумах и в профильных сообществах.

Первоначальные цены на старте проекта: бесплатный тариф с ограниченным доступом к заданиям (до 10 заданий в день, без доступа к ИИ-ассистенту), платный тариф «Про» — 990 рублей в месяц (неограниченное количество заданий, доступ к ИИ-ассистенту с проверкой кода и пояснениями ошибок, расширенная аналитика прогресса), платный тариф «Про+» — 2 990 рублей в месяц (включает все возможности тарифа «Про», а также до 5 обращений к преподавателю в чате для разбора сложных кейсов и доступ к закрытым вебинарам). При оплате за полгода предоставляется скидка 15%, за год — 20%. Для B2B-сегмента (школы, колледжи, другие онлайн-школы) стоимость интеграции сайта в учебный процесс составляет от 5 000 рублей в месяц в зависимости от количества учеников и требуемого функционала. При

заключении договора на год предоставляется скидка 10–20% в зависимости от объёма.

В течение первого года после запуска запланирована доработка проекта, в том числе расширение базы заданий, увеличение количества распознаваемых типов ошибок и улучшение качества работы ИИ-ассистента. Это может привести к повышению цен через год после запуска. Ожидается, что ежемесячная подписка на тариф «Про» может составить до 1 500 рублей, а стоимость годового доступа для B2B-сегмента — до 60 000 рублей за организацию.

Обоснование выбранной ценовой политики основывается на анализе конкурентов. Средняя стоимость индивидуальных занятий с репетитором по информатике составляет 1 500–2 500 рублей за одно занятие (60 минут). Годовые курсы в онлайн-школах («Умскул», «Фоксфорд») стоят 25 000–70 000 рублей за предмет в год (что составляет 2 000–6 000 рублей в месяц). Проект «Мост Знаний» предлагает цену 990–2 990 рублей в месяц, что в 2–6 раз дешевле конкурентов, при этом предоставляя не уступающее, а по ряду параметров (мгновенная проверка кода, персонализация) превосходящее качество подготовки. Низкая цена в сочетании с уникальной технологией ИИ-проверки кода является ключевым конкурентным преимуществом и основным драйвером привлечения клиентов.

4.4 Организационный план

Организационный план описывает внутренне устройство предприятия, управление персоналом и график реализации разрабатываемого проекта или бизнес-плана. Что позволяет осуществлять контроль выполнения задач и позволяет постоянно отслеживать текущее состояние и динамику развития.

Для реализации проекта нами выбрана организационно-правовая форма компании – ООО (общество с ограниченной ответственностью).

ООО предоставляет возможность учредителям объединить свои ресурсы и совместно вести бизнес, что способствует более устойчивому развитию компании. Так же участники ООО не рискуют своим личным имуществом, а только вложенными в уставный капитал средствами, что снижает личные финансовые риски учредителей. ООО обладает более высоким уровнем доверия со стороны партнеров и инвесторов по сравнению с индивидуальными предпринимателями, что может способствовать привлечению инвестиций и установлению деловых связей.

Все это делает выбранную нами форму регистрации, наиболее подходящей для нашего проекта.

Для регистрации предприятия государственную пошлину в размере 4000 рублей платить не будем, так как документы направлены в электронном виде.

Регистрация ООО запланирована на июнь 2026 года, запуск сайта запланирован на июль 2026 года.

Размер уставного капитала (УК) фиксируем при создании организации и прописываем в Уставе в размере 10000 рублей. Уплата производится равными долями между всеми участниками ООО на счет предприятия.

Для ООО будет применяться упрощенная система налогообложения (УСН). Упрощённая система налогообложения (УСН) — это специальный налоговый режим для бизнеса, освобождающий от уплаты налога на прибыль, НДС и налога на имущество. Он позволяет платить всего один налог вместо основных.

УСН может быть двух видов:

- «Доходы»: Ставка составляет (6%). Налог платится со всей полученной выручки. Региональные власти могут снижать ставку до (1%);
- «Доходы минус расходы»: Ставка составляет (15%). Налог платится с чистой прибыли (разницы между выручкой и затратами). Регионы вправе уменьшать её до (5%).

Организация соответствует всем требованиям УСН:

- численность сотрудников менее 130 человек;
- доход должен быть не более 490,5 млн рублей (с учетом коэффициента-дефлятора за 2026 год);
- отсутствие филиалов;
- остаточная стоимость основных средств — не более 150 млн рублей;
- доля участия других компаний в уставном капитале ООО на УСН — не больше 25%.

Объектом налогообложения выбраны доходы. Налог выплачивается по ставке 6%.

На начальном этапе работы не планируется привлечение дополнительного наёмного персонала.

Штат сотрудников показан в таблице 4.8.

Таблицы 4.8 – Штат сотрудников

Должность	Количество	Оклад	РК и СН	Итого ФОТ
Директор	1	27093	18965,10	46058,10
Программист-разработчик (0,5 ст)	1	13547	9482,90	23029,90
Программист-разработчик (0,5 ст)	1	13547	9482,90	23029,90
Программист-разработчик (0,5 ст)	1	13547	9482,90	23029,90
Итого	4	67734	47413,80	115147,80

На начальном этапе штат сотрудников будет включать в себя:

- директора (совмещает обязанности по управлению компанией и менеджмент). Основной задачей директора является стратегическое планирование, организация бизнес-процессов, в том числе подготовка рекламных материалов и прямые продажи продукта);

– программист-разработчик. Оформлены по совместительству на 4-часовой рабочий день. Основными задачами программиста разработчика является разработка ключевых компонентов сервиса, участие в обсуждениях доработок платформы по запросу клиента, доработка функционала платформы, настройка интеграций со сторонними сервисами, а также создание и обновление веб-дизайна сайта и других маркетинговых материалов.

Ведение бухгалтерского учета и отчетности будет передано специализированной компании, что позволит снизить затраты на содержание штатного бухгалтера.

В дальнейшем планируется расширение команды дополнительными преподавателями, оформленными через агентские договора. Что позволит нам работать с преподавателями, как с физическими лицами и не являться для них налоговым агентом. Агентский договор дает нам право платить налог только с суммы своего фактического дохода, т.е. платить налог с разницы между суммами, полученными от учеников и суммами, перечисленными преподавателям.

4.5 Производственный план

В таблице 4.9 представлен календарный план, в котором отражены основные этапы по реализации проекта и сроки их завершения.

Таблица 4.9 – Календарный план

№	Наименование этапа	Дата начала	Дата окончания
1	Анализ рынка и предметной области	01.10.2025	28.10.2025
2	Разработка технического задания	29.10.2025	07.11.2025
3	Определение инструментов и технологий разработки	08.11.2025	12.11.2025

Окончание таблицы 4.9

4	Проектирование архитектуры и базы данных	13.11.2025	25.11.2025
5	Разработка и настройка сайта	26.11.2025	15.02.2026
6	Тестирование программного обеспечения	16.02.2026	23.02.2026
7	Настройка резервного копирования и документация	24.02.2026	01.03.2026
8	Развёртывание и запуск пилотной версии	02.03.2026	05.03.2026
9	Составление бизнес-плана	06.03.2026	25.05.2026
10	Подготовка документов для регистрации ООО	26.05.2026	22.06.2026
11	Регистрация ООО	23.06.2026	27.06.2026
12	Запуск сайта онлайн школы	01.07.2026	31.07.2026

Для визуализации календарного плана используется диаграмма Ганта, представленная на рисунке 4.5.

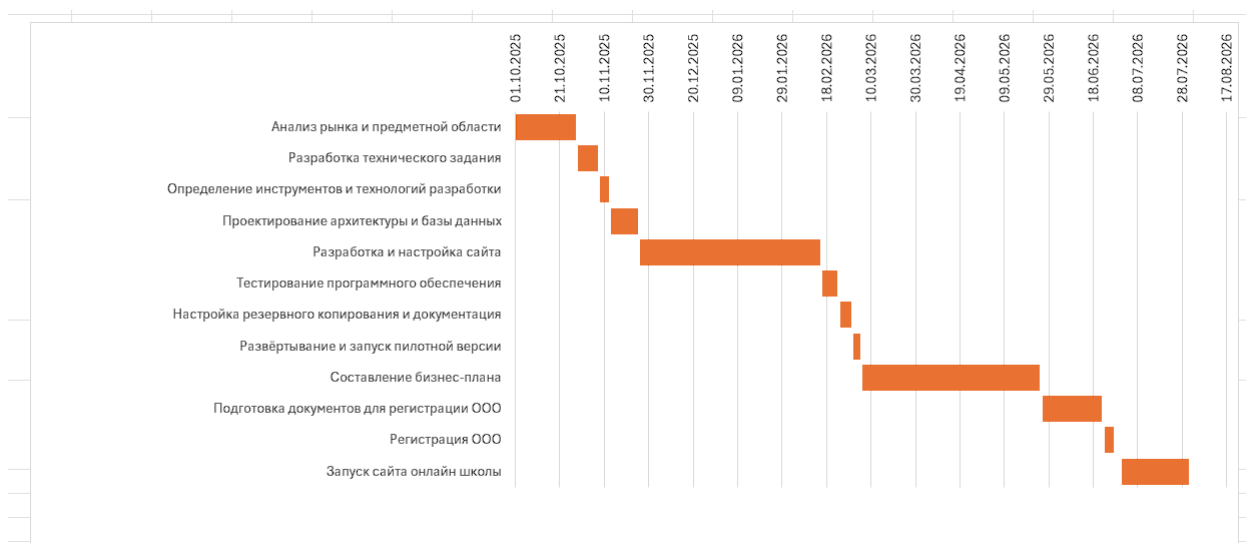


Рисунок 4.5 – Диаграмма Ганта

Производственный план позволяет контролировать и улучшать организацию работы, что в свою очередь оптимизировало сроки работы по проекту.

4.6 Финансовая модель проекта

Финансовая модель описывает текущие и прогнозируемые финансовые операции компании или бизнес-проекта. Он включает в себя множество аспектов, таких как доходы, расходы, и служит основой для принятия финансовых решений и стратегического планирования.

У нас в данном разделе показаны основные финансовые показатели проекта, такие, как издержки компании, предполагаемый срок окупаемости и чистая прибыль.

Таблица 4.10 Инвестиции

Источник инвестирования	Сумма	Пояснение
Собственные вложения	16000	Регистрация ООО, интернет-связь
Итого	16000	

Для работы над разрабатываемым проектом не требуется аренда помещения или покупка дополнительного оборудования.

В рамках единовременных затрат необходимо рассматривать расходы в размере 10000 рублей при регистрации ООО и расходы на интернет и услуги связи при разработке проекта. Постоянные затраты показаны в таблице 4.11.

Затраты, которые относятся к постоянным:

- расходы на интернет и услуг связи;
- заработная плата сотрудников (МРОТ + районный коэффициент и надбавки) с отчислениями в Социальный фонд России (СФР);
- затраты на рекламу и продвижение сайта.

Таблица 4.11 – Постоянные затраты

Показатель	Сумма
Реклама/Продвижение	3000
Интернет-услуги и связь	2000
ФЗП с СФР	132419,97
Итого	137419,97

К переменным затратам, показанная в таблице 4.12, относится налог по упрощенной системе налогообложения (УСН), а также бухгалтерские услуги.

Таблица 4.12 – Переменные затраты

Показатель	Сумма
Налоги (УСН)	2970
Бухгалтерские услуги	5000
Итого	7970

Районный коэффициент в г. Архангельске равен 0,2, а «северные» надбавки – 50 %. В 2026 году МРОТ составляет 27093 рубля. В 2026 году при применении УСН страховые взносы за работников уплачиваются по единому тарифу: 15% с выплат в пределах предельной базы (2 979 000 руб. на человека) и 7,6% свыше этой суммы (регистрация как IT-компания).

Регистрация ООО запланирована на июнь 2026 года, старт онлайн школы запланирован с июля 2026 гола.

В таблицах 4.13 и 4.14 представлен финансовый план проекта на первые два года.

Справочная информация:

- 1-й квартал: январь, февраль, март
- 2-й квартал: апрель, май, июнь
- 3-й квартал: июль, август, сентябрь
- 4-й квартал: октябрь, ноябрь, декабрь.

Финансовый план по кварталам за 2026 год представлен в таблице 4.13, финансовая модель проекта по месяцам за 2026 год – в таблице Б.3 в приложении Б. Так же планы по поступлению от реализации за 2026 – 2027 года предоставлены по месяцам в таблицах Б.1 и Б.2 в приложении Б.

Таблица 4.13 – Финансовый план за 2026 год

Показатели	0-й период	3-й квартал	4-й квартал	Итого
Поступления	16000	165350	736800	918150
Вложения собственника	16000	-	-	
Поступления от реализации	-	165350	736800	902150
Платежи	16000	437180,91	471467,91	908648,82
Единовременные затраты	16000	-	-	16000
Оформление ООО	10000	-	-	10000
Интернет и услуги связи	6000			6000
Постоянные затраты	-	412259,91	412259,91	824519,82
Реклама и продвижение сайта	-	9000	9000	18000
Интернет и услуги связи	-	6000	6000	12000
Заработная плата	-	345 443,4	345 443,4	690 886,8
Отчисления в СФР		51816,51	51816,51	103 633,02
Переменные затраты	-	24921	59208	84129
УСН (6% с дохода)	-	9921	44208	54129
Бухгалтерские услуги		15000	15000	30000
Поступления - платежи	0	-271830,91	265332,09	9501,18
Поступления и платежи нарастающим итогом	0	-271830,91	-6498,82	

По результатам отчета за первый 2026 год сумма чистой прибыли равняется 9501,18 но это нормальный показатель для начала проекта. В первый год идёт период масштабирования и вливание на рынок аналогичных проектов.

По небольшой сумме чистой прибыли по концу года мы все же видим, что покрыли основные издержки на операционные расходы со старта проекта

и в целом за счет увеличения поступлений к концу года мы можем четко понимать, что проект приносит прибыль.

В следующий 2027 год не планируется повышение стоимости на подписки, а прибыль планируем получать за счет раскрутки и продвижения сайта и увеличение количества пользователей.

В таблице 4.14 представлен финансовый план проекта на следующий 2027 год.

Таблица 4.14 – Финансовый план за 2-й год (2027 год)

Показатели	1-й квартал	2-квартал	3-й квартал	4-й квартал	Итого
Поступления	1473600	1788450	569400	1089250	4920700
Поступления от реализации	1473600	1788450	569400	1089250	4920700
Платежи	515675,91	423566,91	461423,91	487614,91	2004281,64
Постоянные затраты	412259,91	412259,91	412259,91	412259,91	1649039,64
Реклама и продвижение сайта	9000	9000	9000	9000	36000
Интернет и услуги связи	6000	6000	6000	6000	24000
Заработная плата	345 443,4	345 443,4	345 443,4	345 443,4	1381773,6
Отчисления в СФР	51816,51	51816,51	51816,51	51816,51	207266,04
Переменные затраты	103416	122307	49164	80355	355242
УСН (6% с дохода)	88415	107307	34164	65355	295242
Бухгалтерские услуги	15000	15000	15000	15000	60000
Поступления - платежи	957924,09	1263883,09	107976,09	601635,09	2931418,36
Поступления и платежи нарастающим итогом	957924,09	2221807,18	2329783,27	2931418,36	

Финансовая модель проекта по кварталам за 2027 год – в таблице Б.4 в приложении Б.

По результатам таблицы 4.14 за 2027 год мы видим, что за счет увеличения количества пользователей сайта и соответственно роста продаж пакетов и соответственно постоянные расходы распределяются уже на большее количество клиентов, рентабельность начинает расти.

Так как у нас проект онлайн школы, то все-таки существуют не значительные риски по сезонному снижению продаж пакетов. Но за счет стабильной прибыли в остальные месяца мы можем контролировать ситуацию.

Рентабельность проекта по чистой прибыли определяется по формуле 4.1:

$$P = \frac{ЧП}{В} \cdot 100\%, \quad (4.1)$$

где ЧП – чистая прибыль, руб.;

В – выручка, руб.

P_1 – рентабельность проекта за 2026 год.

$$P_1 = \frac{9501,18}{918150} \cdot 100\% \approx 0,01\%.$$

Нулевая рентабельность на старте у нас получилась за счет операционных расходов на раскрутку и продвижение сайта, а также расходов на заработную плату ключевых сотрудников компании.

В последующий год мы видим, что за счет увеличения количества пользователей сайта и соответственно роста продаж пакетов и соответственно постоянные расходы распределяются уже на большее количество клиентов рентабельность начинает расти.

Общая сумма чистой прибыли за 2027 год деятельности составляет 2931418,36 рублей.

P_2 – рентабельность проекта за 2027 год.

$$P_2 = \frac{2931418,36}{4920700} \cdot 100\% \approx 59,57\%.$$

За 2027 год рентабельность проекта выросла, значит увеличились доходы.

Срок окупаемости вложений собственника определяется по формуле

$$T = \frac{K}{\text{ЧП}}, \quad (4.2)$$

где К – размер вложений, руб.

$$T = \frac{16000}{9501,18} \approx 1,68 \text{ лет.}$$

Срок окупаемости составляет около 1 года.

4.7 Оценка рисков

В таблице 4.15 представлена информация о внешних угрозах, которые способны нанести ущерб компании.

Таблица 4.15 – Внешние угрозы проекта

№	Риск	Вероятность возникновения, %	Размер потенциального ущерба, руб.	Размер ожидаемого ущерба, руб.	Структура возможного ущерба, %
1	Кибератаки и утечка данных	15	500 000	75 000	12
2	Изменения в законодательстве (ФЗ-152, образовательные стандарты)	25	200 000	50 000	8
3	Экономическая нестабильность (инфляция, снижение доходов)	60	350 000	210 000	29
4	Высокая конкуренция (появление новых игроков)	70	250 000	175 000	24
5	Невостребованность продукта из-за низкого качества	10	400 000	40 000	6

Окончание таблицы 4.15

6	Изменение формата ОГЭ/ЕГЭ со стороны ФИПИ	30	150 000	45 000	7
7	Сбои в работе сторонних API (YandexGPT, Judge0)	20	100 000	20 000	3
8	Падение спроса в летний период (сезонность)	80	100 000	80 000	11
Итого			2 050 000	695 000	100

Для визуализации данных о внешних угрозах используется матрица внешних угроз, представленная на рисунке 4.6. Матрица построена на основе двух параметров: вероятность возникновения риска (ось Y) и размер ожидаемого ущерба (ось X). Это позволяет выделить наиболее критичные риски, требующие первоочередного внимания.



Рисунок 4.6 – Матрица внешних угроз

Таблица 4.16 содержит информацию о внутренних слабостях бизнес-проекта.

Таблица 4.16 – Слабости проекта

№	Риск	Вероятность возникновения, %	Размер потенциального ущерба, руб.	Размер ожидаемого ущерба, руб.	Структура возможного ущерба, %
1	Сбои в работе оборудования и серверов	25	200 000	50 000	19
2	Несоблюдение договорных обязательств перед клиентами	10	150 000	15 000	6
3	Выявление дополнительных затрат на разработку	40	200 000	80 000	30
4	Просчеты в маркетинговой стратегии	35	150 000	52 500	20
5	Нехватка квалифицированных кадров	20	120 000	24 000	9
6	Ошибки в работе ИИ-ассистента	30	150 000	45 000	17
7	Отсутствие обратной связи от пользователей	25	100 000	25 000	9
Итого			1 070 000	266 500	100

Для визуализации данных о внутренних слабостях используется матрица внутренних угроз, показанная на рисунке 4.7.



Рисунок 4.7 – Матрица внутренних угроз

Анализ внутренних угроз показывает, что наиболее значимыми рисками для проекта являются выявление дополнительных затрат на разработку. Эти риски требуют разработки превентивных мероприятий. Также существенное влияние могут оказать сбои в работе оборудования и ошибки в работе ИИ-ассистента, что особенно важно для проекта, где качество работы ИИ является ключевым конкурентным преимуществом.

Таким образом, проведенный анализ рисков позволил выявить основные угрозы для проекта «Мост Знаний». Наиболее критичными для проекта являются экономическая нестабильность и высокая конкуренция среди внешних угроз, а также выявление дополнительных затрат на разработку и просчеты в маркетинговой стратегии среди внутренних. Однако уникальное технологическое преимущество в виде ИИ-проверки кода и доступная цена позволяют проекту сохранять конкурентоспособность даже при наступлении указанных рисков.

ЗАКЛЮЧЕНИЕ

В результате выполнения выпускной квалификационной работы достигнута поставленная цель – разработана концепция и экономическое обоснование создания онлайн-платформы (сайта) с искусственным интеллектом «Мост Знаний» для персонализированного обучения и подготовки к экзаменам по информатике.

В ходе работы был проведен анализ текущего состояния рынка онлайн-образования и адаптивных образовательных платформ в Российской Федерации.

Исследованы существующие аналоги и выявлены их недостатки. Проведён сравнительный анализ ключевых конкурентов.

Разработана концепция SaaS-продукта, включающая архитектуру технологического стека. Обоснован выбор следующих технологий: бэкенд на Python с использованием фреймворка FastAPI, фронтенд на HTML, CSS и чистом JavaScript, база данных PostgreSQL, интеграция с YandexGPT API для реализации ИИ-ассистента и Judge0 API для безопасного выполнения пользовательского кода. Для контейнеризации используется Docker, для оркестрации — Kubernetes, мониторинг обеспечивается стеком Prometheus и Grafana.

Разработана бизнес-модель и финансовый план стартап-проекта. Рассчитаны ключевые показатели эффективности.

Составлен план маркетинговых мероприятий и проанализированы риски проекта. Проведена оценка внешних и внутренних рисков, для каждого из рисков разработаны меры по их минимизации и превентивному воздействию.

Разработанный IT-продукт — сайт «Мост Знаний» с ИИ-ассистентом позволяет автоматически выявлять слабые места ученика и строить индивидуальную траекторию обучения, повышая эффективность подготовки по сравнению с традиционными методами.

Результаты работы могут быть использованы для привлечения инвестиций в стартап, а также в качестве дорожной карты при разработке MVP-версии продукта с последующим выводом на рынок. Разработанные финансовые и маркетинговые планы подтверждают экономическую целесообразность реализации проекта, а также его конкурентоспособность и перспективность в условиях растущего рынка онлайн-образования.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Smart Ranking. Рынок онлайн-образования в России: аналитические материалы [Электронный ресурс]. – Режим доступа: <https://smartranking.ru>, свободный. – Дата обращения: 13.06.2026.
2. Умскул : официальный сайт [Электронный ресурс]. – Режим доступа: <https://umskul.ru>, свободный. – Дата обращения: 13.06.2026.
3. Тетрика : официальный сайт [Электронный ресурс]. – Режим доступа: <https://tetrika-school.ru>, свободный. – Дата обращения: 13.06.2026.
4. Фоксфорд : официальный сайт [Электронный ресурс]. – Режим доступа: <https://foxford.ru>, свободный. – Дата обращения: 13.06.2026.
5. Debian Documentation [Электронный ресурс]. – Режим доступа: <https://www.debian.org/doc>, свободный. – Дата обращения: 13.06.2026.
6. Oracle VM VirtualBox Documentation [Электронный ресурс]. – Режим доступа: <https://www.virtualbox.org/wiki/Documentation>, свободный. – Дата обращения: 13.06.2026.
7. Python Documentation [Электронный ресурс]. – Режим доступа: <https://docs.python.org>, свободный. – Дата обращения: 13.06.2026.
8. MDN Web Docs. HTML Documentation [Электронный ресурс]. – Режим доступа: <https://developer.mozilla.org>, свободный. – Дата обращения: 13.06.2026.
9. MDN Web Docs. CSS Documentation [Электронный ресурс]. – Режим доступа: <https://developer.mozilla.org>, свободный. – Дата обращения: 13.06.2026.

- 10.MDN Web Docs. JavaScript Documentation [Электронный ресурс]. – Режим доступа: <https://developer.mozilla.org>, свободный. – Дата обращения: 13.06.2026.
- 11.FastAPI Documentation [Электронный ресурс]. – Режим доступа: <https://fastapi.tiangolo.com>, свободный. – Дата обращения: 13.06.2026.
- 12.Swagger Documentation [Электронный ресурс]. – Режим доступа: <https://swagger.io/docs>, свободный. – Дата обращения: 13.06.2026.
- 13.ReDoc Documentation [Электронный ресурс]. – Режим доступа: <https://redocly.com/docs>, свободный. – Дата обращения: 13.06.2026.
- 14.OpenAPI Specification [Электронный ресурс]. – Режим доступа: <https://spec.openapis.org>, свободный. – Дата обращения: 13.06.2026.
- 15.PostgreSQL Documentation [Электронный ресурс]. – Режим доступа: <https://www.postgresql.org/docs>, свободный. – Дата обращения: 13.06.2026.
- 16.YandexGPT: документация [Электронный ресурс] // Yandex Cloud. – Режим доступа: <https://yandex.cloud/ru/docs/foundation-models>, свободный. – Дата обращения: 13.06.2026.
- 17.yandex-cloud-ml-sdk Documentation [Электронный ресурс]. – Режим доступа: <https://github.com/yandex-cloud/yandex-cloud-ml-sdk>, свободный. – Дата обращения: 13.06.2026.
- 18.Judge0 Documentation [Электронный ресурс]. – Режим доступа: <https://judge0.com>, свободный. – Дата обращения: 13.06.2026.
- 19.Ngrok Documentation [Электронный ресурс]. – Режим доступа: <https://ngrok.com/docs>, свободный. – Дата обращения: 13.06.2026.
- 20.Docker Documentation [Электронный ресурс]. – Режим доступа: <https://docs.docker.com>, свободный. – Дата обращения: 13.06.2026.

21. Kubernetes Documentation [Электронный ресурс]. – Режим доступа: <https://kubernetes.io/docs>, свободный. – Дата обращения: 13.06.2026.
22. Grafana Documentation [Электронный ресурс]. – Режим доступа: <https://grafana.com/docs>, свободный. – Дата обращения: 13.06.2026.
23. Prometheus Documentation [Электронный ресурс]. – Режим доступа: <https://prometheus.io/docs>, свободный. – Дата обращения: 13.06.2026.
24. GitHub Actions Documentation [Электронный ресурс]. – Режим доступа: <https://docs.github.com/actions>, свободный. – Дата обращения: 13.06.2026.
25. JWT Introduction [Электронный ресурс]. – Режим доступа: <https://jwt.io/introduction>, свободный. – Дата обращения: 13.06.2026.
26. Redis Documentation [Электронный ресурс]. – Режим доступа: <https://redis.io/docs>, свободный. – Дата обращения: 13.06.2026.
27. Упрощенная система налогообложения [Электронный ресурс] // Федеральная налоговая служба. – Режим доступа: <https://www.nalog.gov.ru/rn77/taxation/taxes/usn/>, свободный. – Дата обращения: 13.06.2026.
28. Захарова О. УСН 6% или 15%: что выгоднее выбрать в 2024 году [Электронный ресурс] // Моё дело. – Режим доступа: <https://www.moedelo.org/club/article-knowledge/usn-6-i-15-cto-vygodnee>, свободный. – Дата обращения: 13.06.2026.
29. Рудевич А. УСН в 2026 году: подробный разбор упрощенной системы налогообложения для бизнеса [Электронный ресурс] // REG.RU. – Режим доступа: <https://www.reg.ru/blog/uproshchennaya-sistema-nalogooblozheniya-usn/>, свободный. – Дата обращения: 13.06.2026.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг программного кода

Листинг А.1 - lich_kab_uchenika.html

```
from fastapi import FastAPI, HTTPException, Depends
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel, EmailStr, Field
from typing import Optional
import psycopg2
from passlib.context import CryptContext
import re
import json
import requests
from recommender import recommender
from fastapi.staticfiles import StaticFiles
import os
from fastapi.responses import FileResponse

# ===== ОЧИСТКА LATEX СИМВОЛОВ =====
def clean_latex_symbols(text: str) -> str:
    """Убирает LaTeX-разметку $...$ из текста"""
    if not text:
        return text
    text = re.sub(r'\$([^\$]+)\$', r'\1', text)
    text = text.replace('$', '')
    return text
```

```
app = FastAPI(title="Мост Знаний API")

# ===== СТАТИЧЕСКИЕ ФАЙЛЫ (для открытия HTML через браузер) =====
from fastapi.staticfiles import StaticFiles
from fastapi.responses import FileResponse
import os

app.mount("/static",
StaticFiles(directory="/home/vboxuser/Downloads/glav/"), name="static")

@app.get("/teacher")
async def teacher_page():
    return
FileResponse("/home/vboxuser/Downloads/glav/lich_kab_uchit.html")

@app.get("/student")
async def student_page():
    return
FileResponse("/home/vboxuser/Downloads/glav/lich_kab_uchenika.html")
```

```

# ===== HEALTH CHECK =====
@app.get("/api/health")
async def health_check():
    return {"status": "ok", "service":
"online_school_api"}

app.mount(
    "/videos",
    StaticFiles(directory="/home/vboxuser/Downloads/
glav/videos"),
    name="videos"
)

# ===== CORS =====
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# === ЗАГРУЗКА РЕКОМЕНДАЦИЙ ПРИ СТАРТЕ ===
@app.on_event("startup")
async def startup_event():
    try:
        recommender.load_data()
        print("✅ Рекомендательная система
загружена")
    except Exception as e:
        print(f" Ошибка загрузки рекомендаций: {e}")

# ===== PASSWORD HASHING =====

```

```

pwd_context = CryptContext(schemes=["bcrypt"],
deprecated="auto")

# ===== DATABASE CONFIG =====
DB_CONFIG = {
    "dbname": "school_db",
    "user": "admin",
    "password": "123a",
    "host": "127.0.0.1",
    "port": 5433
}

# ===== MODELS =====
class RegisterRequest(BaseModel):
    first_name: str = Field(..., min_length=2)
    last_name: str = Field(..., min_length=2)
    email: EmailStr
    phone: Optional[str] = None
    password: str = Field(..., min_length=6)
    role: str = Field(default="student")

class LoginRequest(BaseModel):
    login: str
    password: str

# МОДЕЛИ ДЛЯ ВИДЕО-УРОКОВ И ТЕСТОВ
class VideoLessonCreate(BaseModel):
    title: str
    description: Optional[str] = None
    video_url: Optional[str] = None
    teacher_id: int

class QuizQuestionCreate(BaseModel):
    lesson_id: int

```



```

    question_text: str
    option_a: str
    option_b: str
    option_c: str
    option_d: str
    correct_answer: str
    question_order: int

class QuizSubmit(BaseModel):
    lesson_id: int
    student_id: int
    teacher_id: int
    answers: dict

# ===== DATABASE CONNECTION =====
def get_db():
    conn = psycopg2.connect(**DB_CONFIG)
    try:
        yield conn
    finally:
        conn.close()

# ===== HELPER FUNCTIONS =====
def normalize_phone(phone: str) -> str:
    cleaned = re.sub(r'^\d+', '', phone)
    if cleaned.startswith('8'):
        cleaned = '+7' + cleaned[1:]
    elif not cleaned.startswith('+'):
        cleaned = '+' + cleaned
    return cleaned

# ===== REGISTRATION =====
@app.post("/api/register")

```

```

async def register(data: RegisterRequest,
db=Depends(get_db)):
    cur = db.cursor()
    try:
        cur.execute("SELECT id FROM users WHERE
email = %s", (data.email,))
        if cur.fetchone():
            raise HTTPException(status_code=400,
detail="Этот email уже зарегистрирован")

        if data.phone:
            clean_phone =
normalize_phone(data.phone)
            cur.execute("SELECT id FROM users WHERE
phone = %s", (clean_phone,))
            if cur.fetchone():
                raise HTTPException(status_code=400,
detail="Этот номер телефона уже зарегистрирован")

            hashed = pwd_context.hash(data.password)
            phone_to_save = normalize_phone(data.phone)
if data.phone else None
            is_verified = (data.role == "student")

            cur.execute("""
                INSERT INTO users (first_name,
last_name, email, phone, password_hash, role,
is_verified)
                VALUES (%s, %s, %s, %s, %s, %s, %s)
RETURNING id, email, first_name, last_name, role
            """, (data.first_name, data.last_name,
data.email, phone_to_save, hashed, data.role,
is_verified))

```

```

        db.commit()
        user = cur.fetchone()
        full_name = f"{user[2]} {user[3]}".strip()
if user[3] else user[2]

        return {
            "status": "success",
            "message": "Регистрация успешна",
            "user": {
                "id": user[0], "email": user[1],
"first_name": user[2],
                "last_name": user[3], "name":
full_name, "role": user[4]
            },
            "is_verified": is_verified
        }
    except HTTPException:
        raise
    except Exception as e:
        db.rollback()
        raise HTTPException(status_code=500,
detail=str(e))
    finally:
        cur.close()

# ===== LOGIN =====
@app.post("/api/login")
async def login(data: LoginRequest,
db=Depends(get_db)):
    cur = db.cursor()
    try:
        login_value = data.login.strip()
        if '@' in login_value:
            cur.execute("""

```

```

                SELECT id, password_hash,
first_name, last_name, role, is_verified
                FROM users WHERE email = %s
                """, (login_value,))
            else:
                clean_phone =
normalize_phone(login_value)
                cur.execute("""
                SELECT id, password_hash,
first_name, last_name, role, is_verified
                FROM users WHERE phone = %s
                """, (clean_phone,))

                user = cur.fetchone()
                if not user or not
pwd_context.verify(data.password, user[1]):
                    raise HTTPException(status_code=401,
detail="Неверный логин или пароль")

                if not user[5]:
                    raise HTTPException(status_code=403,
detail="Ваш аккаунт на проверке. Ожидайте
подтверждения от администрации.")

                full_name = f"{user[2]} {user[3]}".strip()
if user[3] else user[2]
                return {
                    "status": "success", "message": "Вход
выполнен",
                    "user": {
                        "id": user[0], "first_name":
user[2], "last_name": user[3],
                        "name": full_name, "role": user[4]
                    }
                }

```

```

    }
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(status_code=500,
detail=str(e))
    finally:
        cur.close()

# ===== ADMIN: PENDING TEACHERS =====
@app.get("/api/admin/pending-teachers")
async def get_pending_teachers(db=Depends(get_db)):
    cur = db.cursor()
    try:
        cur.execute("""
            SELECT id, email, first_name, last_name,
phone
            FROM users WHERE role = 'teacher' AND
is_verified = false
            """)
        teachers = cur.fetchall()
        return {
            "status": "success",
            "teachers": [
                {"id": t[0], "email": t[1],
"first_name": t[2], "last_name": t[3], "phone":
t[4]}
                for t in teachers
            ]
        }
    except Exception as e:
        raise HTTPException(status_code=500,
detail=str(e))
    finally:

```

```

        cur.close()

# ===== ADMIN: PENDING ADMINS =====
@app.get("/api/admin/pending-admins")
async def get_pending_admins(db=Depends(get_db)):
    cur = db.cursor()
    try:
        cur.execute("""
            SELECT id, email, first_name, last_name,
phone
            FROM users WHERE role = 'admin' AND
is_verified = false
            """)
        admins = cur.fetchall()
        return {
            "status": "success",
            "admins": [
                {"id": a[0], "email": a[1],
"first_name": a[2], "last_name": a[3], "phone":
a[4]}
                for a in admins
            ]
        }
    except Exception as e:
        raise HTTPException(status_code=500,
detail=str(e))
    finally:
        cur.close()

# ===== ADMIN: VERIFY TEACHER =====
@app.patch("/api/admin/verify-teacher/{user_id}")
async def verify_teacher(user_id: int,
db=Depends(get_db)):
    cur = db.cursor()

```

```

try:
    cur.execute("SELECT id, email, first_name,
last_name, role FROM users WHERE id = %s",
(user_id,))
    user = cur.fetchone()
    if not user or user[4] != "teacher":
        raise HTTPException(status_code=400,
detail="Пользователь не найден или не является
учителем")
    cur.execute("UPDATE users SET is_verified =
true WHERE id = %s", (user_id,))
    db.commit()
    return {"status": "success", "message":
f"Учитель {user[2]} {user[3]} одобрен"}
except HTTPException:
    raise
except Exception as e:
    db.rollback()
    raise HTTPException(status_code=500,
detail=str(e))
finally:
    cur.close()

# ===== ADMIN: VERIFY ADMIN =====
@app.patch("/api/admin/verify-admin/{user_id}")
async def verify_admin(user_id: int,
db=Depends(get_db)):
    cur = db.cursor()
    try:
        cur.execute("SELECT id, email, first_name,
last_name, role FROM users WHERE id = %s",
(user_id,))
        user = cur.fetchone()
        if not user or user[4] != "admin":

```

```

        raise HTTPException(status_code=400,
detail="Пользователь не найден или не является
администратором")
        cur.execute("UPDATE users SET is_verified =
true WHERE id = %s", (user_id,))
        db.commit()
        return {"status": "success", "message":
f"Администратор {user[2]} {user[3]} одобрен"}
    except HTTPException:
        raise
    except Exception as e:
        db.rollback()
        raise HTTPException(status_code=500,
detail=str(e))
    finally:
        cur.close()

# ===== ADMIN: REJECT USER =====
@app.patch("/api/admin/reject-user/{user_id}")
async def reject_user(user_id: int,
db=Depends(get_db)):
    cur = db.cursor()
    try:
        cur.execute("SELECT id, email, first_name,
last_name, role FROM users WHERE id = %s",
(user_id,))
        user = cur.fetchone()
        if not user or user[4] not in ["teacher",
"admin"]):
            raise HTTPException(status_code=400,
detail="Можно отклонить только учителя или админа")
        cur.execute("DELETE FROM users WHERE id =
%s", (user_id,))
        db.commit()

```

```

        return {"status": "success", "message":
f"Заявка {user[2]} {user[3]} отклонена"}
    except HTTPException:
        raise
    except Exception as e:
        db.rollback()
        raise HTTPException(status_code=500,
detail=str(e))
    finally:
        cur.close()

# ===== VIDEO LESSONS & QUIZ ENDPOINTS =====

# ADMIN: Create video lesson
@app.post("/api/admin/create-video-lesson")
async def create_video_lesson(data:
VideoLessonCreate, db=Depends(get_db)):
    cur = db.cursor()
    try:
        cur.execute("""
            INSERT INTO video_lessons (title,
description, video_url, teacher_id)
            VALUES (%s, %s, %s, %s) RETURNING id
            """, (data.title, data.description,
data.video_url, data.teacher_id))
        lesson_id = cur.fetchone()[0]
        db.commit()
        return {"status": "success", "lesson_id":
lesson_id}
    except Exception as e:
        db.rollback()
        raise HTTPException(status_code=500,
detail=str(e))
    finally:

```

```

        cur.close()

# ADMIN: Create quiz question
@app.post("/api/admin/create-quiz-question")
async def create_quiz_question(data:
QuizQuestionCreate, db=Depends(get_db)):
    cur = db.cursor()
    try:
        cur.execute("""
            INSERT INTO quiz_questions
            (lesson_id, question_text, option_a,
option_b, option_c, option_d, correct_answer,
question_order)
            VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
            """, (data.lesson_id, data.question_text,
data.option_a, data.option_b,
            data.option_c, data.option_d,
data.correct_answer, data.question_order))
        db.commit()
        return {"status": "success", "message":
"Вопрос добавлен"}
    except Exception as e:
        db.rollback()
        raise HTTPException(status_code=500,
detail=str(e))
    finally:
        cur.close()

# TEACHER: Get quiz results
@app.get("/api/teacher/quiz-results/{teacher_id}")
async def get_teacher_quiz_results(teacher_id: int,
db=Depends(get_db)):
    cur = db.cursor()
    try:

```

```

        cur.execute("""
            SELECT COUNT(DISTINCT student_id),
COUNT(*), AVG(percentage), MIN(completed_at),
MAX(completed_at)
            FROM student_quiz_results WHERE
teacher_id = %s
            """, (teacher_id,))
        stats = cur.fetchone()

        cur.execute("""
            SELECT vl.id, vl.title, COUNT(r.id),
AVG(r.percentage), MIN(r.percentage),
MAX(r.percentage)
            FROM video_lessons vl
            LEFT JOIN student_quiz_results r ON
vl.id = r.lesson_id
            WHERE vl.teacher_id = %s GROUP BY vl.id,
vl.title ORDER BY vl.created_at DESC
            """, (teacher_id,))
        lessons = cur.fetchall()

        cur.execute("""
            SELECT u.first_name, u.last_name,
vl.title, r.percentage, r.score, r.total_questions,
r.completed_at
            FROM student_quiz_results r
            JOIN users u ON r.student_id = u.id
            JOIN video_lessons vl ON r.lesson_id =
vl.id
            WHERE r.teacher_id = %s ORDER BY
r.completed_at DESC LIMIT 20
            """, (teacher_id,))
        recent = cur.fetchall()

```

```

        return {
            "status": "success",
            "stats": {
                "total_students": stats[0] or 0,
                "total_attempts": stats[1] or 0,
                "avg_percentage": float(stats[2]) if
stats[2] else 0,
                "first_attempt":
stats[3].isoformat() if stats[3] else None,
                "last_attempt": stats[4].isoformat()
if stats[4] else None
            },
            "lessons": [
                {"id": l[0], "title": l[1],
                "attempts": l[2], "avg_score": float(l[3]) if l[3]
else 0,
                "min_score": float(l[4]) if l[4]
else 0, "max_score": float(l[5]) if l[5] else 0}
                for l in lessons
            ],
            "recent_results": [
                {"student_name": f"{r[1]} {r[0]}",
                "lesson_title": r[2], "percentage": float(r[3]),
                "score": r[4], "total": r[5],
                "completed_at": r[6].isoformat() if r[6] else None}
                for r in recent
            ]
        }
    except Exception as e:
        raise HTTPException(status_code=500,
detail=str(e))
    finally:
        cur.close()

```

```

# TEACHER: Get quiz questions for lesson
@app.get("/api/teacher/quiz-questions/{lesson_id}")
async def get_quiz_questions(lesson_id: int,
db=Depends(get_db)):
    cur = db.cursor()
    try:
        cur.execute("""
            SELECT id, question_text, option_a,
option_b, option_c, option_d, question_order
            FROM quiz_questions WHERE lesson_id = %s
ORDER BY question_order
            """, (lesson_id,))
        questions = cur.fetchall()
        return {
            "status": "success",
            "questions": [
                {
                    "id": q[0],
                    "question_text":
clean_latex_symbols(q[1]),
                    "option_a":
clean_latex_symbols(q[2]),
                    "option_b":
clean_latex_symbols(q[3]),
                    "option_c":
clean_latex_symbols(q[4]),
                    "option_d":
clean_latex_symbols(q[5]),
                    "question_order": q[6]
                }
                for q in questions
            ]
        }
    except Exception as e:

```

```

        raise HTTPException(status_code=500,
detail=str(e))
    finally:
        cur.close()

# STUDENT: Submit quiz answers
@app.post("/api/student/submit-quiz")
async def submit_quiz(data: QuizSubmit,
db=Depends(get_db)):
    cur = db.cursor()

    try:
        cur.execute(
            """
            SELECT id, correct_answer
            FROM quiz_questions
            WHERE lesson_id = %s
            """,
            (data.lesson_id,)
        )

        questions = cur.fetchall()

        total_questions = len(questions)
        correct_count = 0

        for q_id, correct_answer in questions:

            user_answer = str(
                data.answers.get(str(q_id), "")
            ).strip().upper()

            correct_answer = str(
                correct_answer

```

```

        ).strip().upper()

        if user_answer == correct_answer:
            correct_count += 1

    percentage = (
        correct_count / total_questions * 100
        if total_questions > 0
        else 0
    )

    passed = percentage >= 70

    cur.execute(
        """
        INSERT INTO student_quiz_results
        (
            student_id,
            lesson_id,
            teacher_id,
            score,
            total_questions,
            percentage,
            answers
        )
        VALUES (%s,%s,%s,%s,%s,%s,%s)
        """,
        (
            data.student_id,
            data.lesson_id,
            data.teacher_id,
            correct_count,
            total_questions,
            percentage,
            answers
        )
    )

```

```

        json.dumps(data.answers)
    )

    db.commit()

    return {
        "success": True,
        "score": correct_count,
        "total_questions": total_questions,
        "percentage": round(percentage, 2),
        "passed": passed
    }

except Exception as e:
    db.rollback()
    raise HTTPException(
        status_code=500,
        detail=str(e)
    )

finally:
    cur.close()

# STUDENT: Get available video lessons
@app.get("/api/student/video-lessons")
async def get_student_videos(student_id: int,
db=Depends(get_db)):
    cur = db.cursor()
    try:
        cur.execute("""
            SELECT
                vl.id,
                vl.title,

```



```

        vl.description,
        vl.video_url,

    CASE
        WHEN MAX(r.percentage) >= 70
        THEN true
        ELSE false
    END AS completed

FROM video_lessons vl

LEFT JOIN student_quiz_results r
    ON vl.id = r.lesson_id
    AND r.student_id = %s

GROUP BY
    vl.id,
    vl.title,
    vl.description,
    vl.video_url

ORDER BY vl.id
""", (student_id,))
videos = cur.fetchall()
return {
    "videos": [
        {
            "id": v[0],
            "title":
clean_latex_symbols(v[1]),
            "description":
clean_latex_symbols(v[2]),
            "video_url": v[3],
            "completed": v[4]

```

```

        }
        for v in videos
    ]
    }
except Exception as e:
    raise HTTPException(status_code=500,
detail=str(e))
finally:
    cur.close()

# === РЕКОМЕНДАТЕЛЬНАЯ СИСТЕМА ===
@app.get("/api/student/recommendations/{student_id}")
)
async def get_student_recommendations(student_id:
int, lesson_id: int = None, top_n: int = 5):
    """Получить рекомендации для ученика"""
    try:
        recommendations =
recommender.get_recommendations(
            user_id=student_id,
            lesson_id=lesson_id,
            top_n=top_n
        )
        return recommendations
    except Exception as e:
        raise HTTPException(status_code=500,
detail=str(e))

@app.post("/api/admin/retrain-recommender")
async def retrain_recommender():
    """Переобучить систему на новых данных"""
    try:
        recommender.load_data()

```

```

        return {"status": "success", "message":
"Система переобучена"}
    except Exception as e:
        raise HTTPException(status_code=500,
detail=str(e))
# ===== AI HELPER (YandexGPT) =====
YANDEX_FOLDER_ID = "b1gst0fn81htpsdlgt6u"
YANDEX_API_KEY = "AQVN1216AIG-
xflQRTV2mmBkZVtfktR4bRHSwXAP"

def get_iam_token_from_api_key(api_key: str) -> str:
    """Получает временный IAM-токен из постоянного
API-ключа"""
    resp = requests.post(
        "https://iam.api.cloud.yandex.net/iam/v1/tok
ens",
        headers={"Content-Type":
"application/json"},
        json={"api_key": api_key}
    )
    resp.raise_for_status()
    return resp.json()["iamToken"]

YANDEX_WORKER_URL = "https://project-
fcmcu.vercel.app/api/yandex-ai"

@app.get("/api/ask-ai")
async def ask_ai(question: str):
    if not question.strip():
        return {"answer": "⚠️ Введите вопрос."}

    def call_yandex_gpt():
        try:
            import requests
            resp = requests.post(

```

```

        YANDEX_WORKER_URL,
        headers={"Content-Type":
"application/json"},
        json={"question": question}, # ←
Простой формат!
        timeout=30
    )
    resp.raise_for_status()
    data = resp.json()
    return
data["result"]["alternatives"][0]["message"]["text"]
except Exception as e:
    return f"⚠️ Ошибка: {str(e)[:100]}"

    try:
        import asyncio
        answer = await
        asyncio.to_thread(call_yandex_gpt)
        return {"answer": answer}
    except Exception as e:
        return {"answer": f"⚠️ Ошибка:
{str(e)[:100]}"}

#
=====
=====
# ЭНДПОИНТЫ ДЛЯ ТЕСТОВ
#
=====
=====
from pydantic import BaseModel
from typing import Dict

```

```

class QuizSubmission(BaseModel):
    student_id: int
    lesson_id: int
    answers: Dict[str, str]

@app.get("/api/quiz/questions")
async def get_quiz_questions(lesson_id: int):
    import asyncpg
    conn = await asyncpg.connect(
        user='admin',
        password='123a',
        database='school_db',
        host='127.0.0.1',
        port=5433
    )
    try:
        questions = await conn.fetch(
            """
            SELECT
                id,
                question_text,
                option_a,
                option_b,
                option_c,
                option_d,
                correct_answer,
                question_order
            FROM quiz_questions
            WHERE lesson_id = $1
            ORDER BY question_order
            """,
            lesson_id
        )
        result = []

```

```

        for q in questions:
            q_dict = dict(q)
            q_dict['question_text'] =
                clean_latex_symbols(q_dict.get('question_text', ''))
            q_dict['option_a'] =
                clean_latex_symbols(q_dict.get('option_a', ''))
            q_dict['option_b'] =
                clean_latex_symbols(q_dict.get('option_b', ''))
            q_dict['option_c'] =
                clean_latex_symbols(q_dict.get('option_c', ''))
            q_dict['option_d'] =
                clean_latex_symbols(q_dict.get('option_d', ''))
            result.append(q_dict)
        return result
    finally:
        await conn.close()

"/api/quiz/submit"
async def submit_quiz(submission: QuizSubmission):
    import asyncpg
    conn = await asyncpg.connect(
        user='admin',
        password='123a',
        database='school_db',
        host='127.0.0.1',
        port=5433
    )
    try:
        questions = await conn.fetch(
            "SELECT id, correct_answer FROM
            quiz_questions WHERE lesson_id = $1",
            submission.lesson_id
        )

```

```

        total_questions = len(questions)
        correct_answers = 0

        for q in questions:
            user_answer =
            submission.answers.get(str(q["id"]),
            "").strip().upper()
            correct_answer =
            q["correct_answer"].strip().upper()
            if user_answer == correct_answer:
                correct_answers += 1

        percentage = (correct_answers /
        total_questions * 100) if total_questions > 0 else 0

        await conn.execute(
            """INSERT INTO student_quiz_results
            (student_id, lesson_id, score,
            total_questions, percentage, answers)
            VALUES ($1, $2, $3, $4, $5, $6)""",
            submission.student_id,
            submission.lesson_id,
            correct_answers,
            total_questions,
            percentage,
            str(submission.answers)
        )

        return {
            "score": correct_answers,
            "total_questions": total_questions,
            "percentage": round(percentage, 2),
            "passed": percentage >= 70
        }

```

```

        finally:
            await conn.close()

@app.get("/api/student/quiz-results")
async def get_student_quiz_results(student_id: int):
    import asyncpg
    conn = await asyncpg.connect(
        user='admin',
        password='123a',
        database='school_db',
        host='127.0.0.1',
        port=5433
    )
    try:
        results = await conn.fetch(
            """SELECT lesson_id, percentage,
            completed_at
            FROM student_quiz_results
            WHERE student_id = $1
            ORDER BY completed_at DESC""",
            student_id
        )
        return [dict(r) for r in results]
    finally:
        await conn.close()

# ===== СООБЩЕНИЯ МЕЖДУ УЧИТЕЛЕМ И УЧЕНИКОМ =====
class MessageSend(BaseModel):
    sender_id: int
    receiver_id: int
    message_text: str

@app.post("/api/messages/send")

```

```

async def send_message(data: MessageSend,
db=Depends(get_db)):
    cur = db.cursor()
    try:
        cur.execute("""
            INSERT INTO messages (sender_id,
receiver_id, message_text)
            VALUES (%s, %s, %s) RETURNING id,
created_at
            """, (data.sender_id, data.receiver_id,
data.message_text))
        db.commit()
        msg_id, created_at = cur.fetchone()
        return {
            "status": "success",
            "message": {
                "id": msg_id,
                "sender_id": data.sender_id,
                "receiver_id": data.receiver_id,
                "message_text": data.message_text,
                "created_at": created_at.isoformat()
            }
        }
    except Exception as e:
        db.rollback()
        raise HTTPException(status_code=500,
detail=str(e))
    finally:
        cur.close()

@app.get("/api/messages/conversation/{user1_id}/{use
r2_id}")
async def get_conversation(user1_id: int, user2_id:
int, db=Depends(get_db)):

```

```

cur = db.cursor()
try:
    cur.execute("""
        SELECT id, sender_id, receiver_id,
message_text, created_at
        FROM messages
        WHERE (sender_id = %s AND receiver_id =
%s)
            OR (sender_id = %s AND receiver_id =
%s)
            ORDER BY created_at ASC
        """, (user1_id, user2_id, user2_id,
user1_id))
    messages = cur.fetchall()

    return {
        "status": "success",
        "messages": [
            {
                "id": m[0],
                "sender_id": m[1],
                "receiver_id": m[2],
                "message_text": m[3],
                "created_at": m[4].isoformat()
            }
            for m in messages
        ]
    }
except Exception as e:
    raise HTTPException(status_code=500,
detail=str(e))
finally:
    cur.close()

```

```

@app.get("/api/messages/conversations/{user_id}")
async def get_conversations(user_id: int,
db=Depends(get_db)):
    cur = db.cursor()
    try:
        cur.execute("""
            SELECT DISTINCT
                CASE
                    WHEN sender_id = %s THEN
receiver_id
                        ELSE sender_id
                    END as other_user_id
            FROM messages
            WHERE sender_id = %s OR receiver_id = %s
            """, (user_id, user_id, user_id))
        user_ids = [row[0] for row in
cur.fetchall()]

        conversations = []
        for other_id in user_ids:
            cur.execute("""
                SELECT id, first_name, last_name,
role
                    FROM users WHERE id = %s
            """, (other_id,))
            user = cur.fetchone()
            if user:
                conversations.append({
                    "user_id": user[0],
                    "name": f"{user[1]} {user[2]}",
                    "role": user[3]
                })

        return {

```

```

                "status": "success",
                "conversations": conversations
            }
        except Exception as e:
            raise HTTPException(status_code=500,
detail=str(e))
        finally:
            cur.close()

# ===== AI TEST GENERATOR (Динамическая генерация
через YandexGPT) =====
import random
import json
import re

@app.get("/api/ai-test/generate")
async def generate_ai_test():
    """Генерирует случайное задание через
YandexGPT"""
    # Улучшенные шаблоны заданий
    task_templates = [
        {
            "type": "condition_analysis",
            "topic": "Условные операторы",
            "difficulty": "easy",
            "prompt": """Сгенерируй ЗАДАНИЕ 12 из
ОГЭ по информатике на анализ условного оператора.

ТРЕБОВАНИЯ:
1. Дай код на Python с условием if/else (можно с
несколькими условиями AND/OR)
2. Перечисли КОНКРЕТНЫЕ 9-12 пар входных данных
(числа)
3. Спроси СКОЛЬКО РАЗ программа напечатает YES/NO

```

4. Ответ должен быть числом от 0 до 12
5. Сделай задание ПОДРОБНЫМ и ПОНЯТНЫМ

Формат ответа ТОЛЬКО JSON:

```
{
  "title": "Анализ условного оператора",
  "description": "Было проведено N запусков программы. При каких значения программа напечатает «YES»?",
  "code": "код на Python с отступами",
  "question": "Сколько было запусков, при которых программа напечатала «YES»?\\n\\nПары чисел: (1, 2); (11, 2); ...",
  "correct_answer": "число",
  "explanation": "Подробное объяснение решения"
}"""
},
{
  "type": "loop_analysis",
  "topic": "Циклы",
  "difficulty": "easy",
  "prompt": ""Сгенерируй ЗАДАНИЕ из ОГЭ по информатике на анализ цикла for или while.
```

ТРЕБОВАНИЯ:

1. Дай код на Python с циклом (for или while)
2. В коде должны быть КОНКРЕТНЫЕ числа (диапазон, шаг)
3. Спроси ЧТО БУДЕТ НАПЕЧАТАНО (число)
4. Сделай задание с суммой, произведением или подсчётом
5. Сделай задание ПОДРОБНЫМ и ПОНЯТНЫМ

Формат ответа ТОЛЬКО JSON:

```
{
  "title": "Анализ цикла",
  "description": "Определите, что будет напечатано в результате выполнения программы:",
  "code": "код на Python с отступами",
  "question": "Какое число будет напечатано?",
  "correct_answer": "число",
  "explanation": "Подробное объяснение: цикл выполняется N раз, суммирует числа от X до Y..."
}"""
},
{
  "type": "array_processing",
  "topic": "Обработка массивов",
  "difficulty": "medium",
  "prompt": ""Сгенерируй ЗАДАНИЕ из ОГЭ по информатике на обработку массива.
```

ТРЕБОВАНИЯ:

1. Дай код на Python с массивом (списком) и циклом
2. В массиве должны быть КОНКРЕТНЫЕ числа [2, 4, 6, 8, 10]
3. Добавь условие (if) для фильтрации элементов
4. Спроси РЕЗУЛЬТАТ (сумма, количество, среднее)
5. Сделай задание ПОДРОБНЫМ и ПОНЯТНЫМ

Формат ответа ТОЛЬКО JSON:

```
{
  "title": "Обработка массива",
  "description": "В программе обрабатывается массив. Определите результат:",
  "code": "код на Python с отступами",
  "question": "Какое число будет напечатано?",
  "correct_answer": "число",
```

```

        "explanation": "Подробное объяснение: массив
содержит [...], условие выбирает элементы > 5, сумма
равна ..."
    }"""

```

```
    },
```

```
    {
```

```
        "type": "string_processing",
```

```
        "topic": "Обработка строк",
```

```
        "difficulty": "medium",
```

```
        "prompt": """Сгенерируй ЗАДАНИЕ из ОГЭ
```

по информатике на обработку строк.

ТРЕБОВАНИЯ:

1. Дай код на Python со строковыми операциями
2. В коде должно быть КОНКРЕТНОЕ слово или фраза
3. Добавь подсчёт символов, букв, слов
4. Спроси РЕЗУЛЬТАТ (число или строка)
5. Сделай задание ПОДРОБНЫМ и ПОНЯТНЫМ

Формат ответа ТОЛЬКО JSON:

```

{
    "title": "Обработка строк",
    "description": "Определите количество
символов/букв в слове:",
    "code": "код на Python с отступами",
    "question": "Какое число будет напечатано?",
    "correct_answer": "число или строка",
    "explanation": "Подробное объяснение: слово
'информатика' содержит 11 букв, из них гласных 5..."
}"""

```

```
    },
```

```
    {
```

```
        "type": "algorithm_analysis",
```

```
        "topic": "Анализ алгоритмов",
```

```
        "difficulty": "medium",
```

```
        "prompt": """Сгенерируй ЗАДАНИЕ из ОГЭ
по информатике на анализ алгоритма.
```

ТРЕБОВАНИЯ:

1. Дай код на Python с вложенными циклами
2. В коде должны быть КОНКРЕТНЫЕ диапазоны (range)
3. Спроси СКОЛЬКО РАЗ выполнится операция или ЧТО БУДЕТ НАПЕЧАТАНО
4. Сделай задание ПОДРОБНЫМ и ПОНЯТНЫМ

Формат ответа ТОЛЬКО JSON:

```

{
    "title": "Вложенные циклы",
    "description": "Сколько раз будет выполнена
внутренняя операция?",
    "code": "код на Python с отступами",
    "question": "Какое число будет напечатано?",
    "correct_answer": "число",
    "explanation": "Подробное объяснение: внешний
цикл 3 раза, внутренний 2 раза, всего 3*2=6
выполнений"
}"""

```

```
    }
```

```
]
```

```
# Выбираем случайный тип задания
```

```
template = random.choice(task_templates)
```

```
# Генерируем задание через YandexGPT
```

```
try:
```

```
    import requests
```

```
    response = requests.post(
```



```

        "https://project-
fcmcu.vercel.app/api/yandex-ai",
        headers={"Content-Type":
"application/json"},
        json={"question": template["prompt"]},
        timeout=30
    )
    response.raise_for_status()
    data = response.json()
    ai_response =
data["result"]["alternatives"][0]["message"]["text"]

    # Парсим JSON из ответа ИИ
    import json as json_module
    import re

    # Ищем JSON в ответе
    json_match = re.search(r'\{[\s\S]*\}',
ai_response)
    if json_match:
        task_data =
json_module.loads(json_match.group())
    else:
        raise ValueError("Не удалось распарсить
JSON")

    # Добавляем метаданные
    task_data["id"] = random.randint(1000, 9999)
    task_data["type"] = template["type"]
    task_data["topic"] = template["topic"]
    task_data["difficulty"] =
template["difficulty"]
    task_data["language"] = "python"

```

```

        return task_data

    except Exception as e:
        print(f"Error generating task with AI: {e}")
        return get_fallback_task()
def get_fallback_task():
    """Возвращает статическое задание если ИИ не
работает"""
    fallback_tasks = [
        {
            "id": 1001,
            "type": "condition_check",
            "title": "Условный оператор",
            "topic": "Логические условия",
            "difficulty": "easy",
            "description": "Было проведено 9
запусков программы. При каких значения программа
напечатает «YES»?",
            "code": """s = int(input())
t = int(input())
if s > 10 or t > 10:
    print("YES")
else:
    print("NO")""",
            "language": "python",
            "question": "Сколько было запусков, при
которых программа напечатала «YES»? \n\nПары чисел:
(1, 2); (11, 2); (1, 12); (11, 12); (-11, -12); (-
11, 12); (-12, 11); (10, 10); (10, 5)",
            "correct_answer": "4",
            "explanation": "Программа напечатает
«YES», когда s > 10 ИЛИ t > 10. Подходящие пары:
(11, 2), (1, 12), (11, 12), (-11, 12) = 4 запуска"
        },
    ]

```

```

{
    "id": 1002,
    "type": "loop_sum",
    "title": "Цикл с параметром",
    "topic": "Циклы",
    "difficulty": "easy",
    "description": "Определите, что будет
напечатано в результате выполнения программы:",
    "code": """s = 0
for i in range(1, 6):
    s = s + i
print(s)""",
    "language": "python",
    "question": "Какое число будет
напечатано?",
    "correct_answer": "15",
    "explanation": "Цикл суммирует числа от
1 до 5: 1+2+3+4+5 = 15"
},
{
    "id": 1003,
    "type": "array_sum",
    "title": "Обработка массива",
    "topic": "Массивы",
    "difficulty": "medium",
    "description": "В программе
обрабатывается массив. Определите результат:",
    "code": """arr = [2, 4, 6, 8, 10]
s = 0
for i in range(len(arr)):
    if arr[i] > 5:
        s = s + arr[i]
print(s)""",
    "language": "python",

```

```

        "question": "Какое число будет
напечатано?",
        "correct_answer": "24",
        "explanation": "Суммируются элементы
больше 5: 6+8+10 = 24"
    }
]
return random.choice(fallback_tasks)

@app.post("/api/ai-test/check")
async def check_ai_test_answer(data: dict):
    """Проверяет ответ через YandexGPT и даёт
развёрнутую обратную связь"""

    task_id = data.get("task_id")
    task_type = data.get("task_type")
    student_answer = data.get("student_answer")
    student_explanation =
data.get("student_explanation", "")
    correct_answer = data.get("correct_answer")
    task_code = data.get("task_code")
    task_question = data.get("task_question")

    # Проверяем правильность
    is_correct =
check_answer_flexible(str(student_answer).strip(),
str(correct_answer).strip())

    # Генерируем обратную связь через YandexGPT
    try:
        ai_feedback = await
generate_ai_feedback_for_test(
            task_type=task_type,
            is_correct=is_correct,

```

```

        student_answer=student_answer,
        correct_answer=correct_answer,
        student_explanation=student_explanation,
        task_code=task_code,
        task_question=task_question
    )
except Exception as e:
    print(f"Error generating AI feedback: {e}")
    ai_feedback = f"Ответ: {'верный' if
is_correct else 'неверный'}. Правильный ответ:
{correct_answer}."

    recommendations =
generate_test_recommendations(task_type, is_correct)

    return {
        "is_correct": is_correct,
        "correct_answer": correct_answer,
        "student_answer": student_answer,
        "ai_feedback": ai_feedback,
        "recommendations": recommendations,
        "task_type": task_type
    }

def check_answer_flexible(student_ans: str,
correct_ans: str) -> bool:
    """Гибкая проверка ответа"""
    student_clean = student_ans.strip().lower()
    correct_clean = correct_ans.strip().lower()

    if student_clean == correct_clean:
        return True

    try:

```

```

        if float(student_clean) ==
float(correct_clean):
            return True
        except:
            pass

        if correct_clean in student_clean or
student_clean in correct_clean:
            return True

        return False

async def generate_ai_feedback_for_test(task_type:
str, is_correct: bool,
                                student_ans
                                student_exp
                                task_questi
on: str) -> str:
    """Генерирует персонализированную обратную связь
через YandexGPT"""

    if is_correct:
        prompt = f"""Ты преподаватель информатики
для подготовки к ОГЭ. Ученик правильно решил
задание!

Тип задания: {task_type}
Код программы:
{task_code}

Вопрос: {task_question}
Правильный ответ: {correct_answer}

```

```

Ответ ученика: {student_answer}
Объяснение ученика: {student_explanation if
student_explanation else 'не предоставлено'}

Похвали ученика и кратко (2-3 предложения) объясни,
почему ответ правильный. Дай совет для дальнейшего
улучшения. Будь дружелюбным и используй эмодзи. """
    else:
        prompt = f"""Ты преподаватель информатики
для подготовки к ОГЭ. Ученик ошибся в задании.

Тип задания: {task_type}
Код программы:
{task_code}

Вопрос: {task_question}
Правильный ответ: {correct_answer}
Ответ ученика: {student_answer}
Объяснение ученика: {student_explanation if
student_explanation else 'не предоставлено'}

Проанализируй ошибку ученика:
1. Объясни, в чём именно ошибка (2-3 предложения)
2. Объясни правильный подход к решению (2-3
предложения)
3. Укажи на конкретные пробелы в знаниях и что нужно
повторить

Будь доброжелательным, поддерживающим и используй
эмодзи. """

    try:
        import requests
        response = requests.post(

```

```

        "https://project-
fcmcu.vercel.app/api/yandex-ai",
        headers={"Content-Type":
"application/json"},
        json={"question": prompt},
        timeout=20
    )
    response.raise_for_status()
    data = response.json()
    return
data["result"]["alternatives"][0]["message"]["text"]
except Exception as e:
    print(f"YandexGPT API error: {e}")
    if is_correct:
        return "✅ Отлично! Вы правильно решили
задание. Так держать! 🙌 "
    else:
        return f"❌ Неправильно. Правильный
ответ: {correct_answer}. Повторите тему
'{task_type}'."

def generate_test_recommendations(task_type: str,
is_correct: bool) -> list:
    """Генерирует рекомендации для улучшения"""

    recommendations_db = {
        "condition_analysis": [
            "Повтори таблицу истинности для
логических операций OR и AND",
            "Потренируйся определять порядок
вычисления сложных условий",
            "Разбери задачи на проверку нескольких
условий одновременно"

```

```

],
"loop_analysis": [
    "Отработай понимание работы range() в Python",
    "Потренируйся считать количество итераций цикла",
    "Разбери разницу между range(1, 6) и range(6)"
],
"array_processing": [
    "Повтори индексацию массивов (с 0 в Python)",
    "Потренируйся проходить по массиву с условием",
    "Разбери задачи на поиск мин/макс элемента в массиве"
],
"string_processing": [
    "Повтори функции работы со строками (len, in, pos)",
    "Отработай перебор символов строки в цикле",
    "Разбери задачи на подсчёт определённых символов"
],
"algorithm_analysis": [
    "Потренируйся считать общее количество итераций вложенных циклов",
    "Разбери принцип работы вложенных циклов (внешний × внутренний)",
    "Порешай задачи на таблицу умножения как пример вложенных циклов"
]
}

```

```

if is_correct:
    recs = recommendations_db.get(task_type, ["Продолжай решать задачи!"])
    return [f"✅ Отлично! Для закрепления: {recs[0]}"]
else:
    return recommendations_db.get(task_type, [{"📚 Повтори теорию и порешай больше задач по этой теме"}])

# ===== МАТЕРИАЛЫ =====
@app.get("/api/materials")
async def get_materials(db=Depends(get_db)):
    """Получить список учебных материалов"""
    cur = db.cursor()
    try:
        cur.execute("""
            SELECT id, title, description,
            file_path, file_type, category, created_at
            FROM materials
            ORDER BY created_at DESC
        """)
        materials = cur.fetchall()

    return {
        "status": "success",
        "materials": [
            {
                "id": m[0],
                "title": m[1],
                "description": m[2],
                "file_path": m[3],

```

```

        "file_type": m[4],
        "category": m[5],
        "created_at": m[6].isoformat()
    }
    for m in materials
]
}
except Exception as e:
    raise HTTPException(status_code=500,
detail=str(e))
finally:
    cur.close()

@app.get("/api/materials/{material_id}/download")
async def download_material(material_id: int,
db=Depends(get_db)):
    """Скачать/открыть файл материала"""
    cur = db.cursor()
    try:
        cur.execute("""
            SELECT title, file_path, file_type
            FROM materials WHERE id = %s
            """, (material_id,))
        material = cur.fetchone()

        if not material:
            raise HTTPException(status_code=404,
detail="Материал не найден")

        file_path = material[1]
        file_type = material[2]

```

```

        if not os.path.exists(file_path):
            raise HTTPException(status_code=404,
detail="Файл не найден на сервере")

        return FileResponse(
            file_path,
            media_type='application/pdf' if
file_type == 'pdf' else 'application/octet-stream',
            filename=f"{material[0]}.{file_type}"
        )
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(status_code=500,
detail=str(e))
    finally:
        cur.close()

# =====
# 🔥 ЗАПУСК СЕРВЕРА (ОБЯЗАТЕЛЬНО!)
# =====
if __name__ == "__main__":
    import uvicorn
    uvicorn.run(
        app,
        host="0.0.0.0",
        port=8000,
        log_level="info"
    )

```

Листинг А.2 - admin-verify.html

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-
width, initial-scale=1.0">
  <title>Мост Знаний | Панель
администратора</title>
  <link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.4.0/css/all.min.css">
  <link
href="https://fonts.googleapis.com/css2?family=Inter
:wght@300;400;500;600;700&display=swap"
rel="stylesheet">
  <style>
    * { margin: 0; padding: 0; box-sizing:
border-box; }
    body {
      font-family: 'Inter', sans-serif;
      background: #f8fafc;
      color: #1e293b;
      padding: 40px 20px;
```

```
    }
    .container { max-width: 1200px; margin: 0
auto; }

    .header { margin-bottom: 32px; }
    .header h1 {
      font-size: 1.8rem;
      font-weight: 700;
      display: flex;
      align-items: center;
      gap: 12px;
      margin-bottom: 8px;
    }
    .header p { color: #64748b; }

    .card {
      background: #fff;
      border-radius: 16px;
      padding: 24px;
      margin-bottom: 24px;
```

```

        border: 1px solid #e2e8f0;
        box-shadow: 0 2px 8px rgba(0,0,0,0.04);
    }

    .card-header {
        display: flex;
        justify-content: space-between;
        align-items: center;
        margin-bottom: 24px;
    }

    .card-title {
        display: flex;
        align-items: center;
        gap: 8px;
        font-weight: 600;
        font-size: 1.1rem;
    }

    .badge {
        background: #e0f2fe;
        color: #0369a1;
        padding: 2px 10px;

```

```

        border-radius: 12px;
        font-size: 0.8rem;
        font-weight: 600;
    }

    .badge-warning {
        background: #fef3c7;
        color: #d97706;
    }

    .btn {
        padding: 8px 16px;
        border-radius: 8px;
        border: none;
        font-weight: 500;
        cursor: pointer;
        display: inline-flex;
        align-items: center;
        gap: 6px;
        transition: all 0.2s;
    }

    .btn-ghost {

```



```

        background: #f1f5f9;
        color: #1e293b;
    }
    .btn-ghost:hover { background: #e2e8f0; }
    .btn-success {
        background: #16a34a;
        color: white;
    }
    .btn-success:hover { background: #15803d; }
    .btn-danger {
        background: #dc2626;
        color: white;
    }
    .btn-danger:hover { background: #b91c1c; }

    .empty-state {
        text-align: center;
        padding: 40px 20px;
        color: #64748b;
    }
    .empty-state i {

```

```

        font-size: 3rem;
        color: #16a34a;
        margin-bottom: 12px;
        background: #dcfce7;
        width: 80px;
        height: 80px;
        border-radius: 50%;
        display: inline-flex;
        align-items: center;
        justify-content: center;
    }

    table {
        width: 100%;
        border-collapse: collapse;
    }
    th, td {
        text-align: left;
        padding: 12px 16px;
        border-bottom: 1px solid #f1f5f9;
    }

```

```

th {
    font-weight: 600;
    color: #64748b;
    font-size: 0.85rem;
    text-transform: uppercase;
    letter-spacing: 0.5px;
}
td { font-size: 0.95rem; }
tr:last-child td { border-bottom: none; }

.role-badge {
    padding: 4px 12px;
    border-radius: 20px;
    font-size: 0.75rem;
    font-weight: 600;
    background: #fef3c7;
    color: #d97706;
    display: inline-block;
}

.actions { display: flex; gap: 8px; }

```

```

/* GRAFANA CONTAINER */
.grafana-panel {
    width: 100%;
    height: 650px;
    border-radius: 12px;
    overflow: hidden;
    border: 1px solid #e2e8f0;
    background: #fff;
}
.grafana-panel iframe {
    width: 100%;
    height: 100%;
    border: none;
}
</style>
</head>
<body>
    <div class="container">
        <div class="header">

```

```

        <h1><i class="fas fa-briefcase"></i>
Панель администратора</h1>

        <p>Заявки на подтверждение</p>
    </div>

    <!-- Teachers Section -->
    <div class="card">
        <div class="card-header">
            <div class="card-title">
                <i class="fas fa-chalkboard-
teacher"></i>

                Преподаватели

                <span class="badge">0</span>

            </div>

            <button class="btn btn-ghost"
onclick="location.reload()">

                <i class="fas fa-sync-alt"></i>
Обновить

            </button>

        </div>

        <div class="empty-state">
            <i class="fas fa-check-circle"></i>

```

```

        <p>Нет заявок от преподавателей</p>
    </div>

    </div>

    <!-- Admins Section -->
    <div class="card">
        <div class="card-header">
            <div class="card-title">
                <i class="fas fa-user-
shield"></i>

                Администраторы

                <span class="badge badge-
warning">1</span>

            </div>

        </div>

        <table>

            <thead>

                <tr>

                    <th>ФИО</th>

                    <th>Email</th>

                    <th>Телефон</th>

                    <th>Роль</th>

```

```

                <th>Действие</th>
            </tr>
        </thead>
        <tbody>
            <tr>
                <td><strong>Олег
Админов</strong></td>
                <td>oleg@admin.ru</td>
                <td>+79991110022</td>
                <td><span class="role-
badge">Администратор</span></td>
                <td class="actions">
                    <button class="btn btn-
success">
                        <i class="fas fa-
check"></i> Одобрить
                    </button>
                    <button class="btn btn-
danger">
                        <i class="fas fa-
times"></i> Отказать
                    </button>
                </td>
            </tr>
        </tbody>
    </table>

```

```

        </tr>
    </tbody>
</table>
</div>

<!-- GRAFANA ANALYTICS -->
<div class="card">
    <div class="card-header">
        <div class="card-title">
            <i class="fas fa-chart-bar"></i>
            Аналитика платформы (Grafana)
        </div>
    </div>
    <div class="grafana-panel">
        <iframe
            src="https://genetics-upload-
hurled.ngrok-free.dev/grafana/d-
solo/bfntuoxtbnsowe/roli?orgId=1&from=1780645024315&
to=1780666624315&timezone=browser&panelId=1&__featur
e.dashboardSceneSolo">
        </iframe>
    </div>

```

```
</div>  
</div>
```

```
</body>  
</html>
```

Полным листонгом можно ознакомиться по ссылке <https://disk.yandex.ru/d/db95DkGyQXgXGg>

ПРИЛОЖЕНИЕ Б

(справочное)

Финансовая модель проекта

Таблица Б.1 –План реализации услуг на 2026 год

Месяц	Бесплатный пакет (чел)	Пакет –Про (чел)	Пакет-Про+(чел)	Платные пакеты всего(чел)	Всего участников(чел)	Сумма пакета-Про	Сумма Пакета -Про+	Итого
Июль	200	50	0	50	250	49500,00	0,00	49500,00
Август	200	55	0	55	255	54450,00	0,00	54450,00
Сентябрь	300	50	10	60	360	49500,00	11900,00	61400,00
Октябрь	350	100	30	130	480	99000,00	35700,00	134700,00
Ноябрь	400	200	45	245	645	198000,00	53550,00	251550,00
Декабрь	600	300	45	345	945	297000,00	53550,00	350550,00
Итого							0,00	902150,00

Таблица Б.2 –План реализации услуг на 2027 год

Месяц	Бесплатный пакет (чел)	Пакет –Про (чел)	Пакет-Про+(чел)	Платные пакеты всего(чел)	Всего участников(чел)	Сумма пакета-Про	Сумма Пакета -Про+	Итого
Январь	1000	350	75	425	1425	346500,00	89250,00	435750,00
Февраль	1200	400	75	475	1675	396000,00	89250,00	485250,00
Март	1400	450	90	540	1940	445500,00	107100,00	552600,00
Апрель	1500	500	90	590	2090	495000,00	107100,00	602100,00
Май	1500	500	90	590	2090	495000,00	107100,00	602100,00
Июнь	1600	500	75	575	2175	495000,00	89250,00	584250,00
Июль	1500	300	30	330	1830	297000,00	35700,00	332700,00
Август	900	100	10	110	1010	99000,00	11900,00	110900,00
Сентябрь	950	85	35	120	1070	84150,00	41650,00	125800,00
Октябрь	1000	250	40	290	1290	247500,00	47600,00	295100,00
Ноябрь	1200	300	40	340	1540	297000,00	47600,00	344600,00
Декабрь	1500	400	45	445	1945	396000,00	53550,00	449550,00
Итого								4920700,00

Таблица Б.3 – Финансовая модель за первый год со старта запуска сайта (2026 год)

Показатели	Июль	Август	Сентябрь	Октябрь	Ноябрь	Декабрь	Итог
Поступления от реализации	49500	54450	61400	134700	251550	350550	902150
Платежи	145 389,97	145 686,97	146 103,97	150 501,97	157 512,97	163 452,97	908 648,82
Постоянные	137 419,97	137 419,97	137 419,97	137 419,97	137 419,97	137 419,97	824 519,82
Реклама/продвижение	3000	3000	3000	3000	3000	3000	18000
Интернет и услуги связи	2000	2000	2000	2000	2000	2000	12000
ФЗП с СФР	132 419,97	132 419,97	132 419,97	132 419,97	132 419,97	132 419,97	794 519,82
Переменные	7970	8267	8684	13082	20093	26033	84129
Налоги (УСН)	2 970	3 267	3 684	8 082	15 093	21 033	54 129
Бухгалтерские услуги	5000	5000	5000	5000	5000	5000	30000
Поступления-платежи	-95 889,97	-91 236,97	-84 703,97	-15 801,97	94 037,03	187 097,03	
Нарастающий итог	-95 889,97	-187 126,94	-271 830,91	-287632,88	-193 595,85	-6 498,82	

Таблица Б.4 – Финансовая модель за первый год со старта запуска сайта (2027 год)

Показатели	Янв.	Февр.	Март	Апр.	Май	Июнь	Июль	Авг.	Сент.	Окт.	Нояб.	Дек.	Итого
Поступления от реализации	435750	485250	552600	602100	602100	584250	332700	110900	125800	295100	344600	449550	4920700
Платежи	168564,97	171534,97	175575,97	173545,97	173545,97	177474,97	162381,97	149073,97	149967,97	160125,97	163095,97	164392,97	2004281,64
Постоянные	137419,97	137419,97	137419,97	137419,97	137419,97	137419,97	137419,97	137419,97	137419,97	137419,97	137419,97	137419,97	1649039,64
Реклама/продвижение	3000	3000	3000	3000	3000	3000	3000	3000	3000	3000	3000	3000	36000
Интернет и услуги связи	2000	2000	2000	2000	2000	2000	2000	2000	2000	2000	2000	2000	24000
ФЗП с СФР	132419,97	132419,97	132419,97	132419,97	132419,97	132419,97	132419,97	132419,97	132419,97	132419,97	132419,97	132419,97	1589039,64
Переменные	31145	34115	38156	41126	41126	40055	24962	11654	12548	22706	25676	31973	355242
Налоги (УСН)	26145	29115	33156	36126	36126	35055	19962	6654	7548	17706	20676	26973	295242
Бухгалтерские услуги	5000	5000	5000	5000	5000	5000	5000	5000	5000	5000	5000	5000	60000
Поступления-платежи	267185,03	313715,03	377024,03	428554,03	428554,03	406775,03	170318,03	-38173,97	-24167,97	134974,03	181504,03	285157,03	2931418,36
Нарастающий итог	267185,03	298500,06	675524,09	1104078,12	1532632,15	1939407,18	2109725,21	2071551,24	2047383,27	2182357,3	2363861,33	2649018,36	

ПРИЛОЖЕНИЕ В

Паспорт проекта

	КРАТКАЯ ИНФОРМАЦИЯ О СТАРТАП-ПРОЕКТЕ		
1.	Название стартап-проекта*	Мост Знаний – онлайн-платформа с ИИ для персонализации обучения	
2.	Технологическое направление в соответствии с перечнем критических технологий РФ	Информационные технологии, искусственный интеллект, технологии обучения	
3.	Рынок НТИ	EduNet (образование)	
4.	Сквозные технологии	Искусственный интеллект, большие данные, распределённые реестры (для сертификации)	
	ИНФОРМАЦИЯ О КОМАНДЕ СТАРТАП-ПРОЕКТА		
5.	Лидер стартап-проекта и его опыт	Доронин Александр Дмитриевич. Опыт в создании веб-проектов (кулинарный сайт в курсовом проекте), знание основ управления проектами и разработки ПО	
6.	Команда стартап-проекта: Доронин Александр Дмитриевич, Груздев Николай Александрович, Локкин Иван Михайлович, Кустышев Сергей Сергеевич		
	ФИО	Роль в проекте	Опыт
	Доронин Александр Дмитриевич	Лидер проекта, full-stack разработчик, DevOps-инженер	Веб-разработка, управление проектами
	Груздев Николай Александрович	Backend-разработчик, Заместитель лидера проекта, DevOps-инженер	Python, машинное обучение, настройка серверов

	Локкин Иван Михайлович	Frontend-разработчик, UX/UI дизайнер, DevOps-инженер	React, Figma, пользовательские интерфейсы
	Кустышев Сергей Сергеевич	DevOps-инженер, контент-менеджер, методист, маркетолог	Создание образовательного контента, анализ рынка
	ПЛАН РЕАЛИЗАЦИИ СТАРТАП-ПРОЕКТА		
7.	Аннотация проекта <i>Указывается краткая информация (не более 1000 знаков, без пробелов) о стартап-проекте (краткий реферат проекта: цели, задачи проекта, ожидаемые результаты, области применения результатов, потенциальные потребительские сегменты)</i>	Онлайн-школа «Мост Знаний» – платформа с ИИ, которая анализирует знания ученика, определяет слабые места и строит индивидуальную траекторию обучения. Цель – повысить эффективность обучения на 30% по сравнению с традиционными методами. Целевая аудитория – школьники 12–18 лет, готовящиеся к ОГЭ/ЕГЭ, и студенты вузов. Ожидаемые результаты: MVP через 6 месяцев, 1000 активных пользователей через год.	
	БАЗОВАЯ БИЗНЕС-ИДЕЯ		
8.	Какой продукт (товар/ услуга/ устройство/ ПО/ технология/ процесс и т.д.) будет продаваться <i>Указывается максимально понятно и емко информация о продукте, лежащем в основе стартап-проекта, благодаря реализации которого планируется получать основной доход</i>	SaaS-платформа с подпиской: доступ к персонализированным курсам, аналитика прогресса, рекомендации ИИ. Дополнительно – платные консультации с преподавателями.	
9.	Какую и чью (какого типа потребителей) проблему решает (с доказательством проблематики)	Проблема: стандартные онлайн-курсы не учитывают индивидуальный уровень ученика, что снижает мотивацию и эффективность. Решение: адаптивная система, которая подстраивается под знания и темп каждого пользователя.	

10	Потенциальные потребительские сегменты <i>Анализ рынка с проведенными PEST, SWOT, анализ конкурентов оформляется в приложение к Паспорту</i>	1. Школьники 12–18 лет (подготовка к экзаменам). 2. Студенты (дополнительное образование). 3. Родители (инвестиции в образование детей). 4. Корпоративные клиенты (обучение сотрудников).
11.	Стек технологий с обоснованием	Backend: Python (Django/FastAPI), PostgreSQL. Frontend: React, TypeScript. ИИ: scikit-learn, TensorFlow (рекомендательные системы). Инфраструктура: Docker, GitLab CI/CD, Yandex Cloud. Выбор обусловлен скоростью разработки, масштабируемостью и поддержкой ML-библиотек.
12	На основе какого научно-технического решения и/или результата будет создан продукт (с указанием использования собственных или существующих разработок) <i>Указывается необходимый перечень научно-технических решений с их кратким описанием для создания и выпуска на рынок продукта</i>	Используются алгоритмы машинного обучения для анализа ответов, определения пробелов и построения персональных траекторий. Основано на исследованиях в области адаптивного обучения и когнитивной психологии.
13	Бизнес-модель <i>Указывается кратко описание способа, который планируется использовать для создания ценности и получения прибыли, в том числе, как планируется выстраивать отношения с потребителями и поставщиками, способы привлечения финансовых и иных ресурсов, какие каналы продвижения и сбыта продукта планируется использовать и развивать, и т.д.</i>	Freemium: платная подписка на продвинутые функции (персонализация, углубленная аналитика). Партнёрство со школами и вузами. Каналы продвижения: соцсети, SEO, вебинары.
14	Ключевое ценностное предложение <i>Очевидные преимущества ваших продуктов или услуг</i>	Индивидуальная траектория для каждого ученика, экономия времени на поиск материалов, повышение успеваемости на 30%, удобный интерфейс и детальная аналитика прогресса.

	Характеристика будущего продукта	
7	Основные технические параметры, включая обоснование соответствия идеи/задела тематическому направлению (лоту)* <i>Необходимо привести основные технические параметры продукта, которые обеспечивают их конкурентоспособность и соответствуют выбранному тематическому направлению</i>	• Точность рекомендаций ИИ: >85%. • Время отклика системы: <200 мс. • Поддержка до 10 000 одновременных пользователей. • Интеграция с Zoom/Google Meet для вебинаров. • Мобильная версия (PWA).
15	Финансовый план <i>Приводятся основные экономические показатели устойчивости бизнеса.</i> <i>Финансовый план 0 этапа, 1-4 годов, основные показатели устойчивости бизнеса приводятся в приложении к Паспорту</i>	Прибыль с 3-го квартала, выручка за год – 2230 тыс. руб., чистая прибыль – 2306 тыс. руб. Рентабельность – 103.41%. (Подробно в приложении – Excel-таблица).
16.	Организационный план	Гибкая методология (Scrum), спринты по 2 недели, ежедневные стендапы, использование Jira/Trello для управления задачами.
17.	Производственный план	Этапы: 1. Прототип (3 мес.). 2. MVP (6 мес.). 3. Запуск бета-теста (9 мес.). 4. Масштабирование (12 мес.)

18	Основные конкурентные преимущества <i>Необходимо привести описание наиболее значимых качественных и количественных характеристик продукта, которые обеспечивают конкурентные преимущества в сравнении с существующими аналогами</i>	• Адаптивный ИИ (конкуренты используют статические курсы). • Быстрое обновление контента. • Интеграция с популярными образовательными платформами. • Доступная цена.
19	Уровень готовности продукта TRL <i>Необходимо указать максимально емко и кратко, насколько проработан стартап-проект</i>	Охватывая этапы от фундаментальных исследований (TRL 2)
3	Каналы продвижения будущего продукта <i>Необходимо указать, какую маркетинговую стратегию планируется применять, привести кратко аргументы в пользу выбора тех или иных каналов продвижения</i> <i>Маркетинговая стратегия приводится в приложении к Паспорту</i>	• Таргетированная реклама (ВК, Яндекс.Директ). • SEO и контент-маркетинг (блог, YouTube, RuTube). • Партнёрства с блогерами и педагогами. • Участие в образовательных выставках.
4	Каналы сбыта будущего продукта <i>Указать какие каналы сбыта планируется использовать для реализации продукта и дать кратко обоснование выбора</i>	• Прямые продажи через сайт. • Партнёрские сети (школы, вузы). • Маркетплейсы образовательных услуг.

	Оценка потенциала «рынка» и рентабельности бизнеса <i>Необходимо привести кратко обоснование сегмента и доли рынка, потенциальные возможности для масштабирования бизнеса</i> <i>Оценку рынка и потенциальные возможности масштабирования представить в приложении к Паспорту</i>	Рынок онлайн-образования в России – 120 млрд руб. Потенциальная аудитория – 5 млн школьников и студентов. Цель – захватить 0,5% рынка за 2 года.
	Риски проекта <i>Укажите основные риски проекта и мероприятия по их нивелированию</i> <i>Анализ рисков с описанием механизмов их нивелирования привести в приложении к Паспорту проекта</i>	• Технологические (ошибки ИИ). • Рыночные (выход крупных конкурентов). • Юридические (законодательство в сфере данных). Меры: тестирование, дифференциация, юридический аудит.
	План дальнейшего развития стартап-проекта <i>Укажите, какие шаги будут предприняты в течение 1-3 лет после выхода на рынок, какие меры поддержки планируется привлечь</i>	1 год: запуск MVP, привлечение 1000 пользователей. 2 год: расширение предметной линейки, выход на B2B-сегмент. 3 год: международная экспансия (СНГ).

Сведения о самостоятельности выполнения работы

Работа «Создание набора инструментов и сервисов для разработки веб- и мобильных приложений» выполнена мной самостоятельно.

Используемые в работе материалы и концепция из публикуемой литературы и других источников имеют ссылки на них.

«__» _____ 2025 г. _____